
Ada Web Application Programmer's Guide

Stephane Carrez

2026-03-13

Contents

1	Introduction	7
1.1	System Components	8
1.2	General Purpose Components	9
1.3	Functional Components	9
2	Installation	10
2.1	Before Building	10
2.2	Getting the sources	11
2.3	Development Host Installation	11
2.3.1	Ubuntu	11
2.3.2	FreeBSD 13	12
2.3.3	Windows	12
2.4	Ada Web Server	13
2.5	Configuration	13
2.6	Build	14
2.7	Installation	14
2.8	Using	14
3	Tutorial	15
3.1	The review web application	15
3.2	Setting up the project	17
3.2.1	Project creation with Dynamo	17
3.2.2	Creating the review module with Dynamo	18
3.3	Designing the data model	18
3.3.1	ArgoUML setup	19
3.3.2	Modelize the domain model in UML	19
3.3.3	Adding relations in the UML model	22
3.3.4	Makefile setup	24
3.3.5	Generating the Ada model	24
3.3.6	Creating the database	24
3.4	Adding a creation form	25
3.4.1	Adding pages	25
3.4.2	The create review form	26
3.4.3	How it works	26
3.4.4	The Review_Bean type declaration	28
3.4.5	The Review_Bean implementation	28

3.4.6	The Review_Bean creation	29
3.4.7	Navigation rules	30
3.5	Creating the module	31
3.5.1	Adding the module operations	31
3.5.2	Saving our review	31
3.5.3	Setting up the permissions	32
3.6	Using database queries	34
3.6.1	Adding database queries	34
3.6.2	Implementing the review list bean	37
3.6.3	Review list bean creation	38
3.6.4	Review list bean declaration	39
3.6.5	Listing the reviews: the XHTML facelet presentation file	39
3.6.6	Understanding the request flow	41
4	AWA Core	42
4.1	Initialization	42
4.2	Configuration	42
4.3	AWA Modules	44
4.3.1	Registration	44
4.3.2	Configuration	45
4.4	AWA Permissions	45
4.4.1	Declaration	45
4.4.2	Checking for a permission	46
4.4.3	Configuring a permission	46
4.4.4	Adding a permission	47
4.4.5	Data Model	47
4.4.6	Queries	47
4.4.7	Queries	47
4.5	AWA Events	48
4.5.1	Declaration	48
4.5.2	Posting an event	48
4.5.3	Receiving an event	49
4.5.4	Event queues and dispatchers	49
4.5.5	Data Model	50
5	AWA Commands	50
5.1	Command Usage	50
5.1.1	SYNOPSIS	50

5.1.2	DESCRIPTION	51
5.1.3	OPTIONS	52
5.2	COMMANDS	53
5.2.1	The start command	53
5.2.2	The setup command	54
5.2.3	The stop command	54
5.2.4	The migrate command	54
5.2.5	The list command	55
5.2.6	The info command	55
5.2.7	The permission command	55
5.3	Integration	56
6	Users Module	58
6.1	Integration	58
6.2	OAuth Authentication Flow	58
6.3	Configuration	59
6.4	Ada Beans	63
6.5	Data model	64
7	Jobs Module	65
7.1	Integration	65
7.2	Writing a job	65
7.3	Registering a job	66
7.4	Scheduling a job	66
7.5	Checking for job completion	66
7.6	Job Service	66
7.7	Ada Beans	67
7.8	Data Model	67
8	Mail Module	68
8.1	Integration	68
8.2	Configuration	68
8.3	Sending an email	69
8.4	Components	69
8.4.1	Mail Recipients	70
8.4.2	Mail Messages	70
8.4.3	Mail Attachments	70
8.5	Ada Beans	70

9	Workspaces Module	71
9.1	Integration	71
9.2	Permissions	71
9.3	Configuration	72
9.4	Events	72
9.4.1	invite-user	72
9.4.2	accept-invitation	73
9.5	Ada Beans	73
9.5.1	Beans	73
9.5.2	Permissions	73
9.5.3	Configuration	73
9.6	Data Model	74
10	Storages Module	75
10.1	Integration	75
10.2	Permissions	75
10.3	Configuration	76
10.4	Creating a storage	76
10.5	Getting the data	77
10.6	Local file	77
10.7	Storage Service	77
10.8	Store Service	78
10.8.1	Database store	78
10.8.2	File System store	78
10.9	Ada Beans	78
10.10	Storage Servlet	79
10.11	Queries	79
10.12	Data model	81
11	Images Module	82
11.1	Integration	82
11.2	Configuration	82
11.3	Ada Beans	83
11.4	Queries	84
11.5	Data model	85
12	Wikis Module	86
12.1	Integration	86
12.2	Permissions	86

12.3	Configuration	87
12.4	Events	88
12.5	Ada Beans	88
12.6	Queries	90
12.7	Data model	92
13	Blogs Module	93
13.1	Integration	93
13.2	Permissions	93
13.3	Configuration	94
13.4	Ada Beans	95
13.5	Queries	98
13.6	Data model	100
14	Counters Module	101
14.1	Integration	101
14.2	Configuration	101
14.3	Counter Declaration	102
14.4	Incrementing the counter	103
14.5	Ada Bean	103
14.6	HTML components	105
14.7	Data model	105
15	Votes Module	107
15.1	Integration	107
15.2	Ada Beans	107
15.3	Javascript integration	109
15.4	Data model	109
16	Tags Module	110
16.1	Integration	110
16.2	Ada Beans	110
16.2.1	Tag_List_Bean	110
16.2.2	Tag_Search_Bean	111
16.2.3	Tag_Info_List_Bean	111
16.3	HTML components	112
16.3.1	Displaying a list of tags	112
16.3.2	Tag editing	112
16.3.3	Tag cloud	113

16.4	Queries	113
16.5	Data model	113
17	Comments Module	115
17.1	Integration	115
17.2	Ada Beans	115
17.2.1	Comment_List_Bean	115
17.3	Data model	117
18	Settings Module	118
18.1	Getting a user setting	118
18.2	Saving a user setting	118
18.3	Integration	118
18.4	Data model	119
19	SEO Module	120
19.1	Integration	120
19.2	Configuration	121
19.3	SEO Servlet	121
20	Setup Application	122
20.1	Integration	122
20.2	Setup Procedure Instantiation	122
20.3	Setup Operation	122
21	Tips	124
21.1	UI Presentation Tips	124
21.1.1	Adding a simple page	124
21.1.2	Add Open Graph	124
21.1.3	Formatting dates	125
21.2	Configuration Tips	125
21.2.1	Adding a permission on user creation	125
21.2.2	Secure configuration	126
21.2.3	Overriding some AWA navigation rules	127
21.3	Trouble shooting Tips	127
21.3.1	No AWA service context	127

1 Introduction

Ada Web Application is a framework to build a Web Application in Ada 2012. The framework provides several ready to use and extendable modules that are common to many web applications. This includes the login, authentication, users, permissions, managing comments, tags, votes, documents, images. It provides a complete blog, question and answers and a wiki module.

AWA simplifies the Web Application development by taking care of user management with Google+, Facebook authentication and by providing the foundations on top of which you can construct your own application. AWA provides a powerful permission management that gives flexibility to applications to grant access and protect your user's resources.

A typical architecture of an AWA application is represented by the picture below:

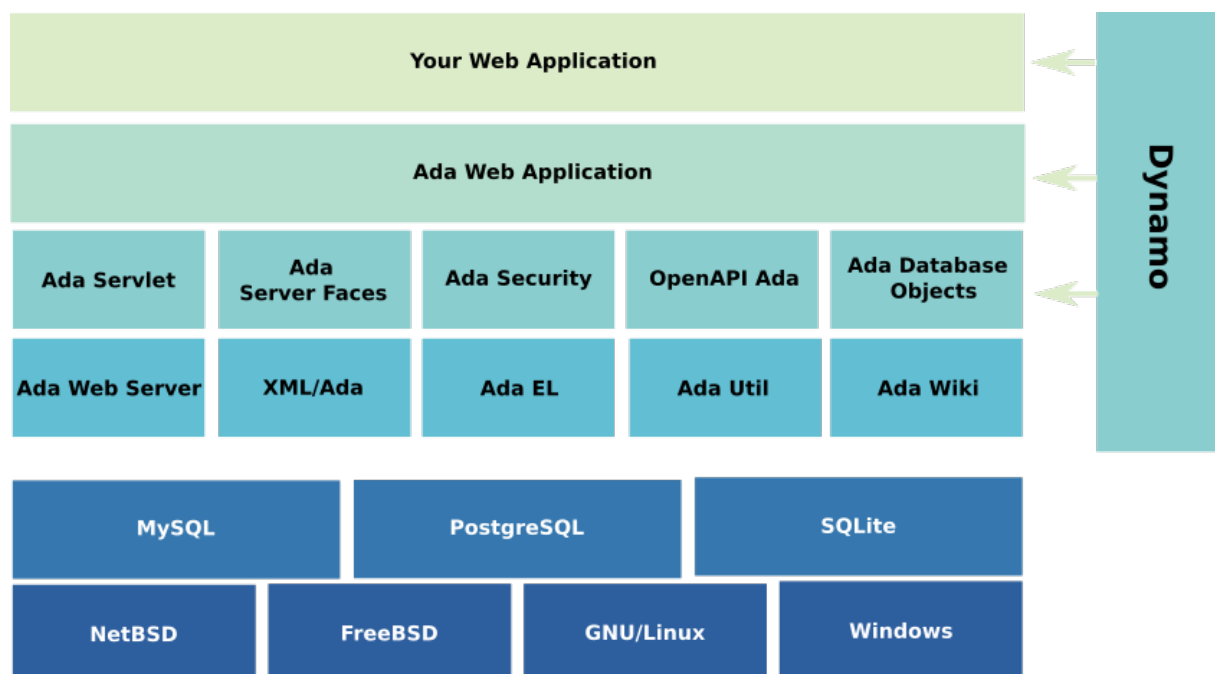


Figure 1: Ada Web Application Architecture

Because your application sits on top of AWA framework, it benefits of all the functionalities that AWA uses for its implementation:

- The Web server is built on top of the Ada Web Server library,
- The presentation layer is using Ada Server Faces which allows to use the same design pattern as the Java Server Faces,
- The database access is provided by Ada Database Objects

Apart from this architecture, the Dynamo tool is used to generate code automatically and help starting the project quickly.

AWA is composed of several configuration components also called modules or plugins. Components are classified in three categories:

- System components,
- General purpose components,
- Functional components.

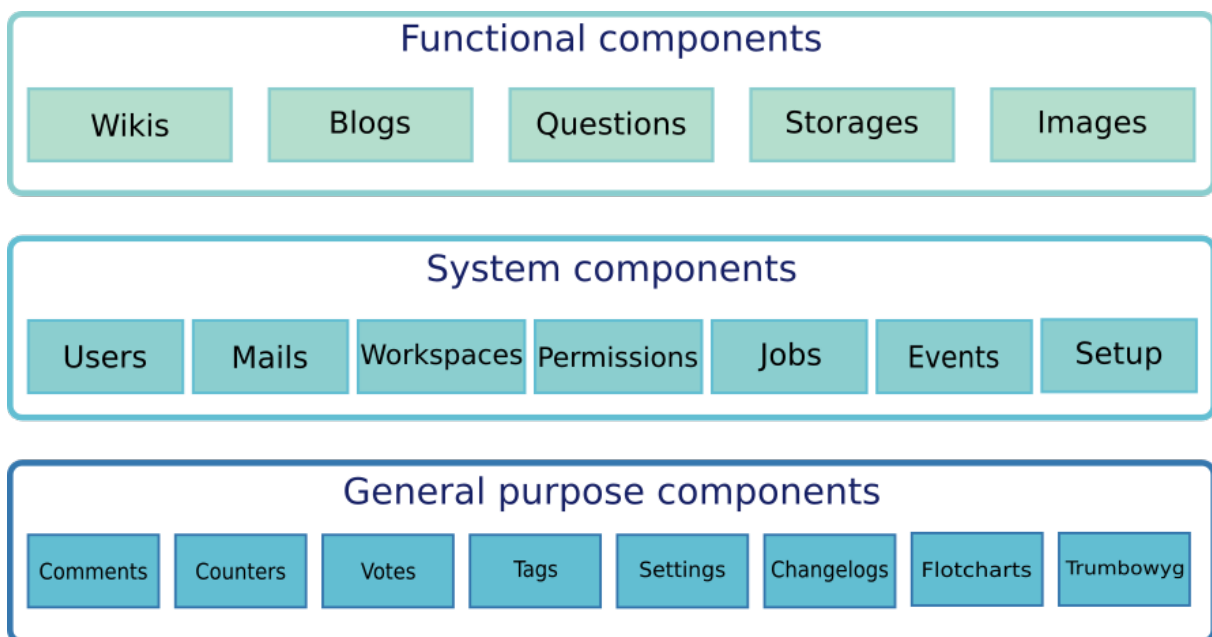


Figure 2: AWA Features

1.1 System Components

The **System Components** represent the core components onto which all other components are based. These component don't provide any real functionality for a final user but they are necessary for the Web application to operate. These components include:

- The Users Module manages the creation, update, removal and authentication of users,
- The Mail Module allows an application to format and send a mail,
- The Jobs Module provides a batch job framework for modules to execute long running actions,
- The Events Module implements an eventing system to share events with other modules,
- The Workspace Module defines a workspace area for other plugins to connect and plug into.

1.2 General Purpose Components

The **General Purpose Components** are components which provide generic functionalities that can be plugged and used by functional components.

- The Tags Module allows to associate general purpose tags to any database entity,
- The Votes Module allows users to vote for objects defined in the application,
- The Comments Module is a general purpose module that allows to associate user comments to any database entity,
- The Counters Module defines a general purpose counter service allowing to associate counters to database entities,
- The Changelogs Module associates logs produced by users to any database entity.
- The SEO Module provides tools for search engine optimization (sitemap generation).

1.3 Functional Components

The **Functional Components** implement a final functionality for a user. They are using the system components such as User Module for the user management but also general purpose components such as Tags Module or Counters Module.

- The Questions Module is a simple question and answer system,
- The Blogs Module is a small blog application which allows users to publish articles,
- The Wikis Module provides a complete Wiki system allowing users to create their own Wiki environment.

To help in the installation process of final applications, the Setup Application is a special component that you can decide to customize to provide an installation and configuration process to your own application.

2 Installation

This chapter explains how to build and install the Ada Web Application framework.

2.1 Before Building

Before building the framework, you will need:

- The GNAT Ada compiler,
- Either the MySQL, PostgreSQL or SQLite development headers installed,
- XML/Ada,
- Ada Web Server.

First get, build and install the above tools and libraries. For the best experience, it is necessary to have the SSL support in Ada Web Server. Indeed, the OpenID Authentication 2.0 can only be used through HTTPS.

The build process may also need the following commands:

- make (GNU make),
- gprbuild,
- gprinstall,
- unzip,
- sqlite3,
- mysql,
- psql,
- xsltproc,
- liblzma libraries (used by Ada LZMA),
- CURL libraries (used by CURL support in Ada Utility Library)

The Ada Web Application library also uses the following projects:

- Ada LZMA,
- Ada Utility Library,
- Ada Expression Language Library,
- Ada Security Library,
- Ada Servlet Library,
- Ada Server Faces Library,
- Ada Wiki Library,
- Ada Database Objects Library,

- Ada Keystore Library,
- OpenAPI Ada Library,
- Dynamo

They are integrated as Git submodules.

2.2 Getting the sources

The AWA framework uses git submodules to integrate several other projects. To get all the sources, use the following commands:

```
1 git clone --recursive git@github.com:stcarrez/ada-awa.git
2 cd ada-awa
```

2.3 Development Host Installation

The PostgreSQL, MySQL and SQLite development headers and runtime are necessary for building the Ada Database Objects driver. The configure script will use them to enable the ADO drivers. The configure script will fail if it does not find any database driver.

2.3.1 Ubuntu

First to get the LZMA and CURL support, it is necessary to install the following packages before configuring AWA:

```
1 sudo apt-get install liblzma-dev libcurl4-openssl-dev
```

MySQL Development installation

```
1 sudo apt-get install libmysqlclient-dev
```

MariaDB Development installation

```
1 sudo apt-get install mariadb-client libmariadb-client-lgpl-dev
```

SQLite Development installation

```
1 sudo apt-get install libsqlite3-dev
```

PostgreSQL Development installation

```
1 sudo apt-get install postgresql-client libpq-dev
```

2.3.2 FreeBSD 13

First to get the LZMA, XML/Ada and CURL support, it is necessary to install the following packages before configuring AWA:

```
1 pkg install gnat12 gprbuild lzma curl-7.87.0 xmlada
```

MariaDB Development installation:

```
1 pkg install mariadb106-client mariadb106-client-10.4.7 mariadb106-  
server-10.6.11
```

SQLite Development installation:

```
1 pkg install sqlite3
```

PostgreSQL Development installation:

```
1 pkg install postgresql15-client-15.1 postgresql15-server-15.1_1
```

2.3.3 Windows

It is recommended to use msys2 available at <https://www.msys2.org/> and use the `pacman` command to install the required packages.

```
1 pacman -S git  
2 pacman -S make  
3 pacman -S unzip  
4 pacman -S base-devel --needed  
5 pacman -S mingw-w64-x86_64-sqlite3
```

For Windows, the installation is a little bit more complex and manual. You may either download the files from MySQL and SQLite download sites or you may use the files provided by Ada Database Objects and Ada LZMA in the `win32` directory.

For Windows 32-bit, extract the files:

```
1 cd ada-ado/win32 && unzip sqlite-dll-win32-x86-3290000.zip  
2 cd ada-lzma/win32 && unzip liblzma-win32-x86-5.2.4.zip
```

For Windows 64-bit, extract the files:

```
1 cd ada-ado/win32 && unzip sqlite-dll-win64-x64-3290000.zip  
2 cd ada-lzma/win32 && unzip liblzma-win64-x64-5.2.4.zip
```

If your GNAT 2019 compiler is installed in `C : /GNAT /2019`, you may install the liblzma, MySQL and SQLite libraries by using msys `cp` with:

```
1 cp ada-lzma/win32/*.dll C:/GNAT/2019/bin
2 cp ada-lzma/win32/*.dll C:/GNAT/2019/lib
3 cp ada-lzma/win32/*.a C:/GNAT/2019/lib
4 cp ada-ado/win32/*.dll C:/GNAT/2019/bin
5 cp ada-ado/win32/*.dll C:/GNAT/2019/lib
6 cp ada-ado/win32/*.lib C:/GNAT/2019/lib
7 cp ada-ado/win32/*.a C:/GNAT/2019/lib
```

2.4 Ada Web Server

The Ada Web Server should be compiled with the SSL support if you want to use the OAuth 2.0 protocol and integrate with Google, GitHub and GitLab authentication systems. The AWS version shipped with GNAT 2019 and GNAT 2020 will not work because it does not support SSL.

You may build AWS by using:

```
1 git clone --recursive -b 20.2 https://github.com/AdaCore/aws
2 cd aws
3 make SOCKET=openssl setup build install
```

2.5 Configuration

The library uses the `configure` script to detect the build environment, check for Ada Utility Library library. The `configure` script provides several standard options and you may use:

- `--prefix=DIR` to control the installation directory,
- `--enable-shared` to enable the build of shared libraries,
- `--disable-static` to disable the build of static libraries,
- `--enable-distrib` to build for a distribution and strip symbols,
- `--disable-distrib` to build with debugging support,
- `--enable-coverage` to build with code coverage support (`-fprofile-arcs -ftest-coverage`),
- `--with-aws=PATH` to control the installation path of Ada Web Server,
- `--with-xmlada=PATH` to control the installation path of XML/Ada,
- `--help` to get a detailed list of supported options.

In most cases you will configure with the following command:

```
1 ./configure
```

By default, the framework will be installed in `/usr/local` directory. If you want to install the framework in a specific directory, use the `--prefix` option as follows:

```
1 ./configure --prefix=/opt/install-awa
```

2.6 Build

After configuration is successful, you can build the library by running:

```
1 make
```

2.7 Installation

The installation is done by running the `install` target:

```
1 make install
```

2.8 Using

To use the library in an Ada project, add the following line at the beginning of your GNAT project file:

```
1 with "awa";
```

Depending on your application, you may also need to add the following GNAT projects which are provided by one or several of the libraries that Ada Web Application relies on:

```
1 with "utilada";
2 with "elada";
3 with "security";
4 with "servletada";
5 with "servletada_aws";
6 with "asf";
7 with "ado_mysql";
8 with "ado_sqlite";
9 with "ado_postgresql";
```

The library comes with several optional modules that you decide to use according to your needs. When you decide to use a module, you should add the GNAT project that corresponds to the module you wish to integrate. For example, to use the `Jobs` and `Wikis` modules, you will need the following lines in your GNAT project:

```
1 with "awa_jobs";
2 with "awa_wikis";
```

3 Tutorial

Ada Web Application is a complete framework that allows to write web applications using the Ada language. Through a complete web application, the tutorial explains various aspects in setting up and building an application by using AWA.

The tutorial assumes that you have already installed the following software on your computer:

- The GNAT Ada compiler,
- The Ada Web Application framework and its associated dependencies (XML/Ada and AWS),
- The Dynamo code generator.

The ArgoUML modelization tool is provided by the Dynamo package. Since this is a Java application, it uses the Java JRE (either 1.8 or 1.11, the OpenJDK 1.11 is recommended).

3.1 The review web application

The review web application allows users to write reviews about a product, a software or a web site and share them to the Internet community. The community can read the review, participate by adding comments and voting for the reviewed product or software.

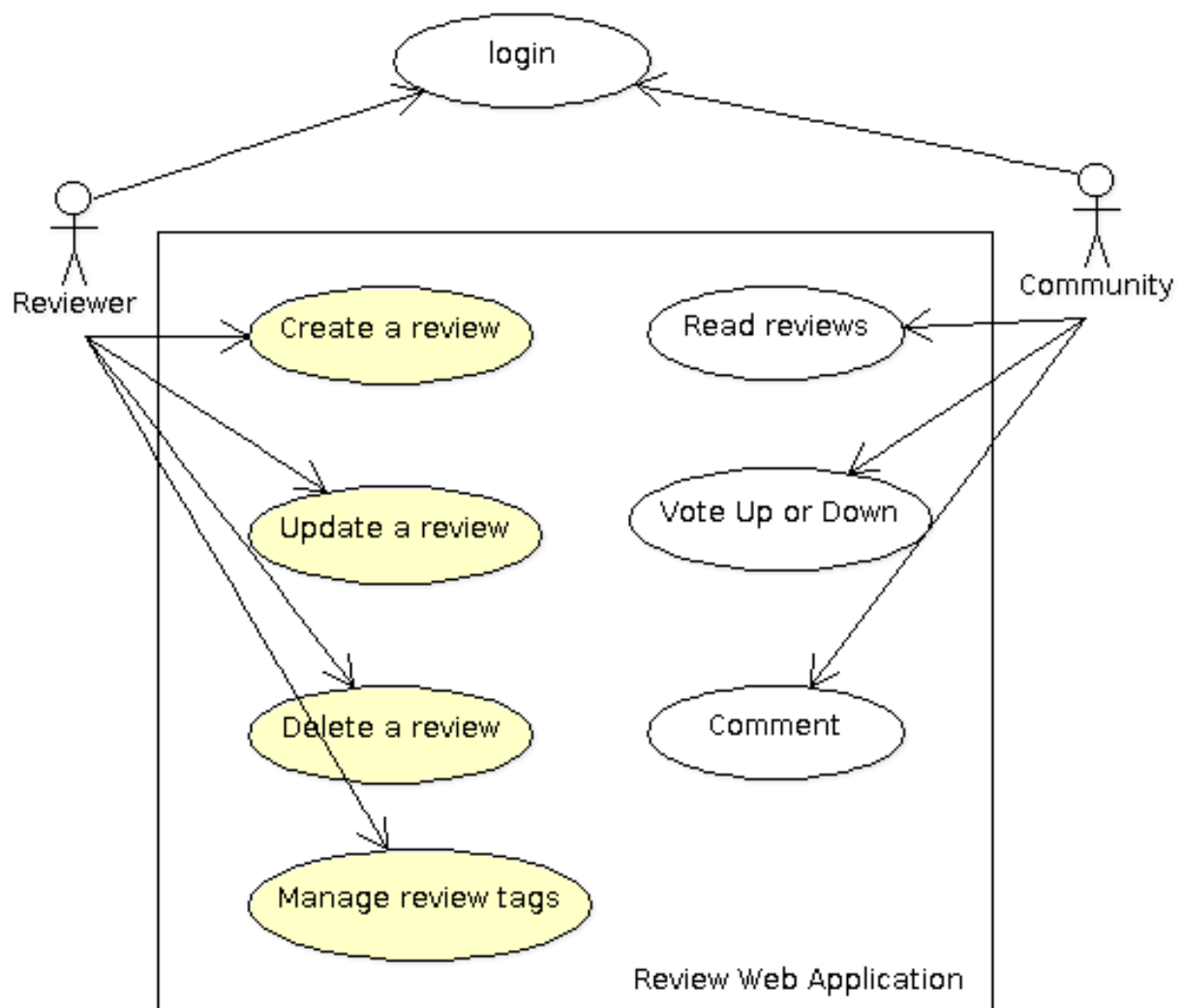


Figure 3: Review Web Application Use Cases

The AWA framework provides several modules that are ready to be used by our application. The login and user management is handled by the framework so this simplifies a lot the design of our application. We will see in the tutorial how we can leverage this to our review application.

Because users of our review web application have different roles, we will need permissions to make sure that only reviewers can modify a review. We will see how the AWA framework leverages the Ada Security library to enforce the permissions.

The AWA framework also integrates three other modules that we are going to use: the Tags Module, the Votes Module and the Comments Module.

Since many building blocks are already provided by the AWA framework, we will be able to concentrate

on our own review application module.

3.2 Setting up the project

3.2.1 Project creation with Dynamo

The first step is to create the new project. Since creating a project from scratch is never easy we will use the Dynamo tool to build our initial review web application. Dynamo is a command line tool that provides several commands that help in several development tasks. For the project creation we will give:

- the output directory,
- the project name,
- the license to be used for the project,
- the project author's email address.

Choose the project name with care as it defines the name of the Ada root package that will be used by the project. For the license, you have the choice between GPL v2, GPL v3, MIT, BSD 3 clauses, Apache 2 or some proprietary license.

```
1 dynamo -o atlas create-project -l apache atlas email@domain.com
```

The Dynamo project creation will build the `atlas` directory and populate it with many files:

- A set of configure, Makefile, GNAT project files to build the project,
- A set of Ada files to build your Ada web application,
- A set of presentation files for the web application.

Once the project is created, we must configure it to find the Ada compiler, libraries and so on. This is done by the following commands:

```
1 cd atlas
2 ./configure
```

At this step, you may even build your new project and start it. The `make` command will build the Ada files and create the `bin/atlas-server` executable that represents the web application.

```
1 make generate build
2 bin/atlas-server start
```

Once the server is started, you may point your browser to the following location:

```
1 http://localhost:8080/atlas/index.html
```

3.2.2 Creating the review module with Dynamo

With the Ada Web Application framework, a web application is composed of modules where each module brings a specific functionality to the application. AWA provides a module for user management, another for comments, tags, votes, and many others. The application can decide to use these modules or not. The AWA module framework helps in defining the architecture and designing your web application.

For the review web application we will create our own module dedicated for the review management. The module will be an Ada child package of our root project package. From the Ada point of view, the final module will be composed of the following packages:

- A **Modules** package represents the business logic of the module. It provides operations to access and manage the data owned by the module.
- A **Beans** package holds the Ada beans that make the link between the presentation layer and business logic.
- A **Models** package holds the data model to access the database content. This package is generated from UML and will be covered by a next tutorial.

To help in setting up a new AWA module, the Dynamo tool provides the `add-module` command. You just have to give the name of the module, which is the name of the Ada child package. Let's create our `reviews` module now:

```
1 dynamo add-module reviews
```

The command generates the new AWA module and modifies some existing files to register the new module in the application. You can build your web application at this stage even though the new module will not do anything yet for you.

3.3 Designing the data model

Our review web application will need to access a database to store the review information. For this, we must define a data model that will describe how the information is stored in the database and how we can access such information from Ada.

A Model Driven Engineering or MDE promotes the use of models to ease the development of software and systems. The Unified Modeling Language is used to modelize various parts of the software. UML is a graphical type modelling language and it has many diagrams but we are only going to use one of them: the Class Diagram.

The class diagram is probably the most powerful diagram to design, explain and share the data model of any application. It defines the most important data types used by an application with the relation

they have with each other. In the class diagram, a class represents an abstraction that encapsulates data member attributes and operations. The class may have relations with others classes.

3.3.1 ArgoUML setup

When using a UML modelization, two specific profiles must be configured before using the tool:

- The Dynamo profile that describes the base data types for our UML model. These types are necessary for the code generator to work correctly.
- The AWA profile that describes the tables and modules provided by AWA. We will need it to get the user UML class definition.

These UML profiles are located in the `/usr/share/dynamo/base/uml` directory after Dynamo and AWA are installed.

When you use the `argouml` command provided by the Dynamo package, the UML profiles should already be accessible and you have nothing to setup.

If you have installed ArgoUML by downloading it, you must setup these UML profiles. To configure ArgoUML, go in the **Edit** -> **Settings** menu and add the directory in the **Default XMI directories** list. Beware that you must restart ArgoUML to be able to use the new profiles.

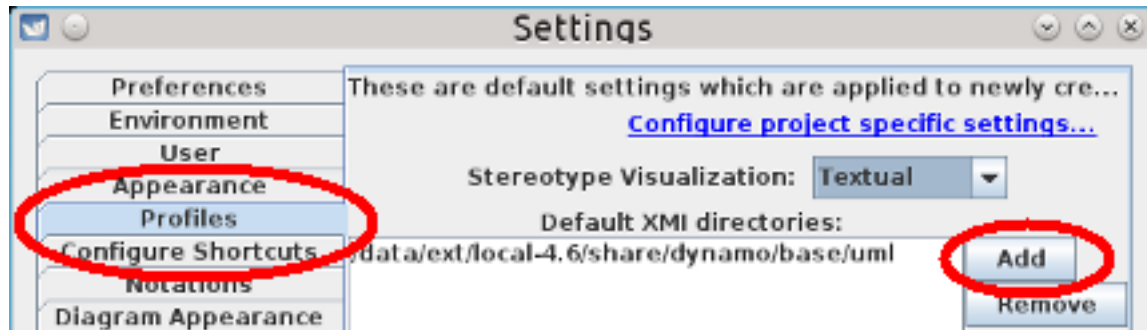


Figure 4: Setting ArgoUML profiles

Once the directory is added, restart ArgoUML, go again in **Edit** -> **Settings** menu and select the **AWA.xmi** and **Dynamo.xmi** profiles. As soon as they are selected and applied on the configuration, you should restart ArgoUML another time for these two profiles to become usable.

3.3.2 Modelize the domain model in UML

The UML model must use a number of Dynamo artifacts for the code generation to work properly. The artifact describes some capabilities and behavior for the code generator to perform its work. Stereotype

names are enclosed within << and >> markers. Dynamo uses the following stereotypes:

- The `DataModel` stereotype must be applied on the package which contains the model to generate. This stereotype activates the code generation (other packages are not generated).
- The `Table` stereotype must be applied to the class. It controls which database table and Ada type will be generated.
- The `PK` stereotype must be defined in at most one attribute of the class. This indicates the primary key for the database table. The attribute type must be an integer or a string. This is a limitation of the Ada code generator.
- The `Version` stereotype must be applied on the attribute that is used for the optimistic locking implementation of the database layer.
- The `AuditTable` stereotype can be applied to some attributes and relations when you want to audit changes to these attributes or relations. When used, the ADO framework will track changes and automatically record them in a specific auditing table.

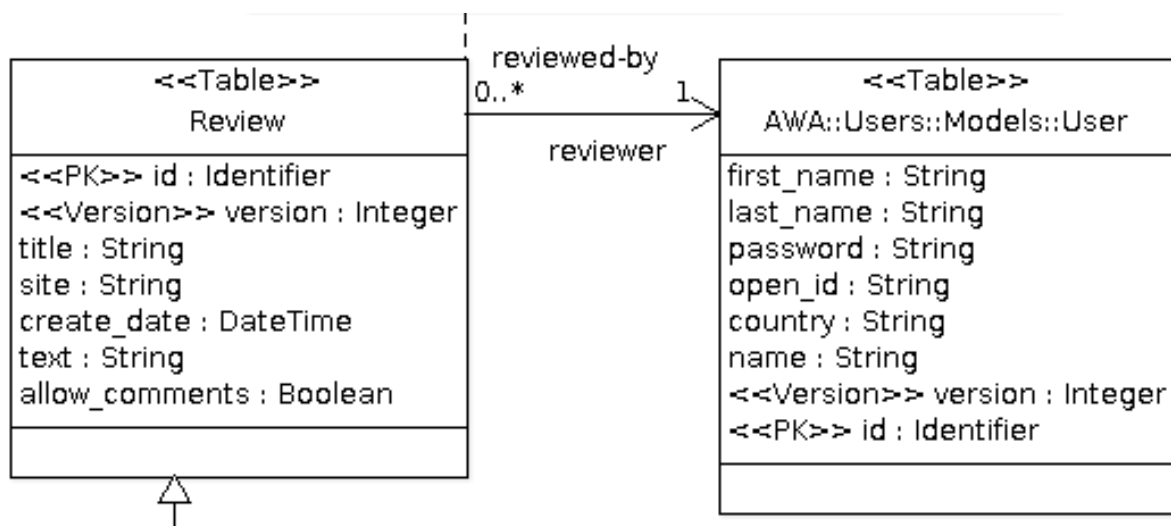


Figure 5: The Review Table UML Model

In our UML model, the `Review` table is assigned the `Table` stereotype so that an SQL table will be created as well as an Ada tagged type to represent our table. The `id` class attribute represents the primary key and thus has the `PK` stereotype. The `version` class attribute is the database column used by the optimistic locking implementation provided by Ada Database Objects. This is why it has the `Version` stereotype. The `title`, `site`, `create_date`, `text` and `allow_comments` attributes represent the information we want to store in the database table. They are general purpose attributes and thus don't need any specific stereotype. For each attribute, the Dynamo code generator will generate a getter and a setter operation that can be used in the Ada code.

To tune the generation, several UML tagged values can be selected and added on the table or on a table attribute. By applying a stereotype to the class, several tagged values can be added. By selecting the **Tagged Values** tab in ArgoUML we can edit and setup new values. For the **Review** table, the `dynamo.table.name` tagged value defines the name of the SQL database table, in our case `atlas_review`.

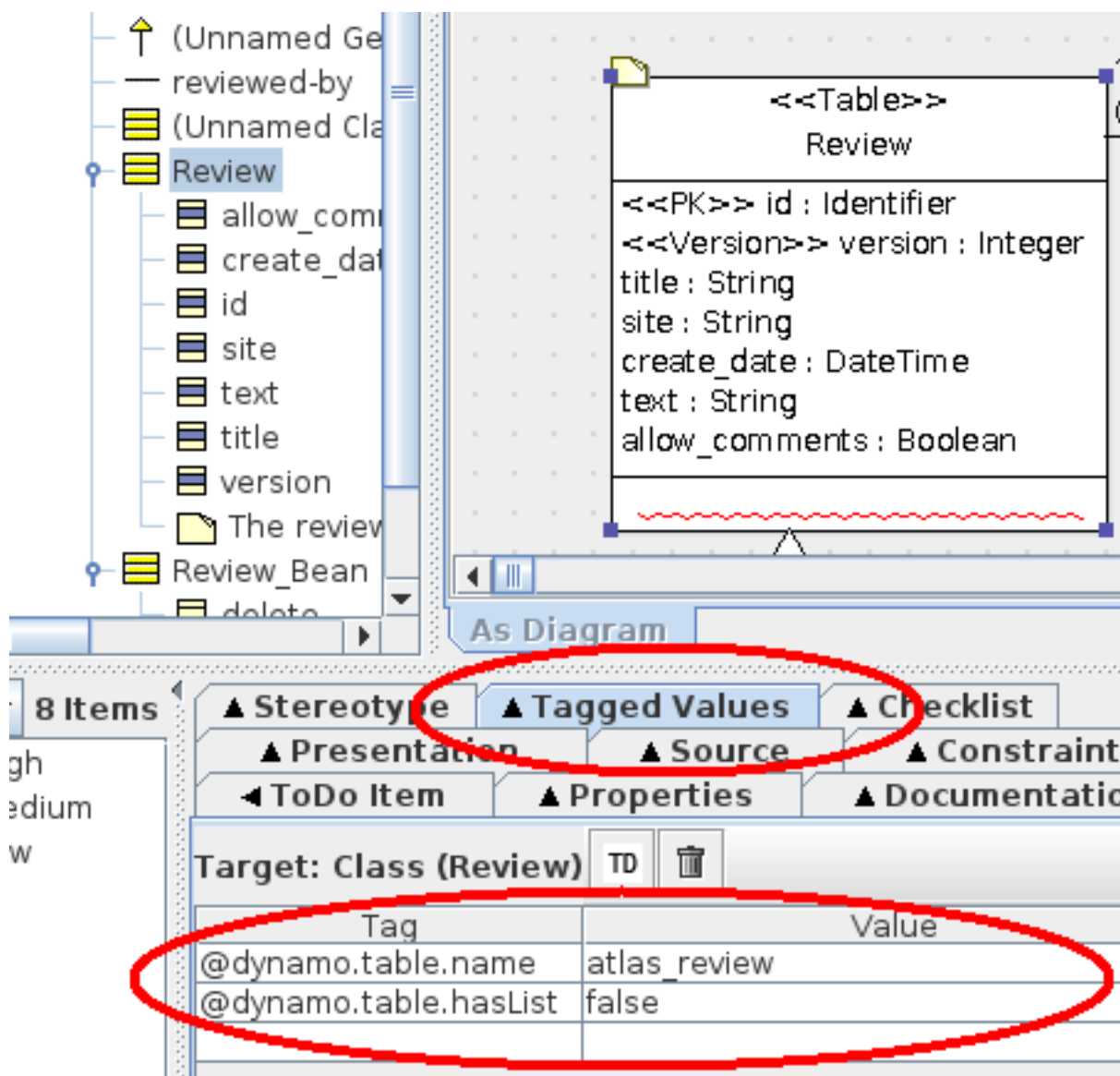


Figure 6: The tagged value for the Review table

The `text` attribute in the `Review` table is a string that can hold some pretty long text. To control the length of the SQL column, we can set the `dynamo.sql.length` tagged value and tell what is that length.

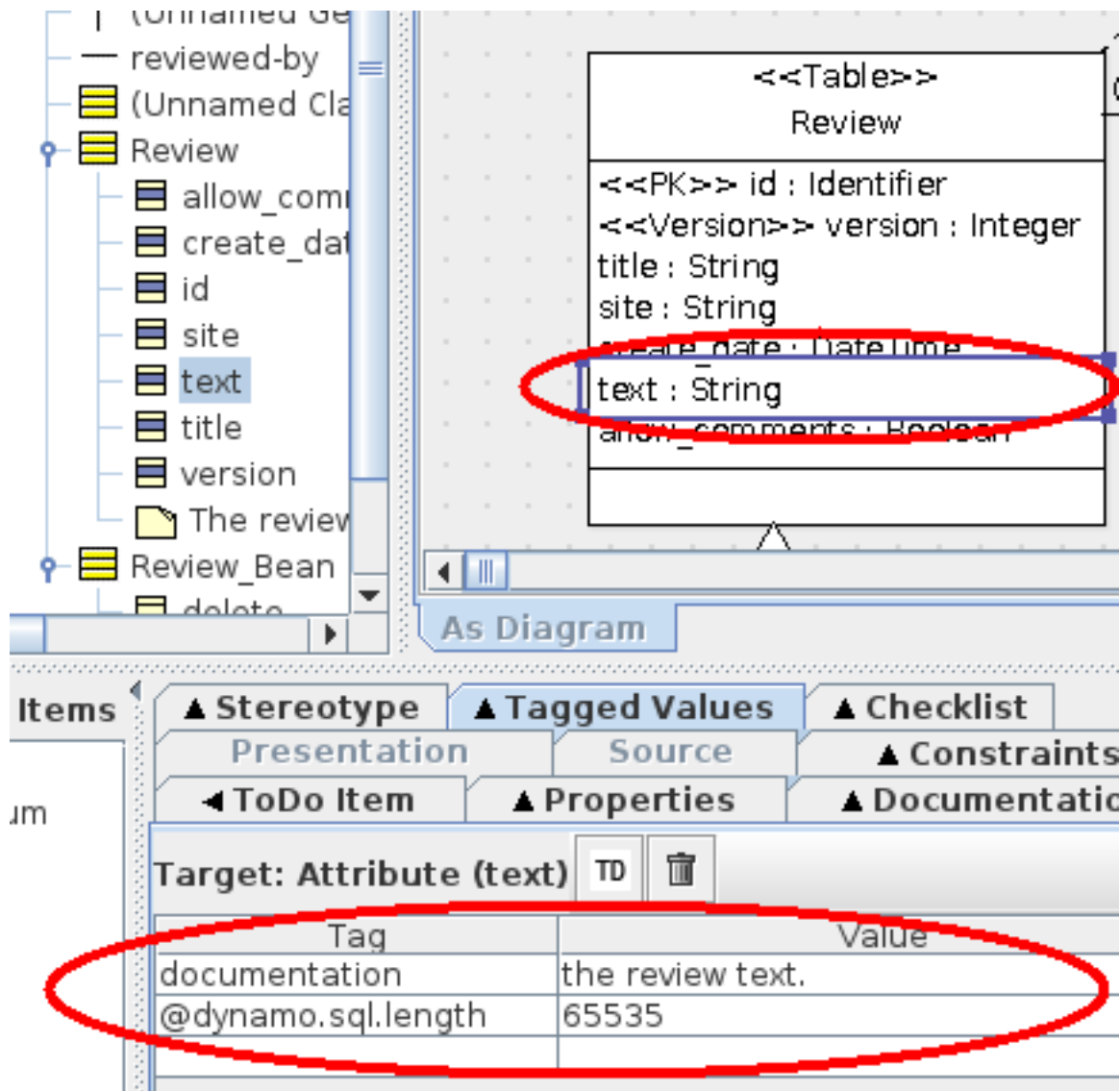


Figure 7: The tagged value for the text column in review table

Once the UML model is designed, it is saved in the project directory `uml`. Dynamo will be able to read the ArgoUML file format (`.zargo` extension) so there is no need to export the UML in XML.

3.3.3 Adding relations in the UML model

The final UML model of our review application is fairly simple. We just added a table and a bean declaration. To benefit from the user management in AWA, we can use the `AWA::Users::Models::User` class that is defined in the AWA UML model. The `reviewed-by` association will create an attribute

`reviewer` in our class. The code generator will generate a `Get_Reviewer` and `Set_Reviewer` operation in the Ada code. The SQL table will contain an additional column `reviewer` that will hold the primary key of the reviewer.

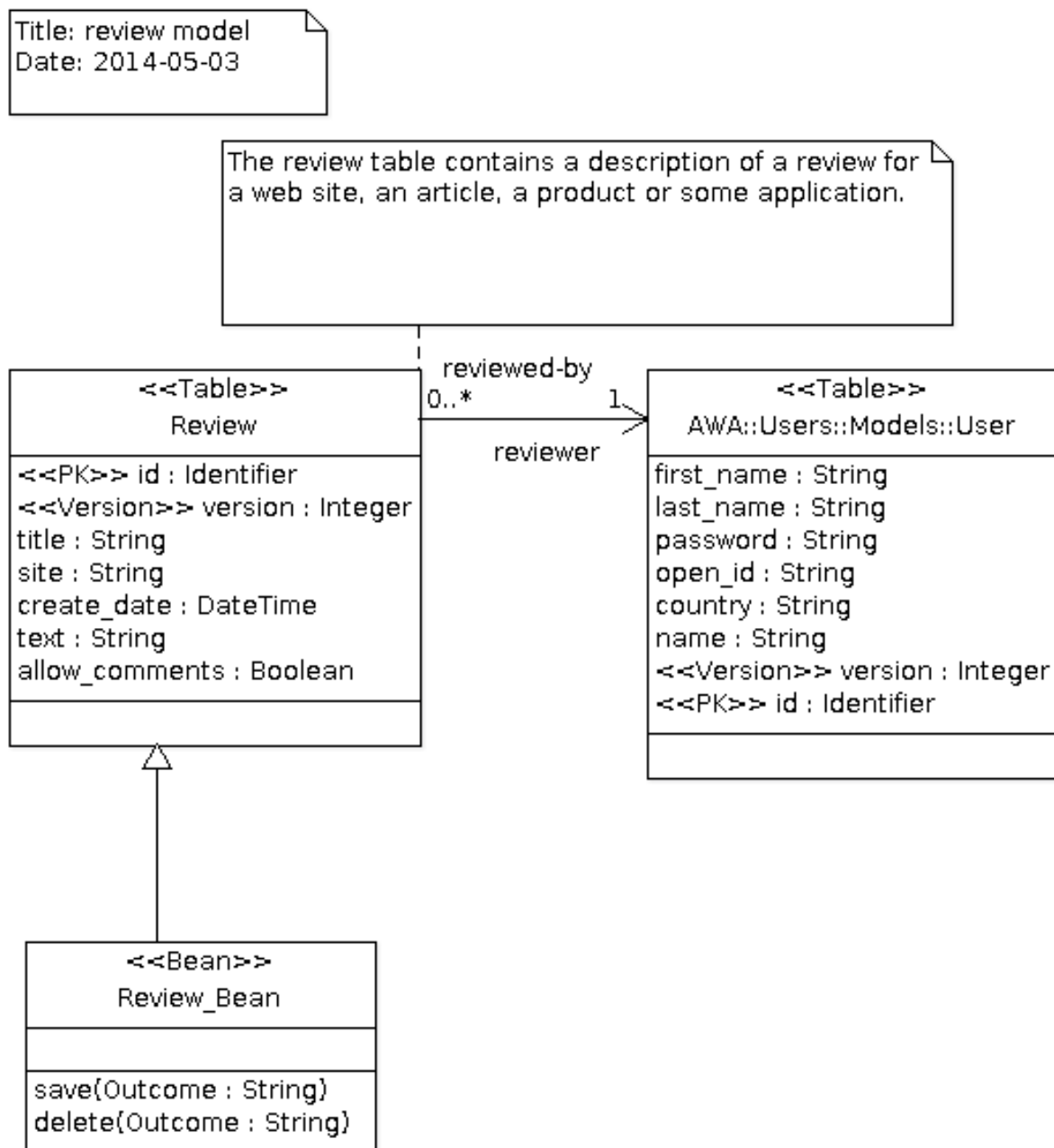


Figure 8: The Review Web Application UML Model

The `Review_Bean` class is an Ada Bean abstract class that will be generated by the code generator.

The `Bean` stereotype activates the bean code generator and the generator will generate some code support that is necessary to turn the `Review_Bean` tagged record into an Ada Bean aware type. We will see in the section that we will only have to implement the `save` and `delete` operation that are described in this UML model.

3.3.4 Makefile setup

The `Makefile` that was generated by the Dynamo `create-project` command must be updated to setup a number of generation arguments for the UML to Ada code generator. Edit the `Makefile` to change `DYNAMO_ARGS` into:

```
1 DYNAMO_ARGS=--package Atlas.Reviews.Models db uml/atlas.zargo
```

The `--package` option tells Dynamo to generate only the model for the specified package. The `db` directory is the directory that will contain the SQL model files.

3.3.5 Generating the Ada model

To run the generator, we can use the `generate` make target:

```
1 make generate
```

The Dynamo code generator reads the file `uml/atlas.zargo` and the UML model it contains and generates:

- the Ada package `Atlas.Reviews.Models` which contains the definition of the `Review` table. The model files are created in the directory `src/models` which is separate from your Ada sources. You can safely remove the files in `src/models` and have them re-built by using Dynamo. It is not recommended to modify these files.
- the SQL files to create the MySQL or SQLite database. Depending on the AWA modules which are used, the generated SQL files will contain additional tables that are used by the AWA modules. The SQL files are generated in the `db/mysql`, `db/sqlite` and `db/postgresql` directories.

3.3.6 Creating the database

Until now we designed our application UML model, we have our Ada code generated, but we need a database with the tables for our application. We can do this by using the `create-database` command in Dynamo. This command needs several arguments:

- The directory that contains the SQL model files. In our case, this is `db`.

- The information to connect to the database, the database name, the user and its password. This information is passed in the form of a database connection string.
- The name of the database administration account to connect to the server and create the new database.
- The optional password for the database administration account.

If the MySQL server is running on your host and the admin account does not have any password, you can use the following command:

```
1 dynamo create-database \  
2 db 'mysql://localhost/demo_atlas?user=demo&password=demo' root
```

The `create-database` creates the database (`demo_atlas`) with the tables that are necessary for the application. It also creates the `demo` user and give it the necessary MySQL grants to connect to the `demo_atlas` database.

3.4 Adding a creation form

We will start with the presentation layer by adding two pages in our web application. A first page will contain the list of reviews and the second page will contain a form to create or update a review.

AWA uses the Facelets technology to allow developers write and design the presentation layer of the web application. This technology is commonly used in J2EE applications. A page is represented by an XML file that contains HTML code, includes some stylesheets, Javascript files and makes the link between the presentation and the web application.

3.4.1 Adding pages

Dynamo provides at least two commands that help in adding presentation files. The `add-page` command adds a simple page that can be edited and filled with real content. We will use it for the creation of the page to display the list of reviews.

```
1 dynamo add-page reviews/list
```

The `add-form` command creates another template of page that includes an HTML form to let a user submit some data to the web application.

```
1 dynamo add-form reviews/edit-review
```

These two commands will create the following files and they can now be modified.

```
1 ./web/reviews/list.xhtml
```

```
2 ./web/reviews/edit-review.xhtml
3 ./web/reviews/forms/edit-review-form.xhtml
```

3.4.2 The create review form

In Facelets, an HTML form is created by using the `<h:form>` component from the HTML JSF namespace. This component will generate the HTML `form` tag and it will also manage the form submission.

The Ada Server Faces provides a set of widget components that facilitate the design of web application. The `<w:inputText>` component renders a title field with an HTML `<label>` and an HTML `<input>` text. We will use it to let the user enter the review title and the site URL being reviewed. The HTML `<textarea>` is provided by the JSF component `<h:inputTextArea>`. The review submit form is defined by the following XML extract:

```
1 <h:form xmlns:h="http://java.sun.com/jsf/html"
2   xmlns:w="http://code.google.com/p/ada-asf/widget">
3   <h:inputHidden id='entity-id' value '#{review.id}' required='false' />
4   <w:inputText title='Title' value '#{review.title}' />
5   <w:inputText title='Site' value '#{review.site}' />
6   <h:inputTextArea rows='20' value '#{review.text}' />
7   <h:commandButton value='Save'
8     action= '#{review.save}' />
9 </h:form>
```

Before closing the `<h:form>` component, we will put a `<h:commandButton>` that will render the form submit button.

3.4.3 How it works

Before going further, let's see how all this works. The principle below is exactly the same for a Java Server Faces application.

First, when the page is rendered the UEL expressions that it contains are evaluated. The `#{review.title}`, `#{review.site}` and `#{review.text}` are replaced by the content provided by the `review` object which is an Ada Bean provided by the `Review_Bean` tagged record.

When the page is submitted by the user, the input values submitted in the form are saved in the `review` bean, again by using the UEL expression. The `<h:commandButton>` action is then executed. This is also an UEL that indicates a method to invoke on the bean.

To sum up, the UEL makes the binding between the presentation layer in Facelets files and the Ada or Java beans.

The Ada Bean layer provides getter and setter to allow the UEL to retrieve and set values. For this, the `Review_Bean` tagged record implements two operations that are defined in the `[Bean]` (<https://github.com/stcarrez/ada-util/source/browse/trunk/src/util-beans-basic.ads>) interface:

```

1 overriding
2 function Get_Value (From : in Review_Bean;
3                     Name : in String) return Util.Beans.Objects.Object;
4
5 overriding
6 procedure Set_Value (From : in out Review_Bean;
7                     Name : in String;
8                     Value : in Util.Beans.Objects.Object);

```

The `Get_Value` operation is called to retrieve one of the Ada Bean member attribute and the `Set_Value` operation is called during form submission to set the member attribute.

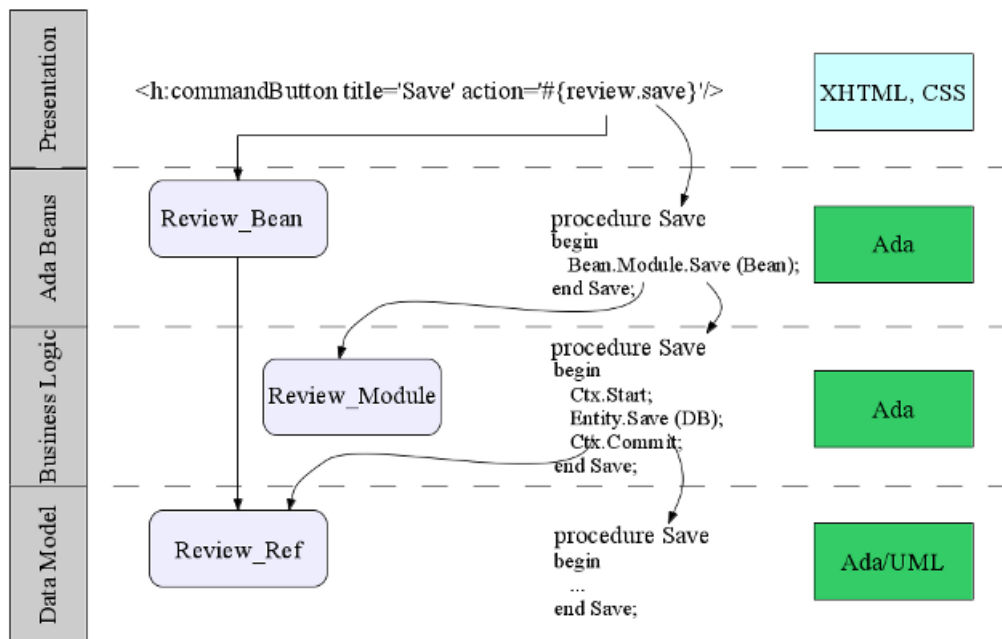


Figure 9: Presentation, Ada Beans and Module interactions

Then the form button is pressed, the HTML form is submitted and received by the server. The `<h:form>` component identifies the form submission and each input component will validate the input fields. When everything has been validated, the `<h:commandButton>` component invokes the `Save` procedure that is declared as follows in the `Review_Bean` tagged record:

```

1 overriding
2 procedure Save (Bean : in out Review_Bean;

```

```
3 Outcome : in out Ada.Strings.Unbounded.Unbounded_String  
    );
```

In the Ada Bean layer, we have to call the business logic to perform the `save` operation.

The business logic part is provided by the Ada module whose initial skeleton was generated by Dynamo. That layer is responsible for defining how the data is created, retrieved and modified. As far as we are concerned, this is rather simple since we only have to verify the permission and save the review object within some transaction. In other modules, several objects may be envolved and more complex rules may be defined for the integrity and validity of these objects.

The last part of the architecture is the data model layer that was in fact generated by Dynamo from the UML model. It is responsible for loading and saving Ada objects into the database.

3.4.4 The Review_Bean type declaration

When we designed our UML model, we have created the `Review_Bean` UML class and gave that class the `Bean` stereotype. We also declared two operations (`save` and `delete`) on that class. With this definition, Dynamo has generated in the `Atlas.Reviews.Models` package the `Review_Bean` abstract type. This type is abstract because we have to implement the `Save` and `Delete` operations. These are the two operations that can be called by an action such as used by the `<h:commandButton>` component.

The `Atlas.Reviews.Models` package is a generated package and it must not be modified. To implement our Ada Bean, we will add the `Review_Bean` type in our own package: the `Atlas.Reviews.Beans` package.

For this the `Review_Bean` type will inherit from the `Atlas.Reviews.Models.Review_Bean` type and it will implement the required operations. The type declaration looks like this:

```
1 package Atlas.Reviews.Beans is  
2 ...  
3 type Review_Bean is new Atlas.Reviews.Models.Review_Bean with record  
4   Module : Atlas.Reviews.Modules.Review_Module_Access := null;  
5 end record;  
6 ...
```

3.4.5 The Review_Bean implementation

The `Save` and `Delete` procedure must be implemented and since the whole business logic is managed by the module layer, we just have to call the associated module procedure as follows:

```
1 overriding
2 procedure Save (Bean : in out Review_Bean;
3               Outcome : in out Ada.Strings.Unbounded.Unbounded_String
4               );
5   begin
6     Bean.Module.Save (Bean);
7   end Save;
8 overriding
9 procedure Delete (Bean : in out Review_Bean;
10                Outcome : in out Ada.Strings.Unbounded.Unbounded_String
11                );
12   begin
13     Bean.Module.Delete (Bean);
14   end Delete;
```

3.4.6 The Review_Bean creation

The AWA framework must be able to create the `review` bean instance when a page is processed. For this, there are three steps that are necessary:

- we must define a create function whose role is to allocate the `Review_Bean` instance and return it. At the same time, the function can setup some pre-defined values for the object. The Dynamo tool has generated for us an example of such function so that there is nothing to do.

```
1 function Create_Review_Bean (Module : in Atlas.Reviews.Modules.
2   Review_Module_Access)
3   return Util.Beans.Basic.ReadOnly_Bean_Access is
4   Object : constant Review_Bean_Access := new Review_Bean;
5   begin
6     Object.Module := Module;
7     return Object.all'Access;
8   end Create_Review_Bean;
```

- the creation function must be registered in the AWA framework under a name that identifies the create function. Again, an example of this registration has been generated by Dynamo and we are going to use it as is.

```
1 Register.Register (Plugin => Plugin,
2                   Name   => "Atlas.Reviews.Beans.Reviews_Bean",
3                   Handler => Atlas.Reviews.Beans.Create_Review_Bean'
4                   Access);
```

- the last step is the configuration step. In the module XML configuration file, we must declare the Ada bean name and indicate what create function must be called to create it. We will use the

managed-bean XML declaration that comes from Java Server Faces. We can declare as many Ada beans as we want each of them with a different name.

```
1 <managed-bean>
2   <description>An example of a bean (change description and bean name)
3   </description>
4   <managed-bean-name>review</managed-bean-name>
5   <managed-bean-class>Atlas.Reviews.Beans.Reviews_Bean</managed-bean-
6   class>
7   <managed-bean-scope>request</managed-bean-scope>
8 </managed-bean>
```

When the UEL expression `#{review.title}` is used, the AWA framework looks for the Ada bean represented by `review` and identified by the `managed-bean-name` entry. It then calls the create function defined by the `managed-bean-class`. The Ada bean object is then stored either in the **request** context, a **session** context or an **application** context. This is defined by the `managed-bean-scope` entry. The **request** scope means that the Ada bean object is created once for each request. Concurrent page accesses will use their own Ada bean object instance. The **session** scope means that the Ada bean object is shared between requests on the same session. The **application** scope means that the Ada bean object is global to the application, shared by every request and every session.

3.4.7 Navigation rules

We have seen that when the review creation form is submitted the `<h:commandButton>` component has invoked the `Save` procedure of our `Review_Bean` object. The review object has been created and saved in the database and we kept the relation between the new review and the user.

We must now decide what should happen for the user to see the result. We could display a new form, update some page content or redirect to a new page. All this is defined by the navigation rules.

The navigation rules is the Java Server Faces mechanism that controls and defines what is the next page or view that must be displayed to a user. The navigation rules are configured in the module XML configuration file.

In the definition below, the navigation rule defines that the user is redirected to the page `/reviews/list.xhtml` if the current page was `/reviews/edit-review.xhtml` and the operation returned `success`.

```
1 <navigation-rule>
2   <from-view-id>/reviews/edit-review.xhtml</from-view-id>
3   <navigation-case>
4     <from-outcome>success</from-outcome>
5     <to-view-id>/reviews/list.xhtml</to-view-id>
6     <redirect/>
```

```
7     </navigation-case>
8 </navigation-rule>
```

3.5 Creating the module

3.5.1 Adding the module operations

Now, we must add two operations on the business logic to save a review and delete a review. The Dynamo code generator provides the `add-module-operation` command that will help us in this task. Let's run it:

```
1 dynamo add-module-operation reviews review Save
2 dynamo add-module-operation reviews review Delete
```

The first parameter is the name of the module where the new operation is added. This is the name of the module that was created by using the `add-module` operation. In our case, this is the `reviews` module.

The second parameter is the name of the database entity or database table if you prefer.

The `add-module-operation` command modifies the Ada module specification and body to define and implement the following operation:

```
1 package Atlas.Reviews.Modules is
2 ...
3 procedure Save (Model : in Review_Module;
4                 Entity : in out Atlas.Reviews.Models.Review_Ref'Class);
5 ...
```

The object to save in the `Review` table is passed as parameter to the `Save` operation. The procedure body that was generated is rather simple but functional: it just saves the object in the database within a transaction. In many cases it is ready to use but you may also need to modify the operation to either change the implementation or even add new parameters.

3.5.2 Saving our review

Before saving our review entity object, we want to associate it with the current user. We have to know who is the current user and for this we can use the AWA service context. The AWA service context is an object that is provided by the `AWA.Services.Contexts` package and that provides some useful contextual information for the business logic:

- It indicates the optional user that is authenticated and is doing the call,

- It gives access to the database connections that the business logic can use,
- It allows to manage database transactions.

The current service context is retrieved by using the `AWA.Services.Contexts.Current` function and we can use the `Get_User` function to know the current user. The `Save` procedure implementation is the following:

```
1 package ASC renames AWA.Services.Contexts;
2 procedure Save (Model : in Review_Module;
3               Entity : in out Atlas.Reviews.Models.Review_Ref'Class)
4               is
5     Ctx : constant ASC.Service_Context_Access := ASC.Current;
6     DB  : ADO.Sessions.Master_Session := AWA.Services.Contexts.
7         Get_Master_Session (Ctx);
8 begin
9     Ctx.Start;
10    if not Entity.Is_Inserted then
11        Entity.Set_Reviewer (Ctx.Get_User);
12        Entity.Set_Create_Date (Ada.Calendar.Clock);
13    end if;
14    Entity.Save (DB);
15    Ctx.Commit;
16 end Save;
```

3.5.3 Setting up the permissions

Because we want to bring some minimal security to the review web application, we are going to setup some permissions that will be enforced by the business logic layer when a save or delete operation is done. The AWA framework uses the Ada Security to implement and enforce permissions. For this we need:

- An Ada definition of the permission,
- Adding a verification to enforce the permission in the new module operations,
- A definition of the permission rules.

3.5.3.1 Generating the permission Dynamo provides the `add-permissions` command to help us in the first task. It generates some Ada code that declares the permissions. It also provides a default configuration for the new permissions.

```
1 dynamo add-permissions reviews review
```

The first parameter is the name of our module where the new permissions are declared and the second parameter is the name of the database entity. The command will modify the Ada module specification and add the following lines:

```
1 package Atlas.Reviews.Modules is
2 ...
3 package ACL_Create_Reviews is new Security.Permissions.Definition ("
  review-create");
4 package ACL_Delete_Reviews is new Security.Permissions.Definition ("
  review-delete");
5 package ACL_Update_Reviews is new Security.Permissions.Definition ("
  review-update");
```

Each of these package instantiation, declares a single permission identified by a name.

3.5.3.2 Enforcing security Now that we have our permission, we can enforce the security in the `Save` and `Delete` operation. This is done by using the `Check` operation provided by the `AWA.Permissions` package.

To verify that the user has the permission to create a new review, we can use the following call:

```
1 AWA.Permissions.Check (Permission => ACL_Create_Reviews.Permission);
```

This operation will verify that the user has the given permission and it will raise the `AWA.Permissions.NO_PERMISSION` exception if this is not the case. By raising such exception, the `Check` procedure acts as a barrier that grants or not the access to the rest of the code.

Now, if we have a review to modify, we will use the update permission and also give the review object to the `Check` operation so that it can verify if that particular review can be modified.

```
1 AWA.Permissions.Check (Permission => ACL_Update_Reviews.Permission,
2                        Entity => Entity);
```

3.5.3.3 Configuring the permission Until now we have created the permission and enforced it in the business logic. We have not defined the rules that tell what is really checked to verify the permission. The configuration part is defined in the XML file `config/reviews.xml` that was generated when the reviews module was created. The `add-permissions` command has modified the XML file to provide some default configuration. It has generated a XML permission for the `review-create`, `review-update` and `review-delete` permissions.

The `review-create` permission is defined as follows:

```
1 <auth-permission>
2   <name>review-create</name>
3 </auth-permission>
```

This XML definition associate the Authenticated Permission controller to the `review-create` permission. With that controller the permission is granted if the security context has a principal (ie, a user

is authenticated).

The `review-update` permission has another definition that we must change. Basically, we want that only the reviewer that created the review can update the review. For this we will use the entity permission controller provided by AWA. The XML definition is the following:

```
1 <entity-permission>
2   <name>review-update</name>
3   <entity-type>altas_review</entity-type>
4   <sql>
5     SELECT r.id FROM atlas_review AS r
6     WHERE r.id = :entity_id AND r.reviewer_id = :user_id
7   </sql>
8 </entity-permission>
```

When the permission is checked, the entity permission controller will use the SQL statement to verify the permission. The SQL statement has three parameters:

- `user_id` is the ID of the user associated with the security context. If there is no authenticated user, the permission is refused.
- `entity_id` is the ID of the database entity as passed to the `Check` procedure and propagated to the permission controller.
- `entity_type` is a unique number that identifies the database entity type or database table if you prefer. It is created and setup automatically according to the entity type defined in the `entity-type` XML member. It is not used in our example.

At the end, the above SQL statement verifies that the review exists and was created by the current user.

3.6 Using database queries

Our next step is now to list the reviews that have been created. We need to add a page that will list the reviews and we need to implement a database query to fetch the information.

3.6.1 Adding database queries

Since we need to access the list of reviews from the XHTML files, we will map the SQL query result to a list of Ada Beans objects. For this, an [XML query mapping|<https://code.google.com/p/ada-ado/wiki/QueryMapping>] is created to tell how to map the SQL query result into some Ada record. The XML query mapping is then processed by Dynamo to generate the Ada Beans implementation. The XML query mapping is also read by AWA to get the SQL query to execute.

A template of the XML query mapping can be added to a project by using the dynamo `add-query` command. The first parameter is the module name (`reviews`) and the second parameter the name of the query (`list`). The command will generate the file `db/reviews-list.xml`.

```
1 dynamo add-query reviews list
```

The generated XML query mapping is an example of a query. You can replace it or update it according to your needs. The first part of the XML query mapping is a **class** declaration that describes the type to represent each row returned by our query. Within the **class**, a set of **property** definition describes the class attributes with their type and name.

```
1 <query-mapping package='Atlas.Reviews.Models'>
2   <class name="Atlas.Reviews.Models.List_Info" bean="yes">
3     <comment>The list of reviews.</comment>
4     <property type='Identifier' name="id">
5       <comment>the review identifier.</comment>
6     </property>
7     <property type='String' name="title">
8       <comment>the review title.</comment>
9     </property>
10    ...
11  </class>
12 </query-mapping>
```

Following the **class** declaration, the **query** declaration describes a query by giving it a name and describing the SQL statement to execute. By having the SQL statement separate and external to the application, we can update, fix and tune the SQL without rebuilding the application. The Dynamo code generator will use the **query** declaration to generate a query definition that can be referenced and used from the Ada code.

The SQL statement is defined within the **sql** XML entity. The optional **sql-count** XML entity is used to associate a count query that can be used for the pagination.

We want to display the review with the author's name and email address. The list will be sorted by date to show the newest reviews first. The SQL to execute is the following:

```
1 <query-mapping package='Atlas.Reviews.Models'>
2   ...
3   <query name='list'>
4     <comment>Get the list of reviews</comment>
5     <sql>
6 SELECT
7     r.id,
8     r.title,
9     r.site,
10    r.create_date,
11    r.allow_comments,
12    r.reviewer_id,
```

```
13     a.name,  
14     e.email,  
15     r.text  
16 FROM atlas_review AS r  
17 INNER JOIN awa_user AS a ON r.reviewer_id = a.id  
18 INNER JOIN awa_email AS e ON a.email_id = e.id  
19 ORDER BY r.create_date DESC  
20     LIMIT :first, :last  
21     </sql>  
22     <sql-count>  
23     SELECT  
24         count(r.id)  
25     FROM atlas_review AS r  
26     </sql-count>  
27 </query>  
28 </query-mapping>
```

The query has two named parameters represented by `:first` and `:last`. These parameters allow to paginate the list of reviews.

The complete source can be seen in the file: `db/reviews-list.xml`.

Once the XML query is written, the Ada code is generated by Dynamo by reading the UML model and all the XML query mapping defined for the application. Dynamo merges all the definitions into the target Ada packages and generates the Ada code in the `src/model` directory. You can use the `generate` make target:

```
1 make generate
```

or run the following command manually:

```
1 dynamo generate db uml/atlas.zargo
```

From the `List_Info` class definition, Dynamo generates the `List_Info` tagged record. The record contains all the data members described in the `class` XML entity description. The `List_Info` represents one row returned by the SQL query. The attributes of the `List_Info` can be accessed from the XHTML files by using UEL expression and the property name defined for each attribute.

To describe the list of rows, Dynamo generates the `List_Info_Beans` package which instantiates the `Util.Beans.Basic.Lists` generic package. This provides an Ada vector for the `List_Info` type and an Ada bean that gives access to the list.

```
1 package Atlas.Reviews.Models is  
2 ...  
3 type List_Info is new Util.Beans.Basic.ReadOnly_Bean with record  
4 ...  
5 package List_Info_Beans is  
6     new Util.Beans.Basic.Lists (Element_Type => List_Info);
```

```

7   package List_Info_Vectors renames List_Info_Beans.Vectors;
8   subtype List_Info_List_Bean is List_Info_Beans.List_Bean;
9   subtype List_Info_Vector is List_Info_Vectors.Vector;
10  Query_List : constant ADO.Queries.Query_Definition_Access;
11  ...
12  end Atlas.Reviews.Models;

```

The generated code can be seen in `src/model/atlas-reviews-models.ads`.

3.6.2 Implementing the review list bean

In order to access the list of reviews from the XHTML facet file, we must create an Ada bean that provides the list of reviews. This Ada bean is modeled in the UML model and we define:

- A set of attributes to manage the review list pagination (`page`, `page_size`, `count`)
- An Ada bean action that can be called from the XHTML facet file (`load`)

The `Review_List_Bean` tagged record will hold the list of reviews for us:

```

1  package Atlas.Reviews.Beans is
2  ...
3  type Review_List_Bean is new Atlas.Reviews.Models.Review_List_Bean
    with record
4      Module      : Atlas.Reviews.Modules.Review_Module_Access := null
5      ;
6      Reviews     : aliased Atlas.Reviews.Models.List_Info_List_Bean;
7      Reviews_Bean : Atlas.Reviews.Models.List_Info_List_Bean_Access;
8  end record;
9  type Review_List_Bean_Access is access all Review_List_Bean'Class;
end Atlas.Reviews.Beans;

```

We must now implement the `Load` operation that was described in the UML model and we are going to use our `list` query. For this, we use the `ADO.Queries.Context` to setup the query to retrieve the list of reviews. A call to `Set_Query` indicates the query that will be used. Since that query needs two parameters (`first` and `last`), we use the `Bind_Param` operation to give the two values. The list of reviews is then retrieved easily by calling the `Atlas.Reviews.Models.List` operation that was generated by Dynamo.

```

1  package body Atlas.Reviews.Beans is
2  ...
3  overriding
4  procedure Load (Into      : in out Review_List_Bean;
5                  Outcome : in out Ada.Strings.Unbounded.
                        Unbounded_String) is
6      Session      : ADO.Sessions.Session := Into.Module.Get_Session;
7      Query        : ADO.Queries.Context;
8      Count_Query  : ADO.Queries.Context;

```

```

 9      First      : constant Natural := (Into.Page - 1) * Into.
        Page_Size;
10      Last       : constant Positive := First + Into.Page_Size;
11      begin
12          Query.Set_Query (Atlas.Reviews.Models.Query_List);
13          Count_Query.Set_Count_Query (Atlas.Reviews.Models.Query_List);
14          Query.Bind_Param (Name => "first", Value => First);
15          Query.Bind_Param (Name => "last", Value => Last);
16          Atlas.Reviews.Models.List (Into.Reviews, Session, Query);
17          Into.Count := ADO.Datasets.Get_Count (Session, Count_Query);
18      end Load;
19 end Atlas.Reviews.Beans;

```

3.6.3 Review list bean creation

The AWA framework must be able to create an instance of the `Review_List_Bean` type. For this, we have to declare and implement a constructor function that allocates an instance of the `Review_List_Bean` type and setup some pre-defined values. When the instance is returned, the list of reviews is not loaded.

```

1  package body Atlas.Reviews.Beans is
2      ...
3      function Create_Review_List_Bean (Module : in Atlas.Reviews.Modules.
        Review_Module_Access)
4          return Util.Beans.Basic.
            Readonly_Bean_Access is
5          Object : constant Review_List_Bean_Access := new
            Review_List_Bean;
6      begin
7          Object.Module      := Module;
8          Object.Reviews_Bean := Object.Reviews'Access;
9          Object.Page_Size   := 20;
10         Object.Page        := 1;
11         Object.Count       := 0;
12         return Object.all'Access;
13     end Create_Review_List_Bean;
14 end Atlas.Reviews.Beans;

```

The constructor function is then registered in the `Atlas.Reviews.Modules` package within the `Initialize` procedure. This registration allows to give a name for this constructor function and be able to specify it in the `managed-bean` bean declaration.

```

1  package body Atlas.Reviews.Modules is
2      ...
3      overriding
4      procedure Initialize (Plugin : in out Review_Module;
5                          App      : in AWA.Modules.Application_Access;

```

```
6           Props : in ASF.Applications.Config) is
7   begin
8       ...
9       Register.Register (Plugin => Plugin,
10          Name      => "Atlas.Reviews.Beans.
11             Review_List_Bean",
12          Handler => Atlas.Reviews.Beans.
13             Create_Review_List_Bean'Access);
12   end Initialize;
13 end Atlas.Reviews.Modules;
```

3.6.4 Review list bean declaration

The managed-bean XML declaration associates a name to a constructor function that will be called when the name is needed. The scope of the Ada bean is set to `request` so that a new instance is created for each HTTP GET request.

```
1 <managed-bean>
2   <description>The list of reviews</description>
3   <managed-bean-name>reviewList</managed-bean-name>
4   <managed-bean-class>Atlas.Reviews.Beans.Review_List_Bean</managed-
   bean-class>
5   <managed-bean-scope>request</managed-bean-scope>
6 </managed-bean>
```

Two other scopes are allowed: `session` and `application`. The `session` scope indicates that the new instance is created and associated with the user browsing session. It allows to share some instance between several HTTP requests. Care must be made when designing the Ada bean instance because concurrent HTTP requests can access and modify the Ada bean concurrently.

The `application` scope associates the new instance globally to the application. It means the instance is shared across all requests concurrently.

3.6.5 Listing the reviews: the XHTML facelet presentation file

To load the reviews to be displayed we will use a JSF 2.2 view action. The review list page has a parameter `page` that indicates the page number to be displayed. The `f:viewParam` allows to retrieve that parameter and configure the `reviewList` Ada bean with it. Then, the `f:viewAction` defines the action that will be executed after the view parameters are extracted, validated and passed to the Ada bean. In our case, we will call the `load` operation on our `reviewList` Ada bean.

```
1 <f:metadata>
2   <f:viewParam id='page' value='#{reviewList.page}' required="false"
   />
```



```

3     <f:viewAction action="#{reviewList.load}"/>
4 </f:metadata>

```

To summarize, the `reviewList` Ada bean is created, then configured for the pagination and filled with the current page content by running our SQL query by running the `Load` procedure.

The easy part is now to render the list of reviews. The XHTML file uses the `h:list` component to iterate over the list items and render each of them. At each iteration, the `h:list` component initializes the Ada bean `review` to refer to the current row in the review list. We can then access each attribute defined in the XML query mapping by using the property name of that attribute. For example `review.title` returns the `title` property.

```

1 <h:list var="review" value="#{reviewList.reviews}">
2   <div class="review" id="p_#{review.id}">
3     <div class="review-title">
4       <h2><a href="#{review.site}">#{review.title}</a></h2>
5       <ul class="review-info">
6         <li><span>By #{review.reviewer_name}</span></li>
7         <li>
8           <h:outputText styleClass="review-date"
9             value="#{review.date}"
10            converter="dateConverter"/>
11         </li>
12         <h:panelGroup rendered="#{review.reviewer_id == user.id}">
13           <li>
14             <a href="#{contextPath}/reviews/edit-review.
15               html?id=#{review.id}">
16               #{reviewMsg.review_edit_label}
17             </a>
18           </li>
19           <li>
20             <a href="#"
21               onclick="return ASF.OpenDialog(this, '
22                 deleteDialog', '#{contextPath}/reviews/
23                 forms/delete-review.html?id=#{review.id
24                 }');">
25               #{reviewMsg.review_delete_label}
26             </a>
27           </li>
28         </h:panelGroup>
29       </ul>
30     </div>
31     <awa:wiki styleClass="review-text post-text" value="#{review.
32       text}" format="dotclear"/>
33   </div>
34 </h:list>

```

3.6.6 Understanding the request flow

Let's see the whole request flow to better understand what happens.

To display the list of reviews, the user's browser makes an HTTP GET request to the page `/reviews/list.html`. This page maps to the XHTML file `web/reviews/list.xhtml` that we created in the Adding pages section.

The Ada Server Faces framework handles the request by first reading the XHTML file and building a tree of components that represent the view to render. Within that tree of component, the `<f:metadata>` component allows to make a pre-initialization of components and Ada beans before the component tree is rendered.

For the pre-initialization, the `reviewList` Ada bean is created because it is referenced in an EL expression used by the `<f:viewParam>` component or by the `<f:viewAction>`. For this creation, the `Create_Review_List_Bean` constructor that we registered is called. The `page` attribute is set on the `reviewList` Ada bean if it was passed as a URL request parameter.

The `load` action is then called by Ada Server Faces which triggers execution of the `Load` procedure and the current review list page is retrieved by executing the SQL query.

As soon as the `load` action terminates, the rendering of the component tree can be processed. The `reviewList` Ada bean contains the information to display and the `<h:list>` component iterates over the list and renders each row at a time.

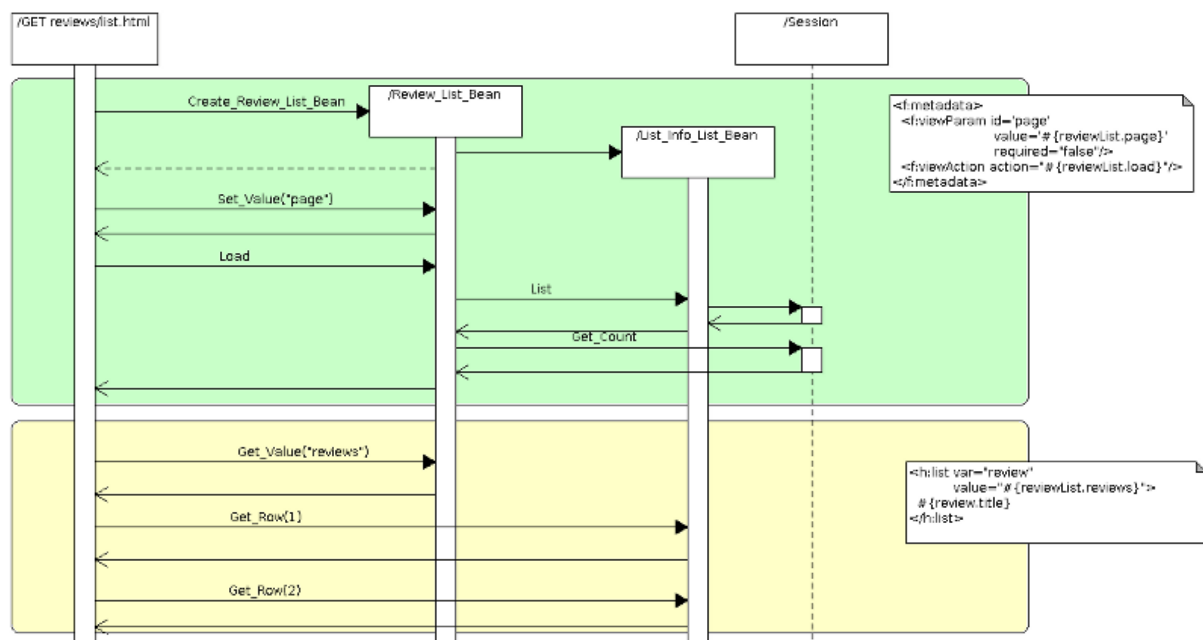


Figure 10: Review list flow

4 AWA Core

4.1 Initialization

The AWA application is represented by the `Application` type which should be extended for the final application to provide the modules and specific components of the final application.

The initialization of an AWA application is made in several steps represented by different procedures of the main `Application` type. The whole initialization is handled by the `Initialize` procedure which gets a first set of configuration properties and a factory to build specific component.

The `Initialize` procedure will perform the following steps:

- It uses the factory to allocate the ASF lifecycle handler, the navigation handler, the security manager, the OAuth manager, the exception handlers.
- It calls the `Initialize_Components` procedure to let the application register all the ASF components. These components must be registered before any configuration file is read.
- It calls the `Initialize_Config`
- It calls the `Initialize_Servlets` procedure to allow the application to register all the servlet instances used by the application.
- It calls the `Initialize_Filters` procedure to allow the application to register all the servlet filter instances. The application servlets and filters must be registered before reading the global configuration file.
- It loads the global application configuration by reading the `awa.xml` file as well as some application specific files. By reading this configuration, some global configuration is established on the servlets, filters, navigation rules. The list of XML files loaded during this step is controlled by the `app.config` configuration.
- It calls the `Initialize_Modules` procedure so that all the application modules can be registered, configured and initialized. Each module brings its own component, servlet and filter. They are configured by their own XML configuration file.
- It loads the module application configuration by reading the XML files described by the `app.config.plugins` configuration. This last step allows the application to setup and update the configuration of all modules that have been registered.

4.2 Configuration

The following global configuration parameter are defined:

Name	Description
awa_url_scheme	The application URL scheme to use when building URL. #{empty app_url_scheme ? 'http://' : app_url_scheme}
awa_url_host	The application URL host to use when building URL. #{empty app_url_host ? 'localhost' : app_url_host}
awa_url_port	The application TCP port to use when building URL. #{empty app_url_port ? ':8080' : app_url_port}
app_url_base	The application URL base to use when building URL. #{empty app_url_base ? 'http://localhost:8080' : app_url_base}
app_oauth_url_base	#{empty app_oauth_url_base ? 'http://localhost:8080' : app_oauth_url_base}
app_page_500_report	Controls the level of information that is printed by the 500 error page when an error occurs. When <code>full</code> , report the full stack track if available, when <code>minimal</code> report minimal information but without a stack track, when <code>'none'</code> don't give any information. #{empty app_page_500_report ? 'full' : app_page_500_report}
view.ext	Defines the extension used for Ada Server Faces presentation pages. .html
view.dir	Defines a list of paths separated by ';' where the XHTML files are searched. The default searches for the 'web' directory in the application search paths. #{fn:composePath(app_search_dirs,'web')}
content-type.default	Defines the default content type for the file servlet. text/plain
ado.queries.load	Controls whether the database query definitions are loaded. true
ado.queries.paths	Defines a list of paths separated by ';' where the database query files are searched. The default searches for the 'db' directory in the application search paths.

Name	Description
	<code>#{fn:composePath(app_search_dirs,'db')}</code>
<code>ado.migrate.paths</code>	Defines a list of paths separated by ';' where the database migration files are searched. The default searches for the 'db/migrate' directory in the application search paths.
	<code>#{fn:composePath(app_search_dirs,'db/migrate')}</code>
<code>bundle.dir</code>	Defines a list of paths separated by ';' where the resource bundle files are searched. The default searches for the 'bundles' directory in the application search paths.
	<code>#{fn:composePath(app_search_dirs,'bundles')}</code>
<code>app.modules.dir</code>	Defines a list of paths separated by ';' where the module configuration files are searched. The default searches for the 'config' directory in the application search paths.
	<code>#{fn:composePath(app_search_dirs,'config')}</code>

4.3 AWA Modules

A module is a software component that can be integrated in the web application. The module can bring a set of service APIs, some Ada beans and some presentation files. The AWA framework allows to configure various parts of a module when it is integrated in an application. Some modules are designed to be re-used by several applications (for example a *mail* module, a *users* module, ...). Other modules could be specific to an application. An application will be made of several modules, some will be generic some others specific to the application.

4.3.1 Registration

The module should have only one instance per application and it must be registered when the application is initialized. The module instance should be added to the application record as follows:

```

1  type Application is new AWA.Applications.Application with record
2      Xxx          : aliased Xxx_Module;
3  end record;
```

The application record must override the `Initialize_Module` procedure and it must register the module instance. This is done as follows:

```
1 overriding  
2 procedure Initialize_Modules (App : in out Application) is  
3 begin  
4     Register (App      => App.Self.all'Access,  
5               Name     => Xxx.Module.NAME,  
6               URI      => "xxx",  
7               Module   => App.User_Module'Access);  
8 end Initialize_Modules;
```

The module is registered under a unique name. That name is used to load the module configuration.

4.3.2 Configuration

The module is configured by using an XML or a properties file. The configuration file is used to define:

- the Ada beans that the module defines and uses,
- the events that the module wants to receive and the action that must be performed when the event is posted,
- the permissions that the module needs and how to check them,
- the navigation rules which are used for the module web interface,
- the servlet and filter mappings used by the module

The module configuration is located in the *config* directory and must be the name of the module followed by the file extension (example: `module-name.xml` or `module-name.properties`).

4.4 AWA Permissions

The *AWA.Permissions* framework defines and controls the permissions used by an application to verify and grant access to the data and application service. The framework provides a set of services and API that helps an application in enforcing its specific permissions. Permissions are verified by a permission controller which uses the service context to have information about the user and other context. The framework allows to use different kinds of permission controllers. The *Entity_Controller* is the default permission controller which uses the database and an XML configuration to verify a permission.

4.4.1 Declaration

To be used in the application, the first step is to declare the permission. This is a static definition of the permission that will be used to ask to verify the permission. The permission is given a unique name that will be used in configuration files:

```
1 with Security.Permissions;
2 ...
3 package ACL_Create_Post is new Security.Permissions.Definition ("blog-
   create-post");
```

4.4.2 Checking for a permission

A permission can be checked in Ada as well as in the presentation pages. This is done by using the `Check` procedure and the permission definition. This operation acts as a barrier: it does not return anything but returns normally if the permission is granted. If the permission is denied, it raises the `NO_PERMISSION` exception.

Several `Check` operation exists. Some require no argument and some others need a context such as some entity identifier to perform the check.

```
1 with AWA.Permissions;
2 ...
3 AWA.Permissions.Check (Permission => ACL_Create_Post.Permission,
4                        Entity      => Blog_Id);
```

4.4.3 Configuring a permission

The *AWA.Permissions* framework supports a simple permission model. The application configuration file must provide some information to help in checking the permission. The permission name is referenced by the `name` XML entity. The `entity-type` refers to the database entity (ie, the table) that the permission concerns. The `sql` XML entity represents the SQL statement that must be used to verify the permission.

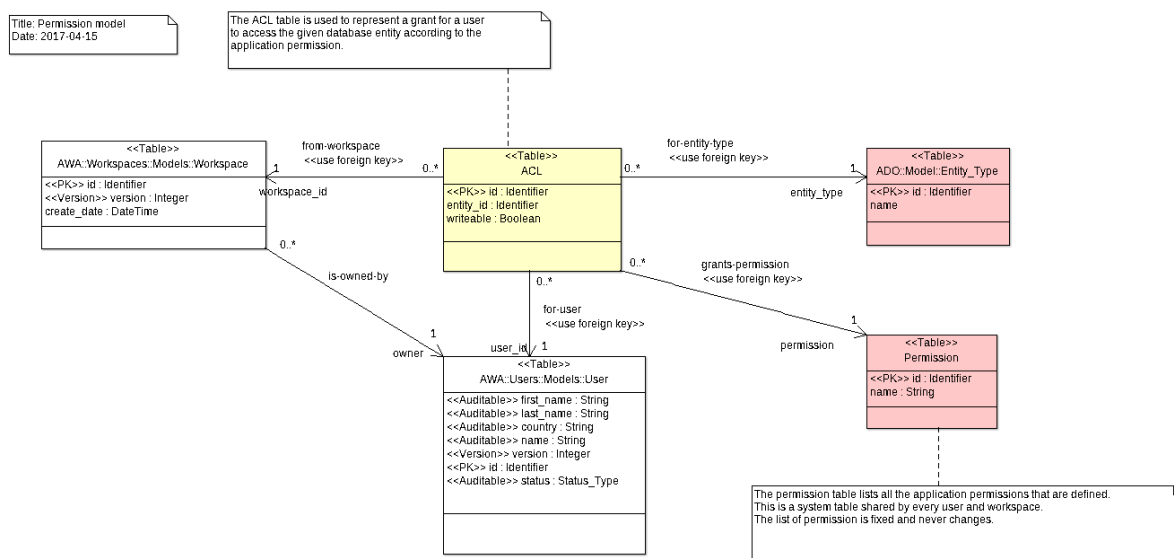
```
1 <entity-permission>
2   <name>blog-create-post</name>
3   <entity-type>blog</entity-type>
4   <description>Permission to create a new post.</description>
5   <sql>
6     SELECT acl.id FROM acl
7     WHERE acl.entity_type = :entity_type
8     AND   acl.user_id = :user_id
9     AND   acl.entity_id = :entity_id
10  </sql>
11 </entity-permission>
```

4.4.4 Adding a permission

Adding a permission means to create an [ACL](#) database record that links a given database entity to the user. This is done easily with the `Add_Permission` procedure:

```
1 with AWA.Permissions.Services;
2 ...
3 AWA.Permissions.Services.Add_Permission (Session => DB,
4                                         User    => User,
5                                         Entity  => Blog);
```

4.4.5 Data Model



4.4.6 Queries

4.4.7 Queries

Name	Description
check-entity-permission	Get the permission for a user and an entity
remove-permission	Delete the permission associated with a user and an object

Name	Description
remove-entity-permission	Delete all the permission associated with an object
remove-user-permission	Delete all the permission associated with a user

4.5 AWA Events

The `AWA.Events` package defines an event framework for modules to post events and have Ada bean methods be invoked when these events are dispatched. Subscription to events is done through configuration files. This allows to configure the modules and integrate them together easily at configuration time.

4.5.1 Declaration

Modules define the events that they can generate by instantiating the `Definition` package. This is a static definition of the event. Each event is given a unique name.

```
1 with AWA.Events.Definition;  
2 ...  
3 package Event_New_User is new AWA.Events.Definition ("new-user");
```

4.5.2 Posting an event

The module can post an event to inform other modules or the system that a particular action occurred. The module creates the event instance of type `Module_Event` and populates that event with useful properties for event receivers.

```
1 with AWA.Events;  
2 ...  
3 Event : AWA.Events.Module_Event;  
4 Event.Set_Event_Kind (Event_New_User.Kind);  
5 Event.Set_Parameter ("email", "harry.potter@hogwarts.org");
```

The module will post the event by using the `Send_Event` operation.

```
1 Manager.Send_Event (Event);
```

4.5.3 Receiving an event

Modules or applications interested by a particular event will configure the event manager to dispatch the event to an Ada bean event action. The Ada bean is an object that must implement a procedure that matches the prototype:

```
1 type Action_Bean is new Util.Beans.Basic.Readonly_Bean ...;  
2 procedure Action (Bean : in out Action_Bean;  
3                 Event : in AWA.Events.Module_Event'Class);
```

The Ada bean method and object are registered as other Ada beans.

The configuration file indicates how to bind the Ada bean action and the event together. The action is specified using an EL Method Expression (See Ada EL or JSR 245).

```
1 <on-event name="new_user">  
2   <action>#{ada_bean.action}</action>  
3 </on-event>
```

4.5.4 Event queues and dispatchers

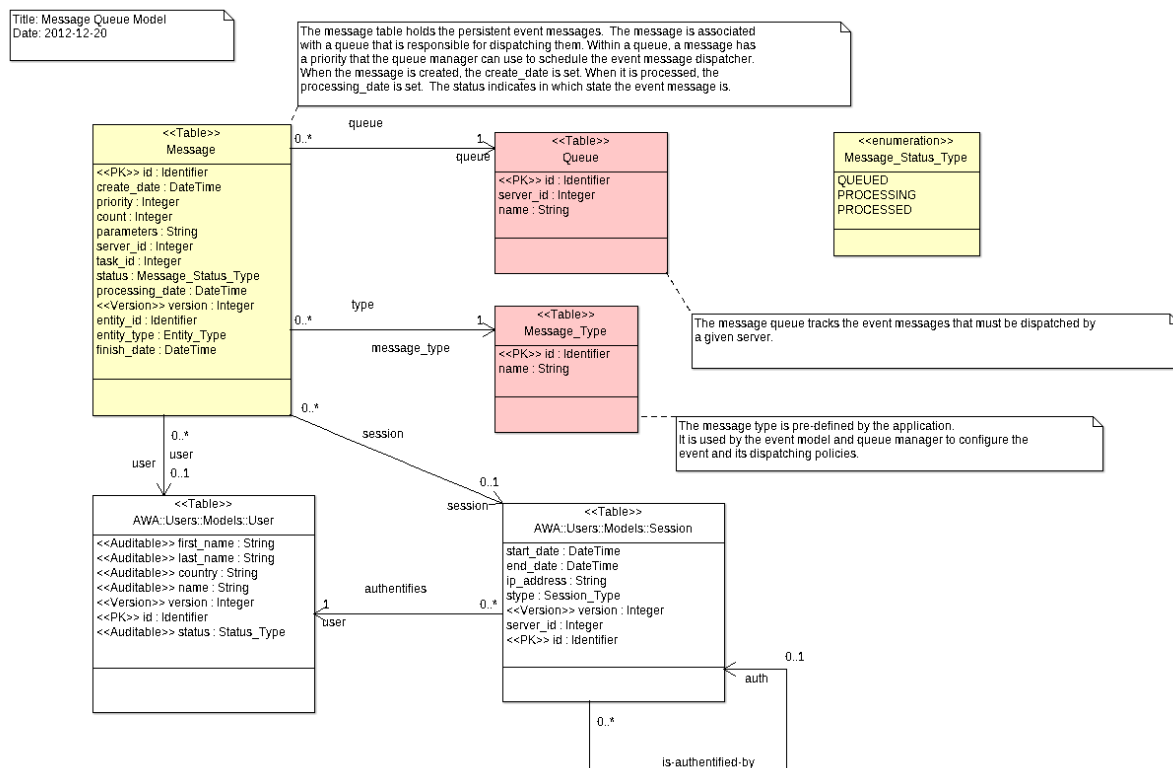
The `AWA.Events` framework posts events on queues and it uses a dispatcher to process them. There are two kinds of dispatchers:

- Synchronous dispatcher process the event when it is posted. The task which posts the event invokes the Ada bean action. In this dispatching mode, there is no event queue. If the action method raises an exception, it will however be blocked.
- Asynchronous dispatcher are executed by dedicated tasks. The event is put in an event queue. A dispatcher task processes the event and invokes the action method at a later time.

When the event is queued, there are two types of event queues:

- A Fifo memory queue manages the event and dispatches them in FIFO order. If the application is stopped, the events present in the Fifo queue are lost.
- A persistent event queue manages the event in a similar way as the FIFO queue but saves them in the database. If the application is stopped, events that have not yet been processed will be dispatched when the application is started again.

4.5.5 Data Model



5 AWA Commands

The `AWA.Commands` package provides a simple framework with commands that allow to start, stop, configure and manage the web application. It is also possible to provide your own commands. The command framework handles the parsing of command line options, identification of the command to execute and execution of the selected command.

5.1 Command Usage

5.1.1 SYNOPSIS

```

driver [-v] [-vv] [-vvv] [-c config-file] command [-k file] [-d dir] [-p password] [-password password]
[-passfile file] [-passenv name] [-passfd fd] [-passask] [-passcmd cmd] [-wallet-key-file file]
  
```

5.1.2 DESCRIPTION

The `AWA.Commands.Drivers` framework integrates the Ada Keystore support to access some sensitive configuration information such as passwords, database connection strings, API secret keys. The use of the Ada Keystore storage is optional. It is enabled when the `-k _file_` option is specified. When such secure storage is used, a primary password to unlock the keystore is necessary. Passwords are retrieved using one of the following options:

- by reading a file that contains the password,
- by looking at an environment variable,
- by using a command line argument,
- by getting the password through the `ssh-askpass(1)` external command,
- by running an external command,
- by using a GPG private key,
- by asking interactively the user for the password,
- by asking through a network socket for the password.

When the Ada Keystore is used, it is global to all the applications that are registered in the Web server container. To allow to differentiate application specific configuration, each configuration parameter is prefixed by the application name.

To create and update the keystore file, it is necessary to use the `akt(1)` tool. The tool provides many commands for the creation, insertion, removal and update of content that is stored in the keystore file.

If the keystore file was locked by using GPG, it is not necessary to specify any specific option to unlock the keystore. All is needed is the availability of the `gpg2(1)` command with the private key to unlock the keystore.

The server global configuration file that is read with the `-c config-file` option can contain the following configuration:

Name	Description
keystore-path	The path to the global keystore file
keystore-masterkey-path	The path of the file that contains the master keys
keystore-password-path	The path of the file that contains the keystore password
gpg-encrypt	When GPG is used, the GPG command to encrypt some content
gpg-decrypt	When GPG is used, the GPG command to decrypt some content

Name	Description
gpg-list-keys	When GPG is used, the GPG command to list the available GPG keys

5.1.3 OPTIONS

The following options are recognized by the command driver:

`-V`

Prints the application version.

`-v`

Enable the verbose mode.

`-vv`

Enable debugging output.

`-vvv`

Enable debugging output.

`-c config-file`

Defines the path of the global server configuration file.

`-k file`

Specifies the path of the keystore file to open.

`-p password`

The keystore password is passed within the command line. Using this method is convenient but is not safe.

`-passenv envname`

The keystore password is passed within an environment variable with the given name. Using this method is considered safer but still provides some security leaks.

`-passfile path`

The keystore password is passed within a file that is read. The file must not be readable or writable by other users or group: its mode must be `r??---`. The directory that contains the file must also satisfy the not readable by other users or group members, This method is safer.

`-passfd fd`

The keystore password is passed within a pipe whose file descriptor number is given. The file descriptor is read to obtain the password. This method is safer.

`-passask`

The keystore password is retrieved by the running the external tool *ssh-askpass*(1) which will ask the password through either KDE, Gnome or another desktop interactive application. The password is retrieved through a pipe that the driver sets while launching the command.

`-passcmd cmd`

The keystore password is retrieved by the running the external command defined in *cmd*. The command should print the password on its standard output without end of line. The password is retrieved through a pipe that the driver sets while launching the command.

`-wallet-key-file file`

Defines the path of a file which contains the wallet master key file.

5.2 COMMANDS

5.2.1 The start command

driver start [`-management-port` *PORT*] [`-port` *PORT*] [`-connection` *COUNT*] [`-upload` *DIR*] [`-tcp-no-delay`] [`-daemon`] [`-keystore` *PATH*]

The `start` command allows to start the server and all applications that are registered to the web container. When a keystore is specified, it is first unlocked by using one of the unlock mechanism (password, GPG, ...). Then, each application is configured by using its own configuration file and a subset of the keystore secured configuration. Once each application is configured, the web server container is started to listen to the TCP/IP port defined by the `--port=PORT` option. Applications are then started and they can serve HTTP requests through the web server container.

At the same time, a management port is created and used exclusively by the `stop` command to stop the web server container. The `start` command will wait on that management port for the `stop` command to be executed. The management port is configured by the `--management=PORT` option. The management port is local to the host and cannot be accessed remotely.

When the `--daemon` option is used, the server will first put itself in the background. This option is supported only under some Unix systems like *GNU/Linux* and *FreeBSD* and more generally every system where the `daemon`(3) C-library call is supported. On other systems, this option has no effect.

The `--tcp-no-delay` option is supported for recent version of Ada Web Server and configure the web server to use the `TCP_NO_DELAY` option of the TCP/IP stack (strongly recommended).

The `--upload=DIR` option indicates to the web server container the directory that can be used to store temporarily the uploaded files that the server receives.

The `--connection=COUNT` option controls the maximum number of active HTTP requests that the server can handle.

5.2.2 The setup command

```
driver setup [-management-port PORT] [-port PORT] [-connection COUNT] [-upload DIR] [-tcp-no-delay] [-daemon] [-keystore PATH] NAME
```

The `setup` command is very close to the `start` command but it starts the Setup Application to configure the application by using a web browser.

5.2.3 The stop command

```
driver stop [-management-port PORT] [-keystore PATH]
```

The `stop` command is used to inform a running web server container to stop. The management port is used to connect to the web server container and stop it. The management port is local to the host and cannot be accessed remotely.

The management port is configured with the `--management-port=PORT` option.

5.2.4 The migrate command

```
driver migrate [-application NAME] [-keystore PATH] [-execute]
```

The `migrate` command is provided to perform the database schema migration when the application or one of its module is upgraded and uses a new database schema. Each module is versionned and the `migrate` command handles the schema upgrade for each module version included and used by the application. It should be safe to execute this command multiple times since once a database schema is migrated the command will detect this and do nothing.

By default the SQL migration scripts are not executed and the command prints the list of SQL scripts that will be executed if the migration is executed.

The `--execute` option runs the SQL migration scripts.

5.2.5 The list command

```
driver list [-application NAME] [-keystore PATH] [-users] [-jobs] [-sessions] [-tables]
```

The `list` command is intended to help in looking at the application database and see some important information. Because the database is specific to each application, it may be necessary to indicate the application name by using the `--application=NAME` option.

The `--tables` option triggers the list of database tables with the number of entries they contain.

The `--users` option triggers the list of users that are registered in the application.

The `--sessions` option triggers the list of user connection sessions.

The `--jobs` option triggers the list of jobs that have been created and scheduled.

5.2.6 The info command

```
driver info [-application NAME] [-keystore PATH] [-long-lines]
```

The `info` command reports the current configuration used by the application. The configuration is extracted from the AWA default XML configuration files and can be overridden by the application specific configuration. The command allows to see what is the actual configuration. The configuration is printed with the configuration name and its associated value.

The list of configuration parameters are grouped in several categories:

- `Database configuration` gives the configuration properties for the database configuration.
- `Server faces configuration` lists the configuration use by the Ada Servlets and Ada Server Faces.
- `AWA Application` lists the core AWA configuration properties.

Depending on whether a module is used by the application, a number of modules are listed with their configuration.

The `--long-lines` option triggers the list of database tables with the number of entries they contain.

5.2.7 The permission command

```
driver permission [-allow PERM] [-deny PERM] [email...]
```

The `permission` command allows to list the permissions defined for users and it also allows to grant or revoke permissions. When no argument are defined, the command lists every permissions granted

for a given user with an email address and when an email address is defined, it lists the permissions granted for that user. The output is similar to:

```
1 ACL      Permission:Workspace:Type:Entity
   Email
2 1601     wiki-space-create:1302:awa_workspace:1302
   user@proton.me
3 98       storage-create:1302:awa_workspace:1302
   user@proton.me
4 99       storage-delete:1302:awa_workspace:1302
   user@proton.me
5 100      folder-create:1302:awa_workspace:1302
   user@proton.me
```

The command provides a `--allow` and a `--deny` option to grant new permissions or revoke existing permissions.

5.3 Integration

The `AWA.Commands` framework is split in several generic packages that must be instantiated. The `AWA.Commands.Drivers` generic package is the primary package that must be instantiated. It provides the core command line driver framework on top of which the commands are implemented. The instantiation needs two parameter: the name of the application and the type of the web server container. When using Ada Web Server, the following instantiation example can be used:

```
1 with Servlet.Server.Web;
2 with AWA.Commands.Drivers;
3 ...
4 package Server_Commands is
5     new AWA.Commands.Drivers
6     (Driver_Name    => "atlas",
7      Container_Type => Servlet.Server.Web.AWS_Container);
```

The `Driver_Name` is used to print the name of the command when some usage or help is printed. The `Container_Type` is used to declare the web container onto which applications are registered and which provides the web server core implementation. The web server container instance is available through the `WS` variable defined in the `Server_Commands` package.

Once the command driver is instantiated, it is necessary to instantiate each command that you wish to integrate in the final application. For example, to integrate the `start` command, you will do:

```
1 with AWA.Commands.Start;
2 ...
3 package Start_Command is new AWA.Commands.Start (Server_Commands);
```

To integrate the `stop` command, you will do:

```
1 with AWA.Commands.Stop;  
2 ...  
3 package Stop_Command is new AWA.Commands.Stop (Server_Commands);
```

The instantiation of one of the command, automatically registers the command to the command driver.

6 Users Module

The `users` module manages the creation, update, removal and authentication of users in an application. The module provides the foundations for user management in a web application.

A user can register himself by using a subscription form. In that case, a verification mail is sent and the user has to follow the verification link defined in the mail to finish the registration process. The user will authenticate using a password.

A user can also use an OAuth/OpenID account and be automatically authenticated and registered to the application. By using an external authentication server, passwords are not stored in the application.

A user can have one or several permissions that allow to protect the application data. User permissions are managed by the `Permissions.Module`.

6.1 Integration

The `User_Module` manages the creation, update, removal of users in an application. It provides operations that are used by the user beans or other services to create and update wiki pages. An instance of the `User_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 type Application is new AWA.Applications.Application with record
2   User_Module : aliased AWA.Users.Modules.User_Module;
3 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,
2           Name    => AWA.Users.Modules.NAME,
3           Module => App.User_Module'Access);
```

6.2 OAuth Authentication Flow

The OAuth/OpenID authentication flow is implemented by using two servlets that participate in the authentication. A first servlet will start the OAuth/OpenID authentication by building the request that the user must use to authenticate through the OAuth/OpenID authorization server. This servlet is implemented by the `AWA.Users.Servlets.Request_Auth_Servlet` type. The servlet will respond to an HTTP `GET` request and it will redirect the user to the authorization server.

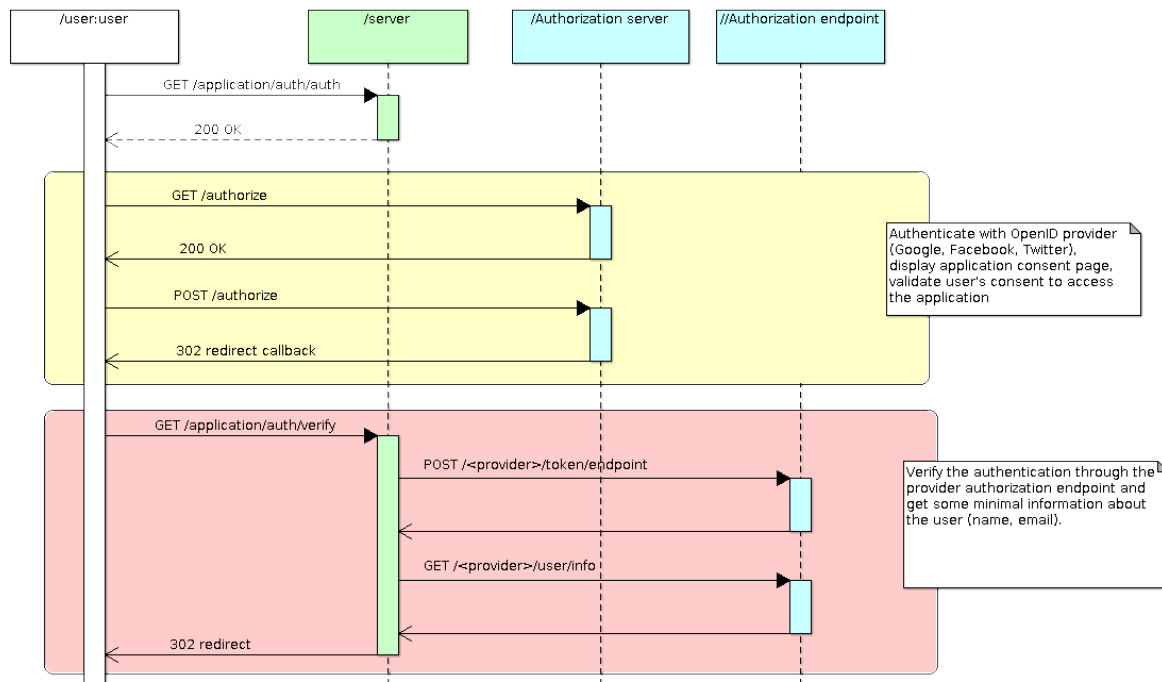


Figure 11: OAuth Authentication Flow

The user will be authenticated by the OAuth/OpenID authorization server and when s/he grants the application to access his or her account, a redirection is made to the second servlet. The second servlet is implemented by `AWA.Users.Servlets.Verify_Auth_Servlet`. It is used to validate the authentication result by checking its validity with the OAuth/OpenID authorization endpoint. During this step, we can retrieve some minimal information that uniquely identifies the user such as a unique identifier that is specific to the OAuth/OpenID authorization server. It is also possible to retrieve the user's name and email address.

These two servlets are provided by the `User_Module` and they are registered under the `openid-auth` name for the first step and under the `openid-verify` name for the second step.

6.3 Configuration

The `users` module uses a set of configuration properties to configure the OpenID integration.

Name	Description
<code>users.server_id</code>	The server id when several instances are used.

Name	Description
	1
users.auth_key	An authentication key used to sign the authentication cookies. 8ef60aad66977c68b12f4f8acab5a4e00a77f6e8
users.allow_register	Allow users to register themselves on the server. When disabled, users can be created by the AWA 'user' command. #{not empty app_login_register ? app_login_register : true}
openid.realm	The REALM URL used by OpenID providers to verify the validity of the verification callback. #{app_url_base}/auth
openid.callback_url	The verification callback URI used by the OpenID provider to redirect the user after authentication. #{app_url_base}/auth/verify
openid.success_url	The URI where the user is redirected after a successful authentication. #{contextPath}/workspaces/main.html
openid.error_url	The URI where the user is redirected when some authenticate error occurred. #{contextPath}/auth/login.html
auth.url.orange	Orange OpenID access point https://openid.orange.fr
auth.provider.orange	Auth provider to use for Orange openid
auth.url.yahoo	Yahoo! OpenID access point https://api.login.yahoo.com/oauth2/request_auth
auth.provider.yahoo	Auth provider to use for Yahoo! yahoo
auth.url.google	Google OpenID access point https://www.google.com/accounts/o8/id
auth.provider.google	Auth provider to use for Google

Name	Description
	openid
auth.url.github	GitHub OAuth2 Connect access point https://github.com/login/oauth/authorize
auth.provider.github	Auth provider to use for GitHub github
github.request_url	GitHub OAuth token access point https://github.com/login/oauth/access_token
github.scope	GitHub OAuth scope read:user, read:email
github.callback_url	GitHub verify callback #{app_oauth_url_base}#{contextPath}/auth/verify
github.issuer	GitHub issuer identification https://github.com
auth.url.gitlab	GitLab OAuth2 Connect access point https://gitlab.com/oauth/authorize
auth.provider.gitlab	Auth provider to use for GitLab yahoo
gitlab.request_url	Gitlab OAuth token access point https://gitlab.com/oauth/token
gitlab.scope	Gitlab OAuth scope openid email
gitlab.callback_url	Gitlab verify callback #{app_oauth_url_base}#{contextPath}/auth/verify
gitlab.issuer	Gitlab issuer identification https://gitlab.com
auth.url.twitter	Twitter OAuth2 Connect access point https://api.twitter.com/oauth/authenticate

Name	Description
auth.provider.twitter	Auth provider to use for Twitter
	yahoo
auth.url.facebook	Facebook OAuth access point https://www.facebook.com/dialog/oauth
auth.provider.facebook	Auth provider to use for Facebook facebook
facebook.callback_url	Facebook verify callback #{app_oauth_url_base}#{contextPath}/auth/verify
facebook.request_url	Facebook request OAuth token access point https://graph.facebook.com/oauth/access_token
facebook.scope	Facebook permission scope public_profile,email
facebook.client_id	Facebook API client ID #{app_facebook_client_id}
facebook.secret	Facebook API secret #{app_facebook_secret}
auth.url.google-plus	Google+ OAuth access point https://accounts.google.com/o/oauth2/auth
auth.provider.google-plus	Auth provider to use for Google+ google-plus
google-plus.issuer	Google+ issuer identification accounts.google.com
google-plus.callback_url	Google+ verify callback #{app_oauth_url_base}#{contextPath}/auth/verify
google-plus.request_url	Google+ request OAuth token access point https://accounts.google.com/o/oauth2/token
google-plus.scope	Google+ permission scope

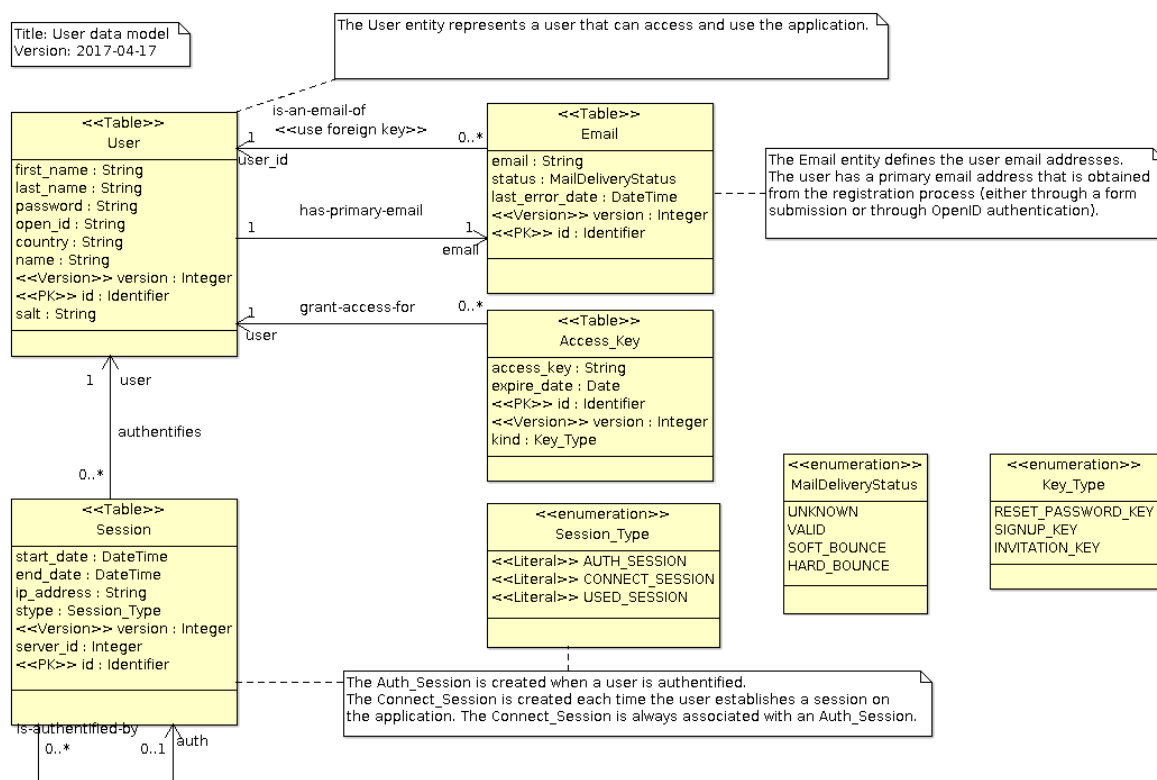
Name	Description
	openid profile email
google-plus.client_id	Google+ API client ID <code>#{app_google_plus_client_id}</code>
google-plus.secret	Google+ API secret <code>#{app_google_plus_secret}</code>
auth-filter.redirect	URI to redirect to the login page <code>#{contextPath}/auth/login.html</code>
verify-access-key.redirect	URI to redirect to the login page <code>#{contextPath}/auth/login.html</code>
verify-access-key.change-password	URI to redirect to the change password page when the access key is verified, the user account is registered but there is no password for authentication <code>#{contextPath}/auth/change-password/</code>
app_login_register	Enable or disable user registration through the web interface <code>#{not empty app_login_register ? app_login_register : true}</code>
app_login_email	Enable or disable user login through the email/password form <code>#{not empty app_login_email ? app_login_email : true}</code>
app_login_openid	Enable or disable user login through OpenID/OAuth2 <code>#{not empty app_login_openid ? app_login_openid : true}</code>
app_login_methods	List of login methods which are enabled for authentication <code>#{not empty app_login_methods ? app_login_methods : 'email,google,facebook'}</code>

6.4 Ada Beans

Several bean types are provided to represent and manage the users. The user module registers the bean constructors when it is initialized. To use them, one must declare a bean definition in the application XML configuration.

Name	Description
login	This bean is used by the login form
register	This bean is used by the registration form
resetPassword	This bean is used by the reset password form
lostPassword	This bean is used by the lost password form
logout	This bean is used by the logout process
user	This bean allows to provide information about the current logged user.

6.5 Data model



7 Jobs Module

The `jobs` module defines a batch job framework for modules to perform and execute long running and deferred actions. The `jobs` module is intended to help web application designers in implementing end to end asynchronous operation. A client schedules a job and does not block nor wait for the immediate completion. Instead, the client asks periodically or uses other mechanisms to check for the job completion.

7.1 Integration

To be able to use the `jobs` module, you will need to add the following line in your GNAT project file:

```
1 with "awa_jobs";
```

An instance of the `Job_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 with AWA.Jobs.Modules;  
2 ...  
3 type Application is new AWA.Applications.Application with record  
4   Job_Module : aliased AWA.Jobs.Modules.Job_Module;  
5 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,  
2           Name   => AWA.Jobs.Modules.NAME,  
3           Module => App.Job_Module'Access);
```

7.2 Writing a job

A new job type is created by implementing the `Execute` operation of the abstract `Job_Type` tagged record.

```
1 type Resize_Job is new AWA.Jobs.Job_Type with ...;
```

The `Execute` procedure must be implemented. It should use the `Get_Parameter` functions to retrieve the job parameters and perform the work. While the job is being executed, it can save result by using the `Set_Result` operations, save messages by using the `Set_Message` operations and report the progress by using `Set_Progress`. It may report the job status by using `Set_Status`.

```
1 procedure Execute (Job : in out Resize_Job) is  
2 begin
```

```
3     Job.Set_Result ("done", "ok");  
4 end Execute;
```

7.3 Registering a job

The `jobs` module must be able to create the job instance when it is going to be executed. For this, a registration package must be instantiated:

```
1 package Resize_Def is new AWA.Jobs.Definition (Resize_Job);
```

and the job definition must be added:

```
1 AWA.Jobs.Modules.Register (Resize_Def.Create'Access);
```

7.4 Scheduling a job

To schedule a job, declare an instance of the job to execute and set the job specific parameters. The job parameters will be saved in the database. As soon as parameters are defined, call the `Schedule` procedure to schedule the job in the job queue and obtain a job identifier.

```
1 Resize : Resize_Job;  
2 ...  
3 Resize.Set_Parameter ("file", "image.png");  
4 Resize.Set_Parameter ("width", "32");  
5 Resize.Set_Parameter ("height", "32");  
6 Resize.Schedule;
```

7.5 Checking for job completion

After a job is scheduled, a unique identifier is allocated that allows to identify it. It is possible to query the status of the job by using the `Get_Job_Status` function:

```
1 Status : AWA.Jobs.Models.Job_Status_Type  
2 := AWA.Jobs.Services.Get_Job_Status (Resize.Get_Identifier);
```

7.6 Job Service

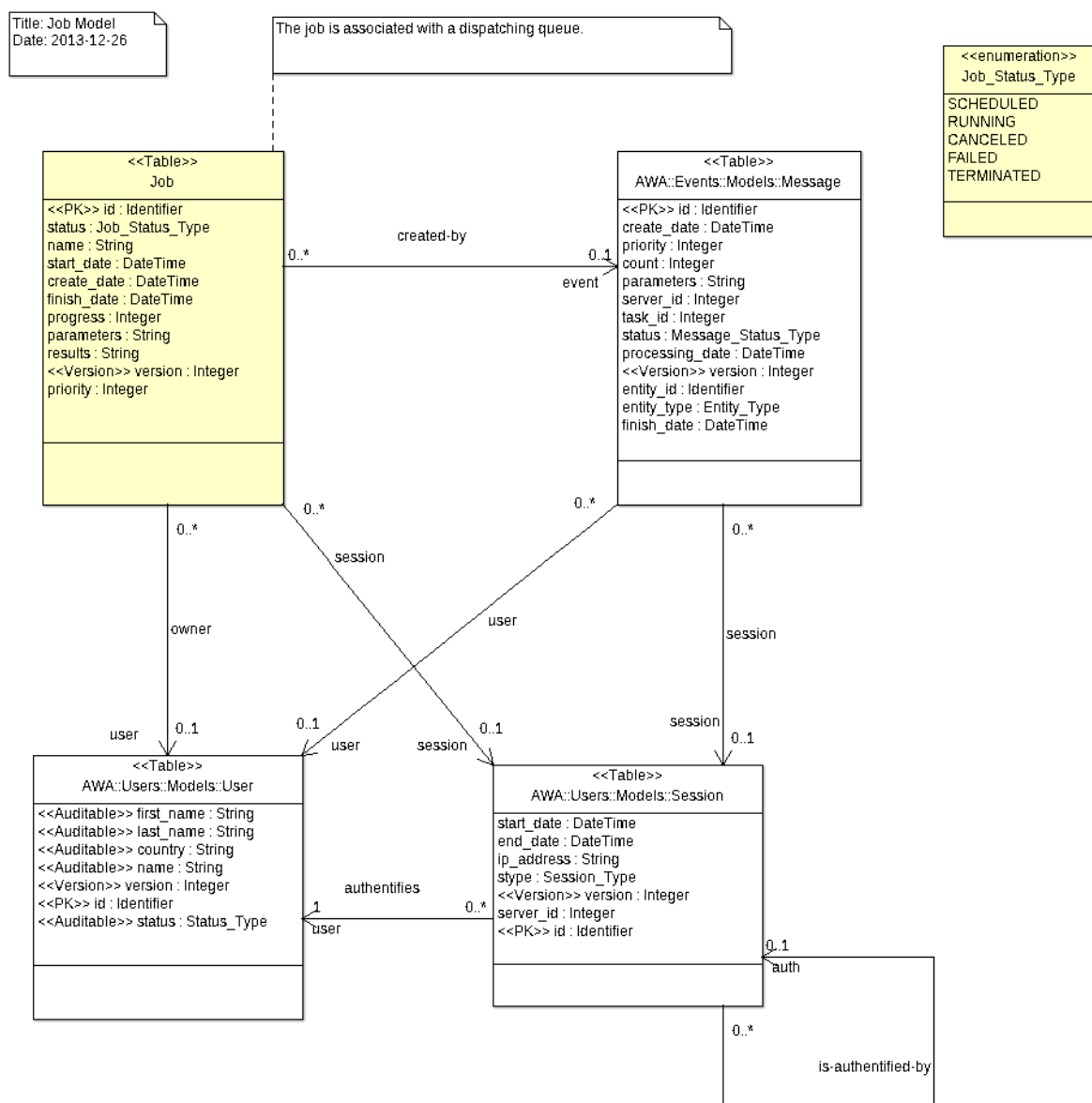
The `AWA.Jobs.Services` package defines the type abstractions and the core operation to define a job operation procedure, create and schedule a job and perform the job work when it is scheduled.

7.7 Ada Beans

Name	Description
------	-------------

jobHandler	The jobHandler is the bean that is created to execute a job.
------------	--

7.8 Data Model



8 Mail Module

The `mail` module allows an application to format and send a mail to users. This module does not define any web interface. It provides a set of services and methods to send a mail when an event is received. All this is done through configuration. The module defines a set of specific ASF components to format and prepare the email.

8.1 Integration

To be able to use the `mail` module, you will need to add the following line in your GNAT project file:

```
1 with "awa_mail";
```

The `Mail_Module` type represents the mail module. An instance of the mail module must be declared and registered when the application is created and initialized. The module instance can be defined as follows:

```
1 type Application is new AWA.Applications.Application with record
2   Mail_Module : aliased AWA.Mail.Modules.Mail_Module;
3 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,
2           Name    => AWA.Mail.Modules.NAME,
3           Module  => App.Mail_Module'Access);
```

8.2 Configuration

The `mail` module needs some properties to configure the SMTP server.

Configuration	Default	Description
mail.smtp.host	localhost	Defines the SMTP server host name
mail.smtp.port	25	Defines the SMTP connection port
mail.smtp.enable	1	Defines whether sending email is enabled or not

8.3 Sending an email

Sending an email when an event is posted can be done by using an XML configuration. Basically, the `mail` module uses the event framework provided by AWA. The XML definition looks like:

```
1 <on-event name="user-register">
2   <action>#{userMail.send}</action>
3   <property name="template">/mail/register-user-message.xhtml</property>
4 </on-event>
```

With this definition, the mail template `/mail/register-user-message.xhtml` is formatted by using the event and application context when the `user-register` event is posted.

8.4 Components

The `AWA.Mail.Components` package defines several UI components that represent a mail message in an ASF view. The components allow the creation, formatting and sending of a mail message using the same mechanism as the application presentation layer. Example:

```
1 <f:view xmlns="mail:http://code.google.com/p/ada-awa/mail">
2   <mail:message>
3     <mail:subject>Welcome</mail:subject>
4     <mail:to name="Iorek Byrnison">Iorek.Byrnison@svalbard.com</mail:to>
5     <mail:body>
6       ...
7     </mail:body>
8     <mail:attachment value="/images/mail-image.jpg"
9       fileName="image.jpg"
10      contentType="image/jpg"/>
11   </mail:message>
12 </f:view>
```

When the view which contains these components is rendered, a mail message is built and initialized by rendering the inner components. The body and other components can use other application UI components to render useful content. The email is send after the `mail:message` has finished to render its inner children.

The `mail:subject` component describes the mail subject.

The `mail:to` component define the mail recipient. There can be several recepients.

The `mail:body` component contains the mail body.

The `mail:attachment` component allows to include some attachment.

8.4.1 Mail Recipients

The `AWA.Mail.Components.Recipients` package defines the UI components to represent the To, From, Cc and Bcc recipients.

The mail message is retrieved by looking at the parent UI component until a `UIMailMessage` component is found. The mail message recipients are initialized during the render response JSF phase, that is when `Encode_End` are called.

8.4.2 Mail Messages

The `AWA.Mail.Components.Messages` package defines the UI components to represent the email message with its recipients, subject and body.

The mail message is retrieved by looking at the parent UI component until a `UIMailMessage` component is found. The mail message recipients are initialized during the render response JSF phase, that is when `Encode_End` are called.

The `<mail:body>` component holds the message body. This component can include a facet labeled `alternative` in which case it will be used to build the `text/plain` mail message. The default content type for `<mail:body>` is `text/html` but this can be changed by using the `type` attribute.

```
1 <mail:body type='text/html'>
2   <facet name='alternative'>
3     The text/plain mail message.
4   </facet>
5   The text/html mail message.
6 </mail:body>
```

8.4.3 Mail Attachments

The `AWA.Mail.Components.Attachments` package defines the UI components to represent a mail attachment. The mail attachment can be an external file or may be provided by an Ada bean object.

8.5 Ada Beans

Name	Description
userMail	Bean used to send an email with a specific template to the user.

9 Workspaces Module

The `Workspaces` module defines a workspace area for other plugins. The workspace is intended to link together all the data objects that an application manages for a given user or group of users. A workspace is a possible mechanism to provide and implement multi-tenancy in a web application. By using the workspace plugin, application data from different customers can be kept separate from each other in the same database.

9.1 Integration

To be able to use the `Workspaces` module, you will need to add the following line in your GNAT project file:

```
1 with "awa_workspaces";
```

The `Workspace_Module` manages the creation, invitation, administration of members which are allowed to access the application. It provides operations that are used by the other modules to setup and control specific permissions associated to the workspace and granted to workspace members. An instance of the `Workspace_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 with AWA.Workspaces.Modules;  
2 ...  
3 type Application is new AWA.Applications.Application with record  
4   Workspace_Module : aliased AWA.Workspaces.Modules.Workspace_Module;  
5 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,  
2           Name   => AWA.Workspaces.Modules.NAME,  
3           URI    => "workspaces",  
4           Module => App.Workspace_Module'Access);
```

9.2 Permissions

Permissions are defined to control who is allowed to create a workspace, invite members in a workspace for cooperation:

Name	Entity type	Description
workspace-create	awa_workspace	Permission to create a workspace.
workspace-invite-user	awa_workspace	Permission to invite a user in the workspace.
workspace-delete-user	awa_workspace	Permission to delete a user from the workspace.
workspaces-create	awa_workspace	

9.3 Configuration

Name	Description
workspaces.permissions_list	A list of permissions to grant to the user when the workspace is created. blog-create,wiki-space-create
workspaces.allow_workspace_create	Indicates whether new users have a dedicated workspace created when they are created and registered. 0

9.4 Events

The [Workspaces](#) module provides several events that are posted when some action are performed.

9.4.1 invite-user

This event is posted when an invitation is created for a user. The event can be used to send the associated invitation email to the invitee. The event contains the following attributes:

key email name message inviter

9.4.2 accept-invitation

This event is posted when an invitation is accepted by a user.

9.5 Ada Beans

9.5.1 Beans

Name	Description
workspace	This bean allows to perform some general workspace actions
memberList	The list of workspace members.
inviteUser	The invitation bean.
workspaceMember	The workspace member bean.

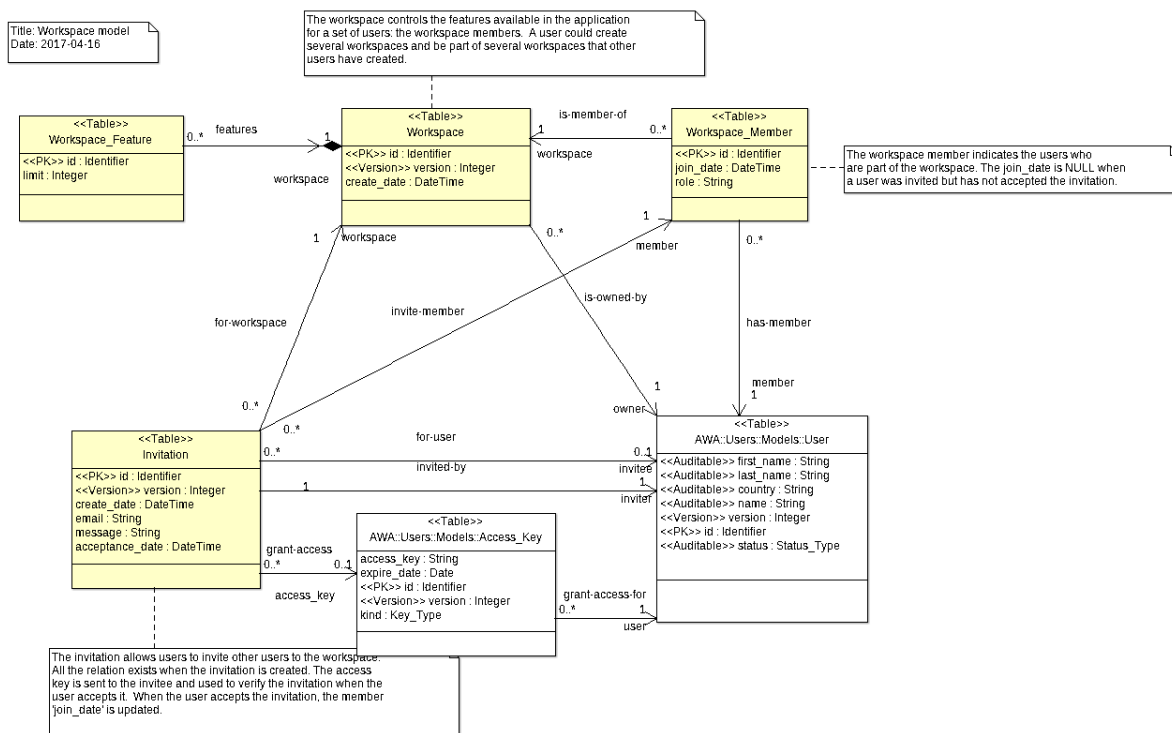
9.5.2 Permissions

Name	Entity type	Description
workspace-create	awa_workspace	Permission to create a workspace.
workspace-invite-user	awa_workspace	Permission to invite a user in the workspace.
workspace-delete-user	awa_workspace	Permission to delete a user from the workspace.
workspaces-create	awa_workspace	

9.5.3 Configuration

Name	Description
workspaces.permissions_list	A list of permissions to grant to the user when the workspace is created. blog-create,wiki-space-create
workspaces.allow_workspace_creation	Indicates whether new users have a dedicated workspace created when they are created and registered.
0	

9.6 Data Model



10 Storages Module

The `storages` module provides a set of storage services allowing an application to store data files, documents, images in a persistent area. The persistent store can be on a file system, in the database or provided by a remote service such as Amazon Simple Storage Service.

10.1 Integration

To be able to use the `storages` module, you will need to add the following line in your GNAT project file:

```
1 with "awa_storages";
```

The `Storage_Module` type represents the storage module. An instance of the storage module must be declared and registered when the application is created and initialized. The storage module is associated with the storage service which provides and implements the storage management operations. An instance of the `Storage_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 with AWA.Storages.Modules;  
2 ...  
3 type Application is new AWA.Applications.Application with record  
4   Storage_Module : aliased AWA.Storages.Modules.Storage_Module;  
5 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,  
2          Name    => AWA.Storages.Modules.NAME,  
3          Module  => App.Storage_Module'Access);
```

10.2 Permissions

Name	Entity type	Description
folder-create	awa_workspace	This permission grants the creation of storage folder for the given user in a specific workspace.
storage-create	awa_workspace	This permission grants the creation of storage file for the given user in a specific workspace.

Name	Entity type	Description
storage-delete	awa_workspace	This permission grants the deletion of storage files for the given user in a specific workspace.

10.3 Configuration

The `storages` module defines the following configuration parameters:

Name	Description
storages.storage_root	The path of the directory that contains storage files stored on the local filesystem. storage
storages.tmp_storage_root	The path of the directory that contains temporary storage files on the local filesystem. tmp
storages.database_max_size	The maximum size of documents store in the database storage. 100000

10.4 Creating a storage

A data in the storage is represented by a `Storage_Ref` instance. The data itself can be physically stored in a file system (`FILE` mode), in the database (`DATABASE` mode) or on a remote server (`URL` mode). To put a file in the storage space, first create the storage object instance:

```
1 Data : AWA.Storages.Models.Storage_Ref;
```

Then setup the storage mode that you want. The storage service uses this information to save the data in a file, in the database or in a remote service (in the future). To save a file in the store, we can use the `Save` operation of the storage service. It will read the file and put in in the corresponding persistent store (the database in this example).

```
1 Service.Save (Into => Data, Path => Path_To_The_File,  
2             Storage => AWA.Storages.Models.DATABASE);
```

Upon successful completion, the storage instance `Data` will be allocated a unique identifier that can be retrieved by `Get_Id` or `Get_Key`.

10.5 Getting the data

Several operations are defined to retrieve the data. Each of them has been designed to optimize the retrieval and

- The data can be retrieved in a local file. This mode is useful if an external program must be launched and be able to read the file. If the storage mode of the data is `FILE`, the path of the file on the storage file system is used. For other storage modes, the file is saved in a temporary file. In that case the `Store_Local` database table is used to track such locally saved data.
- The data can be returned as a stream. When the application has to read the data, opening a read stream connection is the most efficient mechanism.

10.6 Local file

To access the data by using a local file, we must define a local storage reference:

```
1 Data : AWA.Storages.Models.Store_Local_Ref;
```

and use the `Load` operation with the storage identifier. When loading locally we also indicate whether the file will be read or written. A file that is in `READ` mode can be shared by several tasks or processes. A file that is in `WRITE` mode will have a specific copy for the caller. An optional expiration parameter indicate when the local file representation can expire.

```
1 Service.Load (From => Id, Into => Data, Mode => READ, Expire => ONE_DAY);
```

Once the load operation succeeded, the data is stored on the file system and the local path is obtained by using the `Get_Path` operation:

```
1 Path : constant String := Data.Get_Path;
```

10.7 Storage Service

The `Storage_Service` provides the operations to access and use the persistent storage. It controls the permissions that grant access to the service for users.

Other modules can be notified of storage changes by registering a listener on the storage module.

10.8 Store Service

The `AWA.Storages.Stores` package defines the interface that a store must implement to be able to save and retrieve a data content. The store can be a file system, a database or a remote store service.

10.8.1 Database store

The `AWA.Storages.Stores.Databases` store uses the database to save a data content. The data is saved in a specific table in a database blob column. The database store uses another store service to temporarily save the data content in a local file when the application needs a file access to the data.

10.8.2 File System store

The `AWA.Storages.Stores.Files` store uses the file system to save a data content. Files are stored in a directory tree whose path is created from the workspace identifier and the storage identifier. The layout is such that files belonged to a given workspace are stored in the same directory sub-tree.

The root directory of the file system store is configured through the `storage_root` and `tmp_storage_root` configuration properties.

10.9 Ada Beans

Name	Description
storageFolder	This bean allows to create a storage folder.
uploadFile	This bean allows to upload a new file in the storage space.
folderList	This bean gives the list of storage folders in the workspace.
storageList	This bean gives the list of storage files associated with a given folder.
storageInfo	This bean gives some information about a document and its folder.

10.9.0.1 AWA.Storages.Models.Storage_Info The list of documents for a given folder.

Type	Ada	Name	Description
	Identifier	id	the storage identifier.
	String	name	the file name.
	Date	create_date	the file creation date.
	String	uri	the file storage URI.
	AWA.Storage.Models.StorageType	storage_type	the file storage URI.
	String	mime_type	the file mime type.
	Integer	file_size	the file size.
	Boolean	is_public	whether the document is public or not.
	String	user_name	the user name who uploaded the document.
	Integer	thumb_width	the image thumbnail width (or 0).
	Integer	thumb_height	the image thumbnail height (or 0).
	Identifier	thumbnail_id	the image thumbnail identifier.

10.9.0.2 AWA.Storage.Models.Folder_Info The list of folders.

Type	Ada	Name	Description
	Identifier	id	the folder identifier.
	String	name	the folder name.
	Date	create_date	the blog creation date.

10.10 Storage Servlet

The Storage_Servlet type is the servlet that allows to retrieve the file content that was uploaded.

10.11 Queries

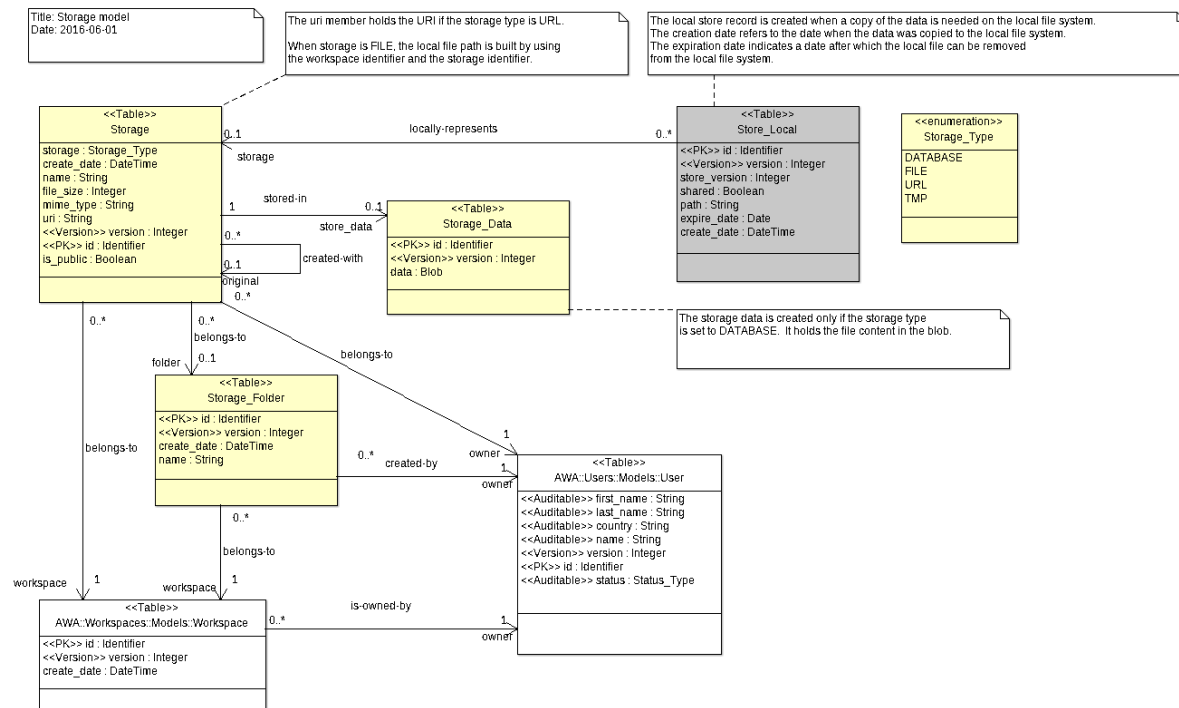
Name	Description
storage-list	Get a list of storage files for a given folder.

Name	Description
storage-folder-list	Get a list of storage folders that a user can see.

Name	Description
storage-get-data	Get the data content of the storage object.
storage-get-local	Get the local data storage that can be used to read locally a storage object.
storage-get-storage	Get the local data storage that can be used to read locally a storage object.
storage-delete-local	Delete the local storage data

Name	Description
storage-info	Get the description of a document.

10.12 Data model



11 Images Module

The `images` module is an extension of the Storages Module that identifies images and provides thumbnails as well as resizing of the original image.

The `images` module uses several other modules:

- the Storage Module to store and manage image content,
- the Jobs Module to schedule image thumbnail generation.

11.1 Integration

To be able to use the `Images` module, you will need to add the following line in your GNAT project file:

```
1 with "awa_images";
```

The `Image_Module` type represents the image module. An instance of the image module must be declared and registered when the application is created and initialized. The image module is associated with the image service which provides and implements the image management operations.

```
1 with AWA.Images.Modules;  
2 ...  
3 type Application is new AWA.Applications.Application with record  
4   Image_Module : aliased AWA.Images.Modules.Image_Module;  
5 end record;
```

And it is registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,  
2           Name   => AWA.Images.Modules.NAME,  
3           Module => App.Image_Module'Access);
```

When the image module is initialized, it registers itself as a listener to the storage module to be notified when a storage file is created, updated or removed. When a file is added, it looks at the file type and extracts the image information if the storage file is an image.

11.2 Configuration

The `Images` module defines the following configuration parameters:

Name	Description
images.thumbnail_command	The command to execute to generate an image thumbnail for the Images module. convert -verbose -resize #{width}x#{height} -background white -gravity center -extent #{width}x#{height} -format jpg -quality 75 #{src} #{dst}

11.3 Ada Beans

The `Image_List_Bean` type is used to represent a list of image stored in a folder.

The `Image_Bean` type holds all the data to give information about an image.

Name	Description
storageFolder	This bean allows to create a storage folder.
imageList	This bean gives the list of images associated with a given folder.
imageInfo	This bean gives the information about an image.

11.3.0.1 AWA.Images.Models.Image_Bean The information about an image.

Type	Ada	Name	Description
	Identifier	folder_id	the image folder identifier.
	String	folder_name	the image folder name.
	Identifier	id	the image file identifier.
	String	name	the image file name.
	Date	create_date	the file creation date.
	String	uri	the file storage URI.
	AWA.Storage.Models.Storage_Type	storage_type	the file storage URI.
	String	mime_type	the file mime type.
	Integer	file_size	the file size.

Type	Ada	Name	Description
	Boolean	is_public	whether the image is public.
	Integer	width	the image width.
	Integer	height	the image height.

11.3.0.2 AWA.Images.Models.Image_Info The list of images for a given folder.

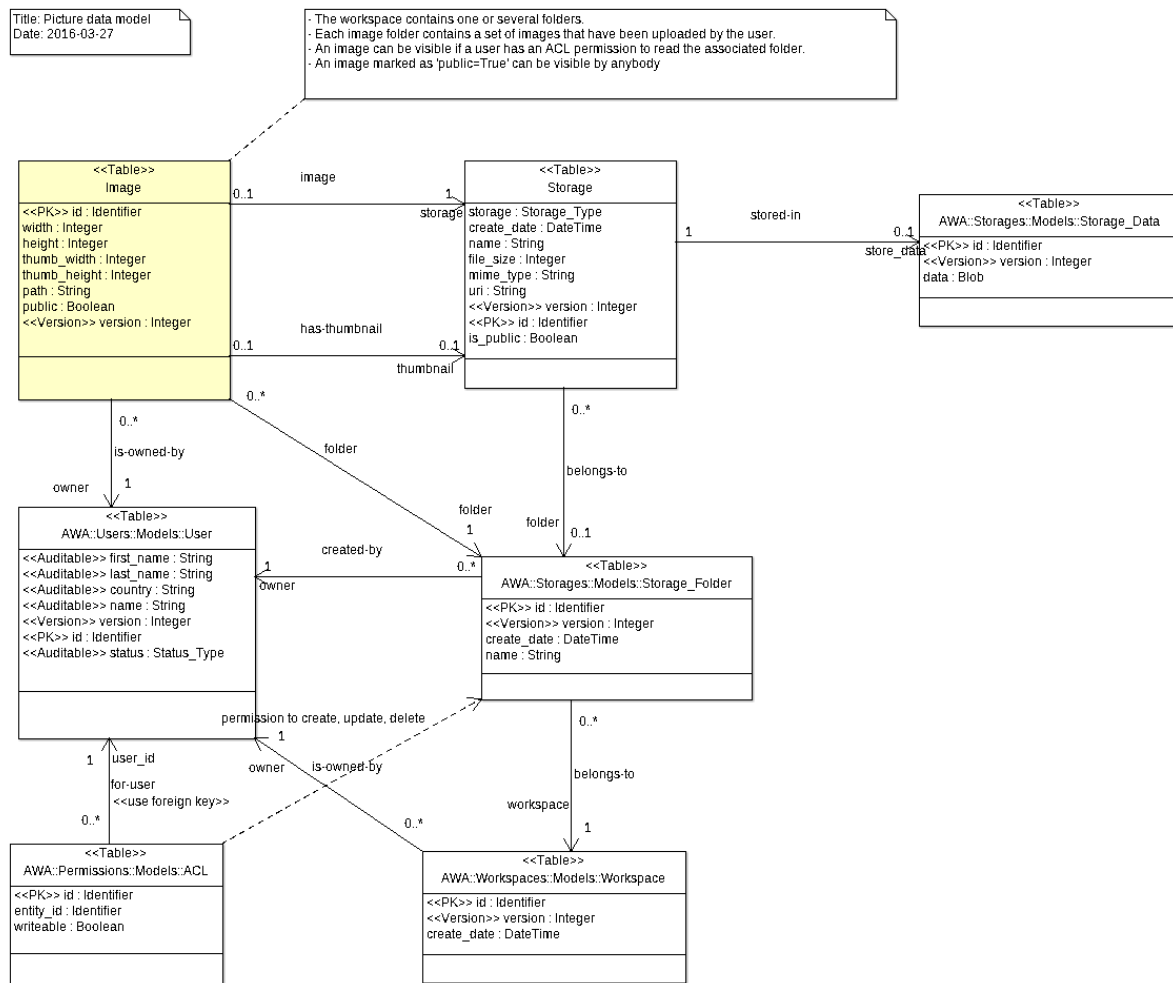
Type	Ada	Name	Description
	Identifier	id	the storage identifier which contains the image data.
	String	name	the image file name.
	Date	create_date	the image file creation date.
	String	uri	the image file storage URI.
	Integer	storage	the image file storage URI.
	String	mime_type	the image file mime type.
	Integer	file_size	the image file size.
	Integer	width	the image width.
	Integer	height	the image height.
	Integer	thumb_width	the image thumbnail width.
	Integer	thumb_height	the image thumbnail height.
	Identifier	thumbnail_id	the image thumbnail identifier.

11.4 Queries

Name	Description
image-info	Get the description of an image.

Name	Description
image-list	Get a list of images for a given folder.

11.5 Data model



12 Wikis Module

The `Wikis` module provides a complete wiki system which allows users to create their own wiki environment with their wiki pages.

12.1 Integration

To be able to use the `Wikis` module, you will need to add the following line in your GNAT project file:

```
1 with "awa_wikis";
```

The `Wiki_Module` manages the creation, update, removal of wiki pages in an application. It provides operations that are used by the wiki beans or other services to create and update wiki pages. An instance of the `Wiki_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 with AWA.Wikis.Modules;
2 ...
3 type Application is new AWA.Applications.Application with record
4   Wiki_Module : aliased AWA.Wikis.Modules.Wiki_Module;
5 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,
2           Name    => AWA.Wikis.Modules.NAME,
3           URI     => "wikis",
4           Module  => App.Wiki_Module'Access);
```

12.2 Permissions

Permissions are defined to control who is allowed to create, update and delete wiki pages:

Name	Entity type	Description
wiki-space-create	awa_workspace	Grants the creation of a wiki space for the given user in a specific workspace.
wiki-space-update	awa_wiki_space	Grants the edit permission on the wiki space owner only

Name	Entity type	Description
wiki-space-delete	awa_wiki_space	Granta the delete permission on the wiki space owner only
wiki-page-view	awa_wiki_space	Grant the view wiki page permission when the page is public or when an ACL link exist between the user and the wiki space
wiki-page-create	awa_wiki_space	Grant the create wiki page permission when an ACL link exist between the user and the wiki space
wiki-page-update	awa_wiki_space	Grants the permission to edit the wiki page to the given user and for the given wiki space.
wiki-page-delete	awa_wiki_space	Grants the permission to delete the wiki page to the given user and for the given wiki space.

12.3 Configuration

Name	Description
wikis.image_prefix	The URL base prefix to be used for Wiki images. <code>#{contextPath}/wikis/images/</code>
wikis.page_prefix	The URL base prefix to be used for Wiki pages. <code>#{contextPath}/wikis/view/</code>
wikis.wiki_copy_list	A list of wiki page ID top copy when a new wiki space is created.
wikis.page_uri	The wiki page URI to write in generated sitemaps. The URI must be absolute and specify the host and domain. This EL expression is evaluated twice: a first time during application setup and a second time for each URI that must be generated. The wiki space ID is passed in the <code>#{wikiSpaceId}</code> and the wiki page name in <code>#{wikiPage}</code> (be careful to escape the '#' by using <code>#{wikiSpaceId}</code> due to the double EL evaluation). <code>#{app_url_base}/wikis/view/#{wikiSpaceId}/#{wikiPage}</code>

12.4 Events

The `wikis` exposes a number of events which are posted when some action are performed at the service level.

Event name	Description
wiki-create-page	This event is posted when a new wiki page is created.
wiki-create-content	This event is posted when a new wiki page content is created. Each time a wiki page is modified, a new wiki page content is created and this event is posted.

12.5 Ada Beans

Several bean types are provided to represent and manage the blogs and their posts. The blog module registers the bean constructors when it is initialized. To use them, one must declare a bean definition in the application XML configuration.

Name	Description
wikiView	The wiki page with all its information to display it.
wikiImageInfo	The information about an image used by a wiki page.
wikiPageInfo	The wiki page information bean gives the various statistics and information about a wiki page.
wikiFormatList	A localized list of wiki page formats to be used for a <code>f:selectItems</code>
adminWiki	The list of wikis and pages that the current user can access and update.
adminWikiSpace	The wiki space bean to create and edit the wiki space configuration.
wikiPage	The wiki page bean gives the full content and information about a wiki page.
wikiList	The list of wiki pages.
wikiVersionList	The list of wiki page versions.
wikiTagSearch	The wiki tag search bean.
wikiTagCloud	The list of tags associated with a wiki page entities.
wikiTags	The wiki tag editor bean.

Name	Description
wikiPageStats	The counter statistics for a wiki page

12.5.0.1 AWA.Wikis.Models.Wiki_View_Info The information about a wiki page.

Type	Ada	Name	Description
Identifier	id		the wiki page identifier.
String	name		the wiki page name.
String	title		the wiki page title.
Boolean	is_public		whether the wiki is public.
Nullable_Integer	version		the last version.
Nullable_Integer	page_count		the number of times the page was displayed.
Nullable_Date	date		the wiki page creation date.
AWA.Wikis.Models.Format_Type	format		the wiki page format.
String	content		the wiki page content.
String	save_comment		the wiki version comment.
String	left_side		the wiki page left side panel.
String	right_side		the wiki page right side panel.
AWA.Wikis.Models.Format_Type	side_format		the wiki side format.
String	author		the wiki page author.
Identifier	acl_id		the acl Id if there is one.

12.5.0.2 AWA.Wikis.Models.Wiki_Page_Info The information about a wiki page.

Type	Ada	Name	Description
Identifier	id		the wiki page identifier.
String	name		the wiki page name.
String	title		the wiki page title.

Type	Ada	Name	Description
	Boolean	is_public	whether the wiki is public.
	Integer	last_version	the last version.
	Integer	read_count	the read count.
	Date	create_date	the wiki creation date.
	String	author	the wiki page author.

12.5.0.3 AWA.Wikis.Models.Wiki_Version_Info The information about a wiki page version.

Type	Ada	Name	Description
	Identifier	id	the wiki page identifier.
	String	comment	the wiki page version comment.
	Date	create_date	the wiki page creation date.
	Integer	page_version	the page version.
	String	author	the wiki page author.

12.5.0.4 AWA.Wikis.Models.Wiki_Info The list of wikis.

Type	Ada	Name	Description
	Identifier	id	the wiki space identifier.
	String	name	the wiki name.
	Boolean	is_public	whether the wiki is public.
	Date	create_date	the wiki creation date.
	Integer	page_count	the number of pages in the wiki.

12.6 Queries

Name	Description
wiki-page	Get the content of a wiki page.
wiki-page-id	Get the content of a wiki page.
wiki-page-content	Get only the content of a wiki page (for template evaluation).
wiki-page-name-count	Count the occurrence of a wiki page name

Name	Description
wiki-page-list	Get the list of wiki pages
wiki-page-tag-list	Get the list of wiki pages filtered by a tag

Name	Description
wiki-version-list	Get the list of wiki page versions

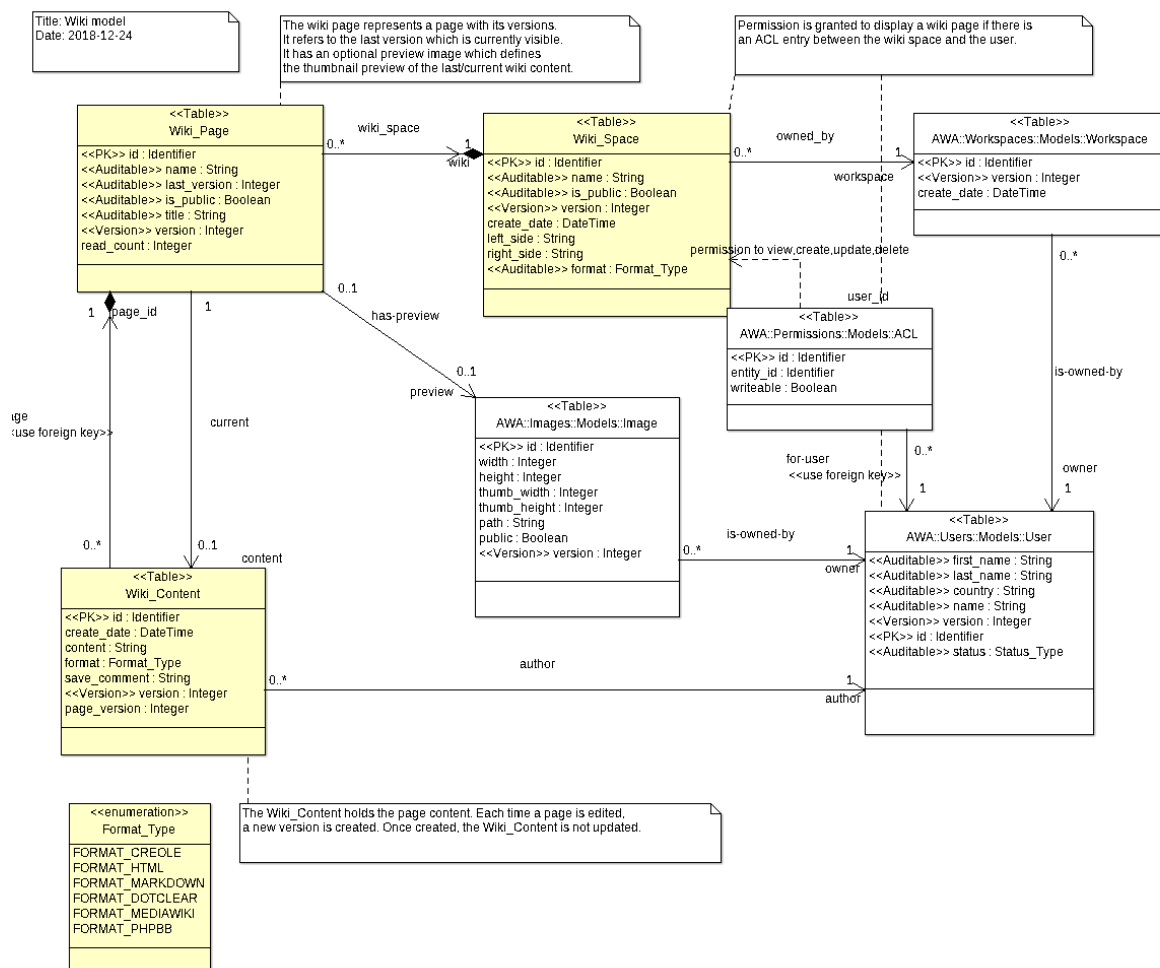
Name	Description
wiki-list	Get the list of wikis that the current user can update

Name	Description
wiki-image-get-data	Get the data content of the Wiki image (original image).
wiki-image-width-get-data	Get the data content of the Wiki image for an image with a given width.
wiki-image-height-get-data	Get the data content of the Wiki image for an image with a given height.

Name	Description
wiki-image	Get the description of an image used in a wiki page.

Name	Description
page-access-stats	Get statistics about the wiki page access.

12.7 Data model



13 Blogs Module

The `blogs` module is a small blog application which allows users to publish articles. A user may own several blogs, each blog having a name and its own base URI. Within a blog, the user may write articles and publish them. Once published, the articles are visible to anonymous users.

The `blogs` module uses several other modules:

- the Counters Module to track page display counter to a blog post,
- the Tags Module to associate one or several tags to a blog post,
- the Comments Module to allow users to write comments on a blog post,
- the Images Module to easily add images in blog post.
- the SEO Module for the generation of sitemap entries for the blog posts.

13.1 Integration

To be able to use the `Blogs` module, you will need to add the following line in your GNAT project file:

```
1 with "awa_blogs";
```

The `Blog_Module` type manages the creation, update, removal of blog posts in an application. It provides operations that are used by the blog beans or other services to create and update posts. An instance of the `Blog_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 with AWA.Blogs.Modules;  
2 ...  
3 type Application is new AWA.Applications.Application with record  
4   Blog_Module : aliased AWA.Blogs.Modules.Blog_Module;  
5 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,  
2           Name   => AWA.Blogs.Modules.NAME,  
3           URI    => "blogs",  
4           Module => App.Blog_Module'Access);
```

13.2 Permissions

Permissions are defined to control who is allowed to create, update and delete blog posts:

Name	Entity type	Description
blog-create	awa_workspace	Permission to create a new blog.
blog-delete	awa_blog	Permission to delete a blog.
blog-create-post	awa_blog	Permission to create a new post.
blog-update-post	awa_blog	Permission to modify a post.
blog-delete-post	awa_blog	Permission to delete a post.
blog-add-comment	awa_blog	
blog-publish-comment	awa_blog	Permission to change the publish status of a comment.
blog-delete-comment	awa_blog	Permission to delete the comment.

13.3 Configuration

The **Blogs** module defines the following configuration parameters:

Name	Description
blogs.image_prefix	The URL base prefix to be used for blog post images. <code>#{contextPath}/blogs/images/</code>
blogs.post_uri	The post URI to write in generated sitemaps. The URI must be absolute and specify the host and domain. This EL expression is evaluated twice: a first time during application setup and a second time for each URI that must be generated. The blog post base URI is defined in the variable <code>#{uri}</code> (be careful to escape the '#' by using <code>#{uri}</code> due to the double EL evaluation). <code>#{app_url_base}/blogs/posts/#{uri}</code>

Name	Description
blogs.image_uri	The image URI to write in generated sitemaps. Similar to the blogs.post_uri, this must be absolute and specify the host and domain. This EL expression is evaluated twice: a first time during application setup and a second time for each URI that must be generated. The blog post base URI is defined in the variable <code>#{post_id}</code> (be careful to escape the '#' by using <code>#{post_id}</code> due to the double EL evaluation). <code>#{app_url_base}/blogs/images/#{post_id}/#{image_id}/default/#{image_title}</code>

13.4 Ada Beans

Several bean types are provided to represent and manage the blogs and their posts. The blog module registers the bean constructors when it is initialized. To use them, one must declare a bean definition in the application XML configuration.

Name	Description
post	This bean describes a blog post for the creation or the update
postList	This bean describes a blog post for the creation or the update
postStatusList	A localized list of post statuses to be used for a <code>f:selectItems</code>
postAccessStats	The counter statistics for a blog post
blogFormatList	A localized list of blog post formats to be used for a <code>f:selectItems</code>
adminBlog	The list of blogs and posts that the current user can access and update.
blog	Information about the current blog.
feed_blog	Information about the RSS feed blog.
blogTagSearch	The blog tag search bean.
blogTagCloud	A list of tags associated with all post entities.
postComments	A list of comments associated with a post.
postAdminComments	A list of all comments associated with a post (for admin purposes).

Name	Description
postNewComment	The bean to allow a user to post a new comment. This is a specific bean because the permission to create a new post is different from other permissions. If the permission is granted, the comment will be created and put in the COMMENT_WAITING state. An email will be sent to the post author for approval.
blogPublishComment	The bean to allow the blog administrator to change the publication status of a comment.
blogDeleteComment	The bean to allow the blog administrator to change the publication status of a comment.
commentEdit	The bean to allow a user to edit the comment. This is a specific bean because the permission to edit the comment is different from other permissions.
blogStats	This bean provides statistics about the blog

13.4.0.1 AWA.Blogs.Models.Admin_Post_Info The Admin_Post_Info describes a post in the administration interface.

Type	Ada	Name	Description
	Identifier	id	the post identifier.
	String	title	the post title.
	String	uri	the post uri.
	Date	date	the post publish date.
	AWA.Blogs.Models.Post_Status_Type	status	the post status.
	Natural	read_count	the number of times the post was read.
	String	username	the user name.
	Natural	comment_count	the number of comments for this post.

13.4.0.2 AWA.Blogs.Models.Post_Info The Post_Info describes a post to be displayed in the blog page

Type	Ada	Name	Description
	Identifier	id	the post identifier.
	String	title	the post title.
	String	uri	the post uri.
	Date	date	the post publish date.
	String	username	the user name.
	AWA.Blogs.Models.Format_Type	format_type	the post page format.
	String	summary	the post summary.
	String	text	the post text.
	Boolean	allow_comments	the post allows to add comments.
	Natural	comment_count	the number of comments for this post.

13.4.0.3 AWA.Blogs.Models.Comment_Info The comment information.

Type	Ada	Name	Description
	Identifier	id	the comment identifier.
	Identifier	post_id	the post identifier.
	String	title	the post title.
	String	author	the comment author's name.
	String	email	the comment author's email.
	Date	date	the comment date.
	AWA.Comments.Models.Format_Type	format_type	the comment format type.
	String	comment	the comment text.
	AWA.Comments.Models.Status_Type	status	the comment status.

13.4.0.4 AWA.Blogs.Models.Blog_Info The list of blogs.

Type	Ada	Name	Description
	Identifier	id	the blog identifier.
	String	title	the blog title.
	String	uid	the blog uuid.
	Date	create_date	the blog creation date.
	Integer	post_count	the number of posts published.

13.5 Queries

Name	Description
blog-admin-post-list	Get the list of blog posts
blog-admin-post-list-date	Get the list of blog posts

Name	Description
blog-post-list	Get the list of public visible posts
blog-post-tag-list	Get the list of public visible posts filtered by a tag

Name	Description
comment-list	Get the list of comments associated with given database entity

Name	Description
blog-list	Get the list of blogs that the current user can update

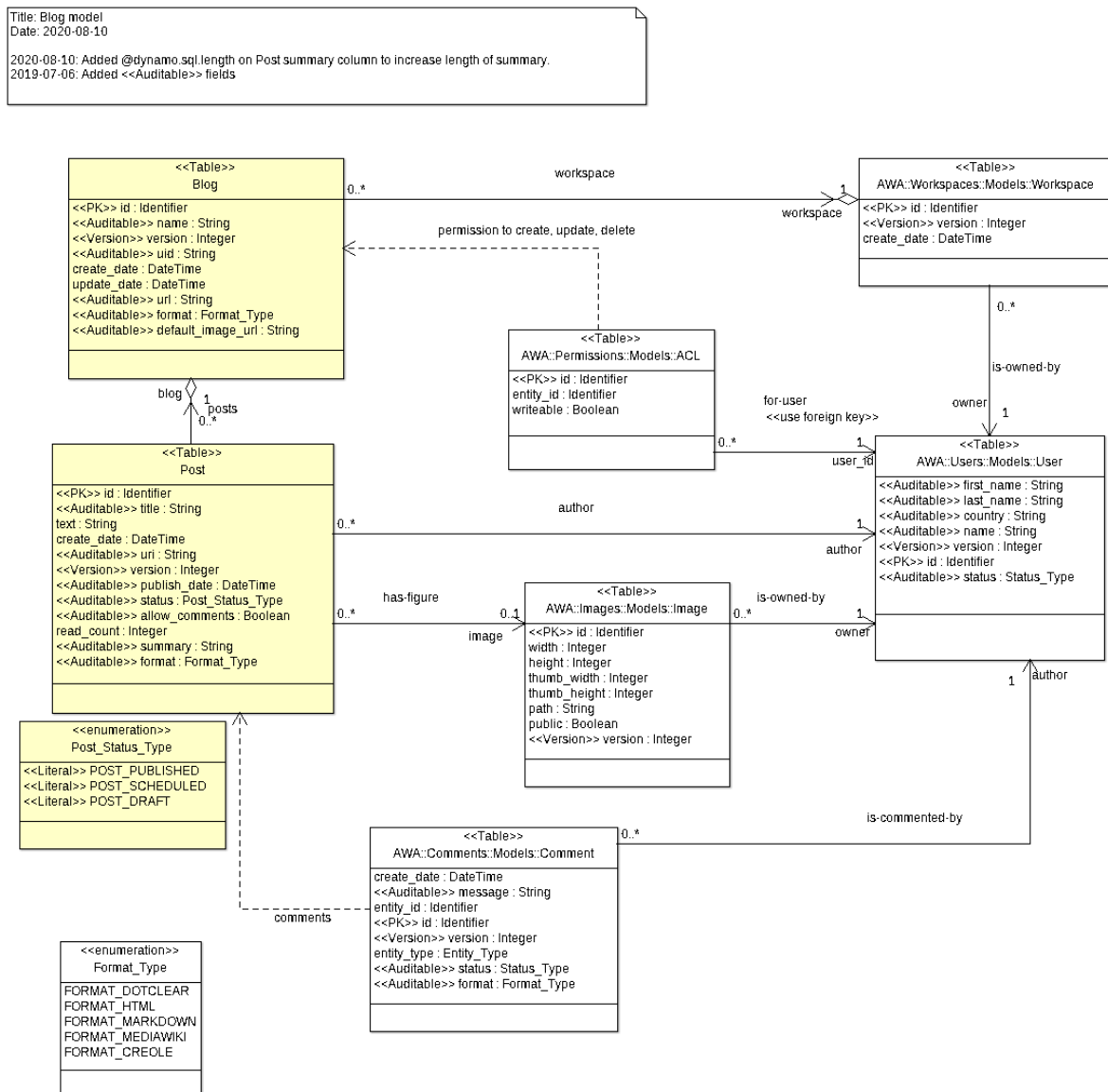
Name	Description
blog-tag-cloud	Get the list of tags associated with all the database entities of a given type

Name	Description
blog-image-get-data	Get the data content of the Wiki image (original image).
blog-image-width-get-data	Get the data content of the Wiki image for an image with a given width.
blog-image-height-get-data	Get the data content of the Wiki image for an image with a given height.

Name	Description
blog-image	Get the description of an image used in a blog post.

Name	Description
post-publish-stats	Get statistics about the post publication on a blog.
post-access-stats	Get statistics about the post publication on a blog.

13.6 Data model



14 Counters Module

The `counters` module defines a general purpose counter service that allows to associate counters to database entities. For example it can be used to track the number of times a blog post or a wiki page is accessed. The `counters` module maintains the counters in a table on a per-day and per-entity basis. It allows to update the full counter in the target database entity table.

14.1 Integration

The `Counter_Module` manages the counters associated with database entities. To avoid having to update the database each time a counter is incremented, counters are kept temporarily in a `Counter_Table` protected type. The table contains only the partial increments and not the real counter values. Counters are flushed when the table reaches some limit, or, when the table is oldest than some limit. Counters are associated with a day so that it becomes possible to gather per-day counters. The table is also flushed when a counter is incremented in a different day.

To be able to use the `Counters` module, you will need to add the following line in your GNAT project file:

```
1 with "awa_counters";
```

An instance of the `Counter_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 with AWA.Counters.Modules;  
2 ...  
3 type Application is new AWA.Applications.Application with record  
4   Counter_Module : aliased AWA.Counters.Modules.Counter_Module;  
5 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,  
2           Name   => AWA.Counters.Modules.NAME,  
3           Module => App.Counter_Module'Access);
```

14.2 Configuration

The `counters` module defines the following configuration parameters:

Name	Description
counters.counter_age_limit	The maximum age limit in seconds for a pending counter increment to stay in the internal table. When a pending counter reaches this age limit, the pending counter increments are flushed and the table is cleared. The default is 5 minutes. 300
counters.counter_limit	The maximum number of different counters which can be stored in the internal table before flushing the pending increments to the database. When this limit is reached, the pending counter increments are flushed and the table is cleared. 1000

14.3 Counter Declaration

Each counter must be declared by instantiating the `Definition` package. This instantiation serves as identification of the counter and it defines the database table as well as the column in that table that will hold the total counter. The following definition is used for the read counter of a wiki page. The wiki page table contains a `read_count` column and it will be incremented each time the counter is incremented.

```
1 with AWA.Counters.Definition;  
2 ...  
3 package Read_Counter is  
4     new AWA.Counters.Definition  
5     (AWA.Wikis.Models.WIKI_PAGE_TABLE, "read_count");
```

When the database table does not contain any counter column, the column field name is not given and the counter definition is defined as follows:

```
1 with AWA.Counters.Definition;  
2 ...  
3 package Login_Counter is  
4     new AWA.Counters.Definition (AWA.Users.Models.USER_PAGE_TABLE);
```

Sometimes a counter is not associated with any database entity. Such counters are global and they are assigned a unique name.

```
1 with AWA.Counters.Definition;  
2 ...  
3 package Start_Counter is
```

```
4      new AWA.Counters.Definition (null, "startup_counter");
```

14.4 Incrementing the counter

Incrementing the counter is done by calling the `Increment` operation. When the counter is associated with a database entity, the entity primary key must be given. The counter is not immediately incremented in the database so that several calls to the `Increment` operation will not trigger a database update.

```
1  with AWA.Counters;  
2  ...  
3  AWA.Counters.Increment (Counter => Read_Counter.Counter, Key => Id);
```

A global counter is also incremented by using the `Increment` operation.

```
1  with AWA.Counters;  
2  ...  
3  AWA.Counters.Increment (Counter => Start_Counter.Counter);
```

14.5 Ada Bean

The `Counter_Bean` allows to represent a counter associated with some database entity and allows its control by the `<awa:counter>` HTML component. To use it, an instance of the `Counter_Bean` should be defined in a another Ada bean declaration and configured. For example, it may be declared as follows:

```
1  type Wiki_View_Bean is new AWA.Wikis.Models.Wiki_View_Info  
2  with record  
3      ...  
4      Counter : aliased Counter_Bean  
5          (Of_Type => ADO.Objects.KEY_INTEGER,  
6            Of_Class => AWA.Wikis.Models.WIKI_PAGE_TABLE);  
7  end record;
```

The counter value is held by the `Value` member of `Counter_Bean` and it should be initialized programatically when the Ada bean instance is loaded (for example through a `load` action). The `Counter_Bean` needs to know the database entity to which it is associated and its `Object` member must be initialized. This is necessary for the `<awa:counter>` HTML component to increment the associated counter when the page is displayed. Below is an extract of such initialization:

```
1  procedure Load  
2      (Bean      : in out Wiki_View_Bean;  
3       Outcome  : in out Ada.Strings.Unbounded.Unbounded_String) is
```



```
4 begin
5   ...
6   Bean.Counter.Value := Bean.Get_Read_Count;
7   ADO.Objects.Set_Value (Bean.Counter.Object, Bean.Get_Id);
8 end Load;
```

The `Stat_List_Bean` allows to retrieve the list of counters per day for a given database entity. It needs a special managed bean configuration that describes the database entity type, the counter name and SQL query name.

The example below from the Wikis Module declares the bean `wikiPageStats`. The database entity is `awa_wiki_page` which is the name of the database table that holds wiki page. The SQL query to retrieve the result is `page-access-stats`.

```
1 <description>The counter statistics for a wiki page</description>
2 <managed-bean-name>wikiPageStats</managed-bean-name>
3 <managed-bean-class>AWA.Counters.Beans.Stat_List_Bean</managed-bean-
  class>
4 <managed-bean-scope>request</managed-bean-scope>
5 <managed-property>
6   <property-name>entity_type</property-name>
7   <property-class>String</property-class>
8   <value>awa_wiki_page</value>
9 </managed-property>
10 <managed-property>
11   <property-name>counter_name</property-name>
12   <property-class>String</property-class>
13   <value>read_count</value>
14 </managed-property>
15 <managed-property>
16   <property-name>query_name</property-name>
17   <property-class>String</property-class>
18   <value>page-access-stats</value>
19 </managed-property>
20 </managed-bean>
```

A typical XHTML view that wants to use such bean, should call the `load` action at beginning to load the counter statistics by running the SQL query.

```
1 <f:view contentType="application/json; charset=UTF-8"
2   xmlns:f="http://java.sun.com/jsf/core"
3   xmlns:h="http://java.sun.com/jsf/html">
4   <f:metadata>
5     <f:viewAction action='#{wikiPageStats.load}'/>
6   </f:metadata>
7   {"data":<h:list value="#{wikiPageStats.stats}"
8     var="stat">["#{stat.date}", #{stat.count}],</h:list>[0,0]]}
9 </f:view>
```

14.6 HTML components

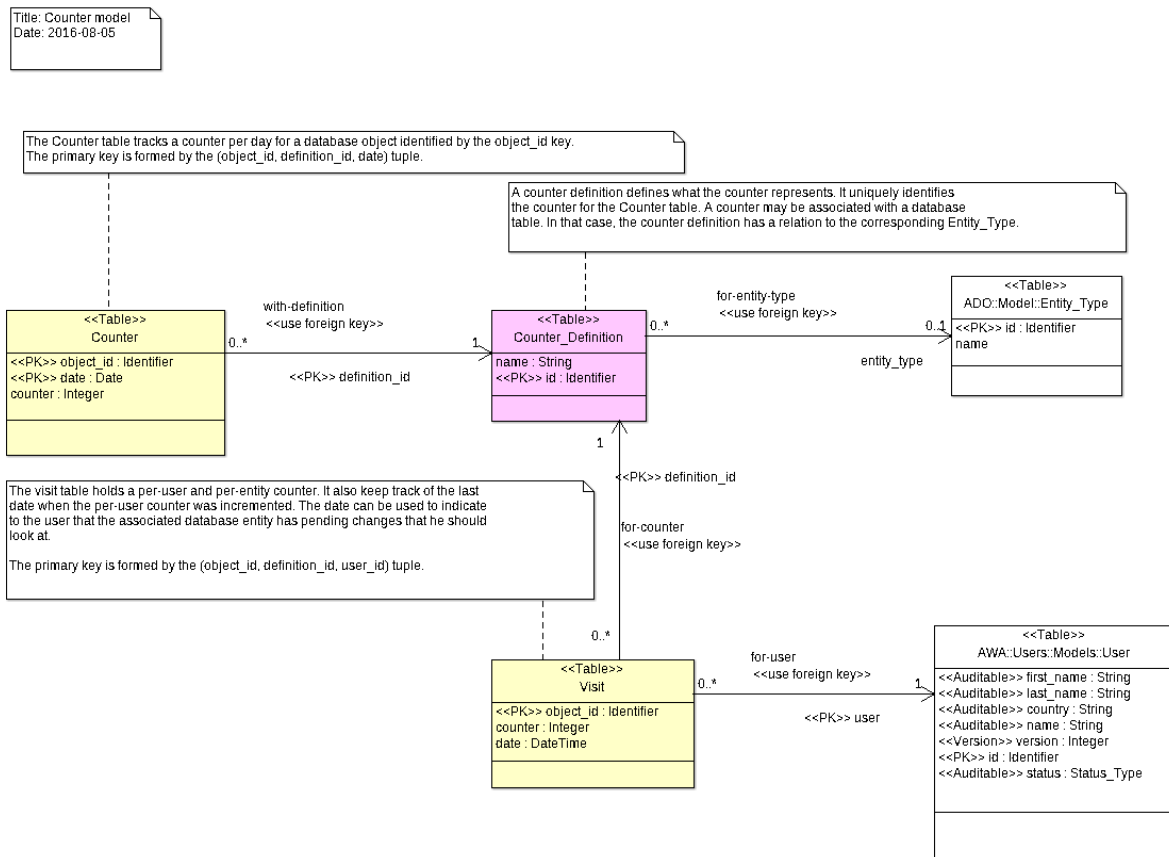
The `<awa:counter>` component is an Ada Server Faces component that allows to increment and display easily the counter. The component works by using the `Counter_Bean` Ada bean object which describes the counter in terms of counter definition, the associated database entity, and the current counter value.

```
1 <awa:counter value="#{wikiPage.counter}"/>
```

When the component is included in a page the `Counter_Bean` instance associated with the EL `value` attribute is used to increment the counter. This is similar to calling the `AWA.Counters.Increment` operation from the Ada code.

14.7 Data model

The `counters` module has a simple database model which needs two tables. The `Counter_Definition` table is used to keep track of the different counters used by the application. A row in that table is created for each counter declared by instantiating the `Definition` package. The `Counter` table holds the counters for each database entity and for each day. By looking at that table, it becomes possible to look at the daily access or usage of the counter.



15 Votes Module

The `votes` module allows users to vote for objects defined in the application. Users can vote by setting a rating value on an item (+1, -1 or any other integer value). The `votes` module makes sure that users can vote only once for an item. A global rating is associated with the item to give the vote summary. The vote can be associated with any database entity and it is not necessary to change other entities in your data model.

15.1 Integration

To be able to use the `votes` module, you will need to add the following line in your GNAT project file:

```
1 with "awa_votes";
```

The `Vote_Module` manages the votes on entities. It provides operations that are used by the vote beans or other services to vote for an item. An instance of the `Vote_Module` must be declared and registered in the AWA application.

The module instance can be defined as follows:

```
1 type Application is new AWA.Applications.Application with record
2   Vote_Module : aliased AWA.Votes.Modules.Vote_Module;
3 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,
2           Name    => AWA.Votes.Modules.NAME,
3           URI     => "votes",
4           Module  => App.Vote_Module'Access);
```

15.2 Ada Beans

The `Vote_Bean` is a bean intended to be used in presentation files (XHTML facelet files) to vote for an item. The managed bean can be easily configured in the application XML configuration file. The `permission` and `entity_type` are the two properties that should be defined in the configuration. The `permission` is the name of the permission that must be used to verify that the user is allowed to vote for the item. The `entity_type` is the name of the entity (table name) used by the item. The example below defines the bean `questionVote` defined by the question module.

```
1 <managed-bean>
```

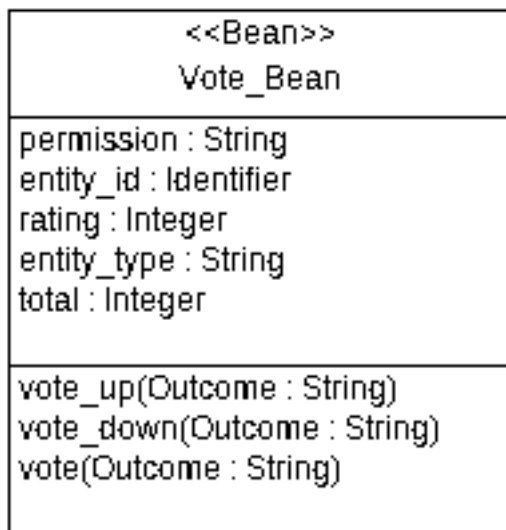
```

2   <description>The vote bean that allows to vote for a question.</
    description>
3   <managed-bean-name>questionVote</managed-bean-name>
4   <managed-bean-class>AWA.Votes.Beans.Votes_Bean</managed-bean-class>
5   <managed-bean-scope>request</managed-bean-scope>
6   <managed-property>
7       <property-name>permission</property-name>
8       <property-class>String</property-class>
9       <value>answer-create</value>
10  </managed-property>
11  <managed-property>
12      <property-name>entity_type</property-name>
13      <property-class>String</property-class>
14      <value>awa_question</value>
15  </managed-property>
16 </managed-bean>

```

The vote concerns entities for the `awa_question` entity table. The permission `answer-create` is used to verify that the vote is allowed.

Title: Bean types for the vote module
Date: 2013-01-12



The managed bean defines three operations that can be called: `vote_up`, `vote_down` and `vote` to setup specific ratings.

15.3 Javascript integration

The `votes` module provides a Javascript support to help users vote for items. The Javascript file `/js/awa-votes.js` must be included in the Javascript page. It is based on jQuery and ASF. The vote actions are activated on the page items as follows in XHTML facelet files:

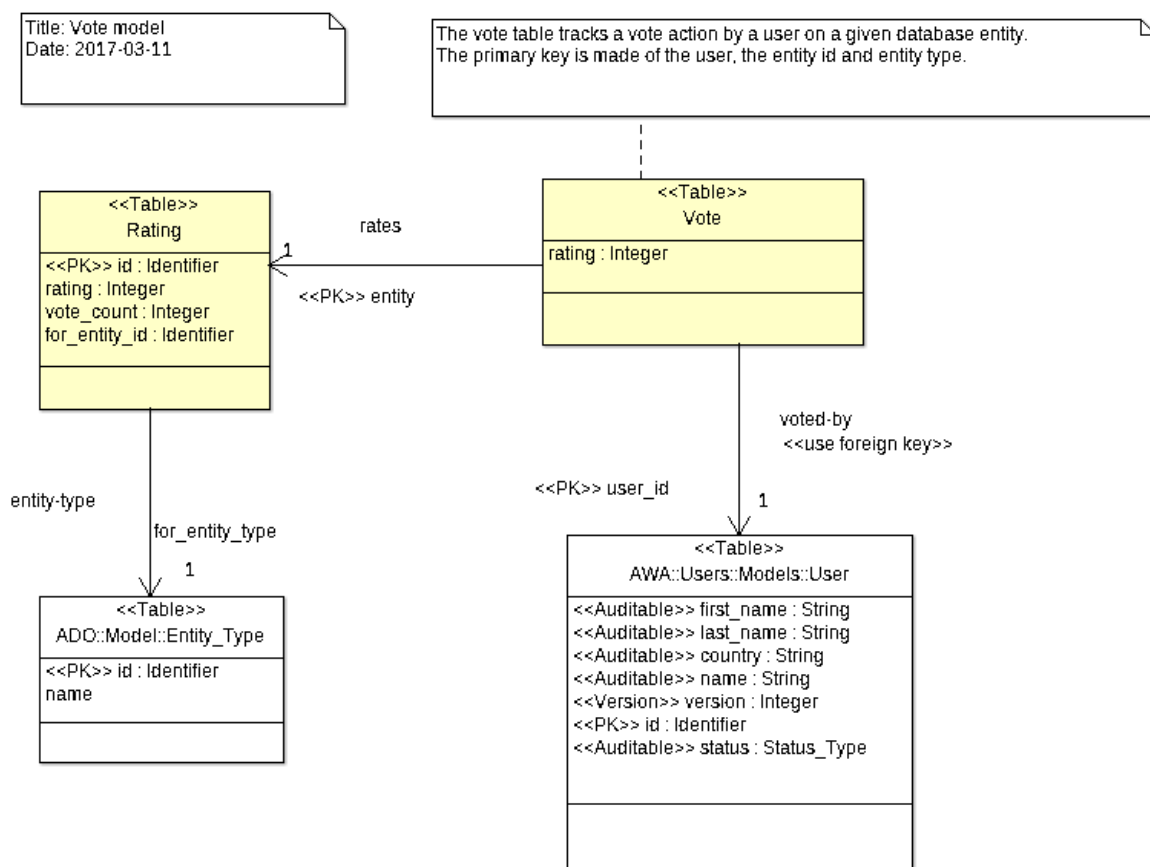
```

1 <util:script>
2   $('question-vote').votes({
3     voteUrl: "#{contextPath}/questions/ajax/questionVote/vote?id=",
4     itemPrefix: "vote_for-"
5   });
6 </util:script>

```

When the vote up or down HTML element is clicked, the `vote` operation of the managed bean `questionVote` is called. The operation will update the user's vote for the selected item (in the example "a question").

15.4 Data model



16 Tags Module

The `Tags` module allows to associate general purpose tags to any database entity. It provides a JSF component that allows to insert easily a list of tags in a page and in a form. An application can use the bean types defined in `AWA.Tags.Beans` to define the tags and it will use the `awa:tagList` component to display them. A tag cloud is also provided by the `awa:tagCloud` component.

16.1 Integration

The `Tag_Module` manages the tags associated with entities. It provides operations that are used by the tag beans together with the `awa:tagList` and `awa:tagCloud` components to manage the tags. An instance of the `Tag_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 type Application is new AWA.Applications.Application with record
2   Tag_Module : aliased AWA.Tags.Modules.Tag_Module;
3 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,
2           Name    => AWA.Tags.Modules.NAME,
3           URI     => "tags",
4           Module  => App.Tag_Module'Access);
```

16.2 Ada Beans

Several bean types are provided to represent and manage a list of tags. The tag module registers the bean constructors when it is initialized. To use them, one must declare a bean definition in the application XML configuration.

16.2.1 Tag_List_Bean

The `Tag_List_Bean` holds a list of tags and provides operations used by the `awa:tagList` component to add or remove tags within a `h:form` component. A bean can be declared and configured as follows in the XML application configuration file:

```
1 <managed-bean>
2   <managed-bean-name>questionTags</managed-bean-name>
3   <managed-bean-class>AWA.Tags.Beans.Tag_List_Bean</managed-bean-class>
4   <managed-bean-scope>request</managed-bean-scope>
```

```
5    <managed-property>
6      <property-name>entity_type</property-name>
7      <property-class>String</property-class>
8      <value>awa_question</value>
9    </managed-property>
10   <managed-property>
11     <property-name>permission</property-name>
12     <property-class>String</property-class>
13     <value>question-edit</value>
14   </managed-property>
15 </managed-bean>
```

The `entity_type` property defines the name of the database table to which the tags are assigned. The `permission` property defines the permission name that must be used to verify that the user has the permission to add or remove the tag. Such permission is verified only when the `awa:tagList` component is used within a form.

16.2.2 Tag_Search_Bean

The `Tag_Search_Bean` is dedicated to searching for tags that start with a given pattern. The auto complete feature of the `awa:tagList` component can use this bean type to look in the database for tags matching a start pattern. The declaration of the bean should define the database table to search for tags associated with a given database table. This is done in the XML configuration with the `entity_type` property.

```
1 <managed-bean>
2   <managed-bean-name>questionTagSearch</managed-bean-name>
3   <managed-bean-class>AWA.Tags.Beans.Tag_Search_Bean</managed-bean-
4     class>
5   <managed-bean-scope>request</managed-bean-scope>
6   <managed-property>
7     <property-name>entity_type</property-name>
8     <property-class>String</property-class>
9     <value>awa_question</value>
10  </managed-property>
11 </managed-bean>
```

16.2.3 Tag_Info_List_Bean

The `Tag_Info_List_Bean` holds a collection of tags with their weight. It is used by the `awa:tagCloud` component.

```
1 <managed-bean>
2   <managed-bean-name>questionTagList</managed-bean-name>
```



```

3    <managed-bean-class>AWA.Tags.Beans.Tag_Info_List_Bean</managed-bean-
      class>
4    <managed-bean-scope>request</managed-bean-scope>
5    <managed-property>
6      <property-name>entity_type</property-name>
7      <property-class>String</property-class>
8      <value>awa_question</value>
9    </managed-property>
10  </managed-bean>

```

16.2.3.1 AWA.Tags.Models.Tag_Info The tag information.

Type	Ada	Name	Description
	String	tag	the tag name.
	Natural	count	the number of references for the tag.

16.3 HTML components

16.3.1 Displaying a list of tags

The `awa:tagList` component displays a list of tags. Each tag can be rendered as a link if the `tagLink` attribute is defined. The list of tags is passed in the `value` attribute. When rendering that list, the `var` attribute is used to setup a variable with the tag value. The `tagLink` attribute is then evaluated against that variable and the result defines the link.

```

1  <awa:tagList value='{questionList.tags}' id='qtags' styleClass="
    tagedit-list"
2      tagLink="{contextPath}/questions/tagged.html?tag=#{
        tagName}"
3      var="tagName"
4      tagClass="tagedit-listelement tagedit-listelement-old"/>

```

16.3.2 Tag editing

The `awa:tagList` component allows to add or remove tags associated with a given database entity. The tag management works with the jQuery plugin Tagedit. For this, the page must include the `/js/jquery.tagedit.js` Javascript resource.

The tag edition is active only if the `awa:tagList` component is placed within an `h:form` component. The `value` attribute defines the list of tags. This must be a `Tag_List_Bean` object.

```
1 <awa:tagList value='{question.tags}' id='qtags'  
2             autoCompleteUrl='{contextPath}/questions/lists/tag-search  
                 .html' />
```

When the form is submitted and validated, the procedure `Set_Added` and `Set_Deleted` are called on the value bean with the list of tags that were added and removed. These operations are called in the `UPDATE_MODEL_VALUES` phase (ie, before calling the action's bean operation).

16.3.3 Tag cloud

The `awa:tagCloud` component displays a list of tags as a tag cloud. The tags list passed in the value attribute must inherit from the `Tag_Info_List_Bean` type which indicates for each tag the number of times it is used.

```
1 <awa:tagCloud value='{questionTagList}' id='cloud' styleClass="tag-  
    cloud"  
2             var="tagName" rows="30"  
3             tagLink="{contextPath}/questions/tagged.html?tag=#{  
                tagName}"  
4             tagClass="tag-link" />
```

16.4 Queries

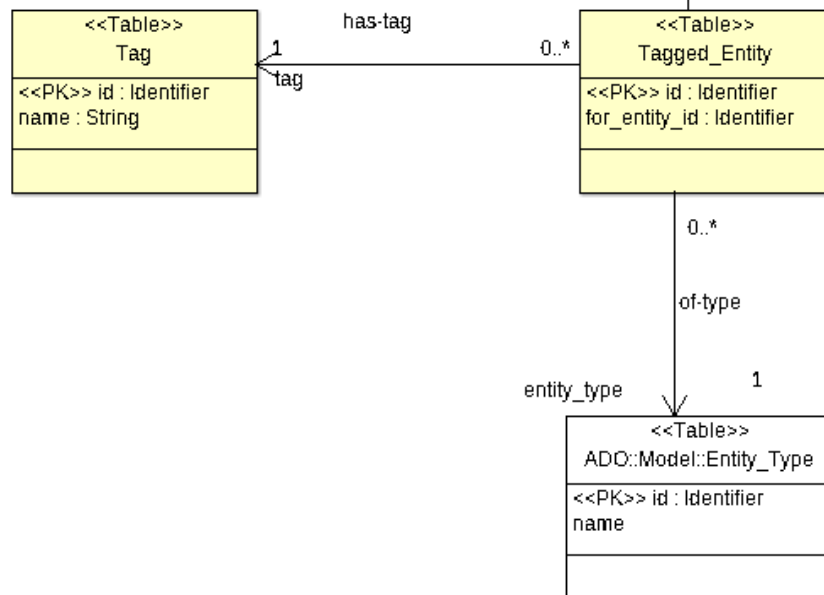
Name	Description
check-tag	Check and get the tag identifier associated with a given tag and entity
tag-list	Get the list of tags associated with a given database entity
tag-search	Get the list of tag names that match some string
tag-list-all	Get the list of tags associated with all the database entities of a given type
tag-list-for-entities	Get the list of tags associated with a set of entities of the same type.

16.5 Data model

The database model is generic and it uses the `Entity_Type` provided by Ada Database Objects to associate a tag to entities stored in different tables. The `Entity_Type` identifies the database table and the stored identifier in `for_entity_id` defines the entity in that table.

Title: Tag model
Date: 2013-02-23

The Tagged_Entity represents a tag associated with a given database entity.



17 Comments Module

The `Comments` module is a general purpose module that allows to associate user comments to any database entity. The module defines several bean types that allow to display a list of comments or edit and publish a new comment.

17.1 Integration

The `Comment_Module` manages the comments associated with entities. It provides operations that are used by the comment beans to manage the comments. An instance of the `Comment_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 type Application is new AWA.Applications.Application with record
2   Comment_Module : aliased AWA.Comments.Modules.Comment_Module;
3 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,
2           Name   => AWA.Comments.Modules.NAME,
3           URI    => "comments",
4           Module => App.Comment_Module'Access);
```

17.2 Ada Beans

Several bean types are provided to represent and manage a list of tags. The tag module registers the bean constructors when it is initialized. To use them, one must declare a bean definition in the application XML configuration.

17.2.1 Comment_List_Bean

The `Comment_List_Bean` holds a list of comments and provides operations used by the `awa:tagList` component to add or remove tags within a `h:form` component. A bean can be declared and configured as follows in the XML application configuration file:

```
1 <managed-bean>
2   <managed-bean-name>postCommentList</managed-bean-name>
3   <managed-bean-class>AWA.Comments.Beans.Comment_List_Bean</managed-
   bean-class>
4   <managed-bean-scope>request</managed-bean-scope>
5   <managed-property>
6     <property-name>entity_type</property-name>
```

```

7      <property-class>String</property-class>
8      <value>awa_post</value>
9    </managed-property>
10   <managed-property>
11     <property-name>permission</property-name>
12     <property-class>String</property-class>
13     <value>blog-comment-post</value>
14   </managed-property>
15   <managed-property>
16     <property-name>sort</property-name>
17     <property-class>String</property-class>
18     <value>oldest</value>
19   </managed-property>
20   <managed-property>
21     <property-name>status</property-name>
22     <property-class>String</property-class>
23     <value>published</value>
24   </managed-property>
25 </managed-bean>

```

The `entity_type` property defines the name of the database table to which the comments are assigned. The `permission` property defines the permission name that must be used to verify that the user has the permission to add or remove the comment.

17.2.1.1 AWA.Comments.Models.Comment_Info The comment information.

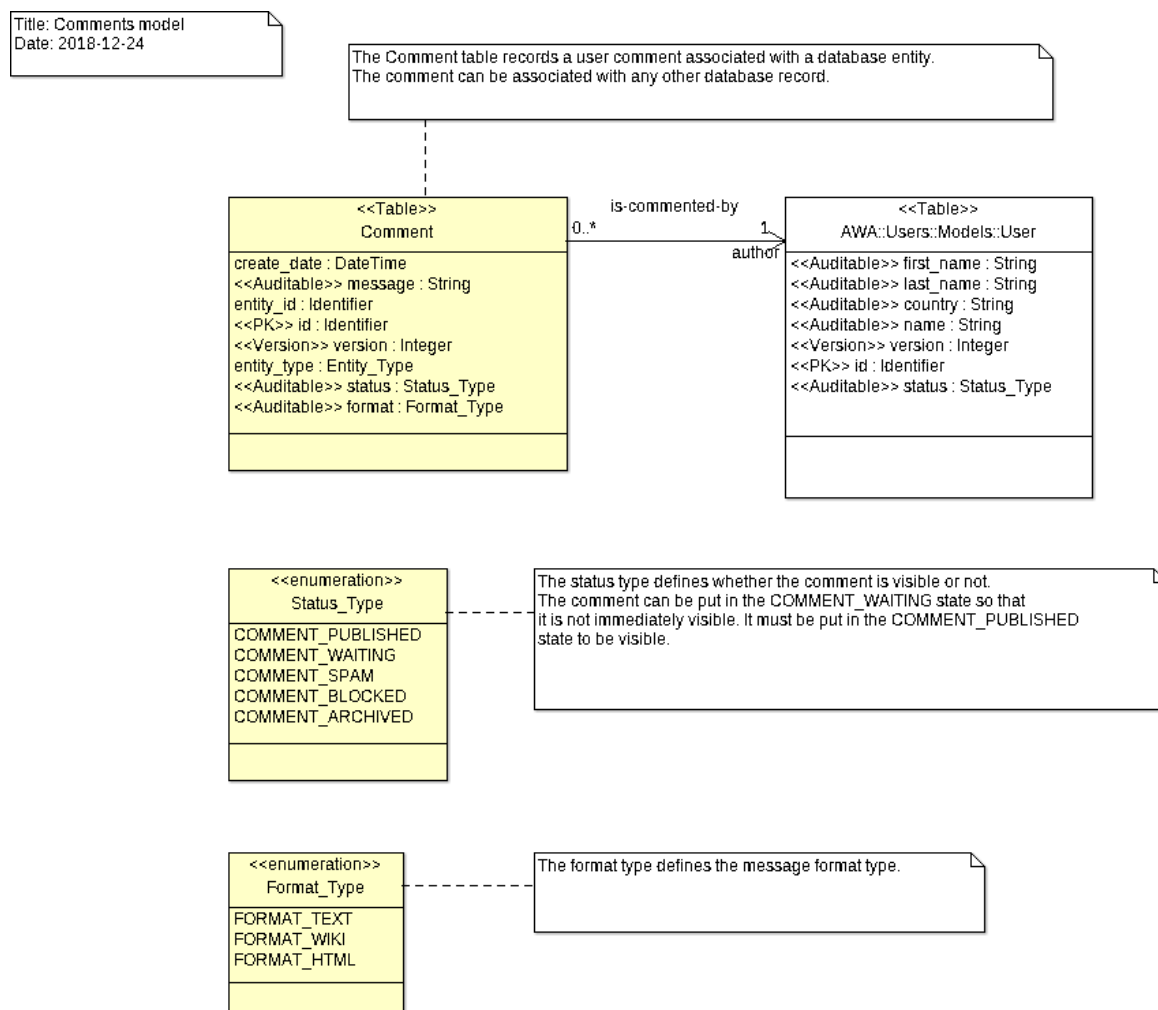
Type	Ada	Name	Description
	Identifier	id	the comment identifier.
	String	author	the comment author's name.
	String	email	the comment author's email.
	Date	date	the comment date.
	AWA.Comments.Models.Format_Type	format_type	the comment format type.
	String	comment	the comment text.
	AWA.Comments.Models.Status_Type	status	the comment status.

Name	Description
comment-list	Get the list of comments associated with given database entity
all-comment-list	Get the list of comments associated with given database entity

Name	Description
------	-------------

17.3 Data model

The database model is generic and it uses the `Entity_Type` provided by Ada Database Objects to associate a comment to entities stored in different tables. The `Entity_Type` identifies the database table and the stored identifier in `for_entity_id` defines the entity in that table.



18 Settings Module

The `Settings` module provides management of application and user settings. A setting is identified by a unique name in the application. It is saved in the database and associated with a user.

18.1 Getting a user setting

Getting a user setting is as simple as calling a function with the setting name and the default value. If the setting was modified by the user and saved in the database, the saved value will be returned. Otherwise, the default value is returned. For example, if an application defines a `row-per-page` setting to define how many rows are defined in a list, the user setting can be retrieved with:

```
1 Row_Per_Page : constant Integer := AWA.Settings.GetUser_Setting ("row-  
per-page", 10);
```

18.2 Saving a user setting

When a user changes the setting value, we just have to save it in the database. The setting value will either be updated if it exists or created.

```
1 AWA.Settings.Set_User_Setting ("row-per-page", 20);
```

18.3 Integration

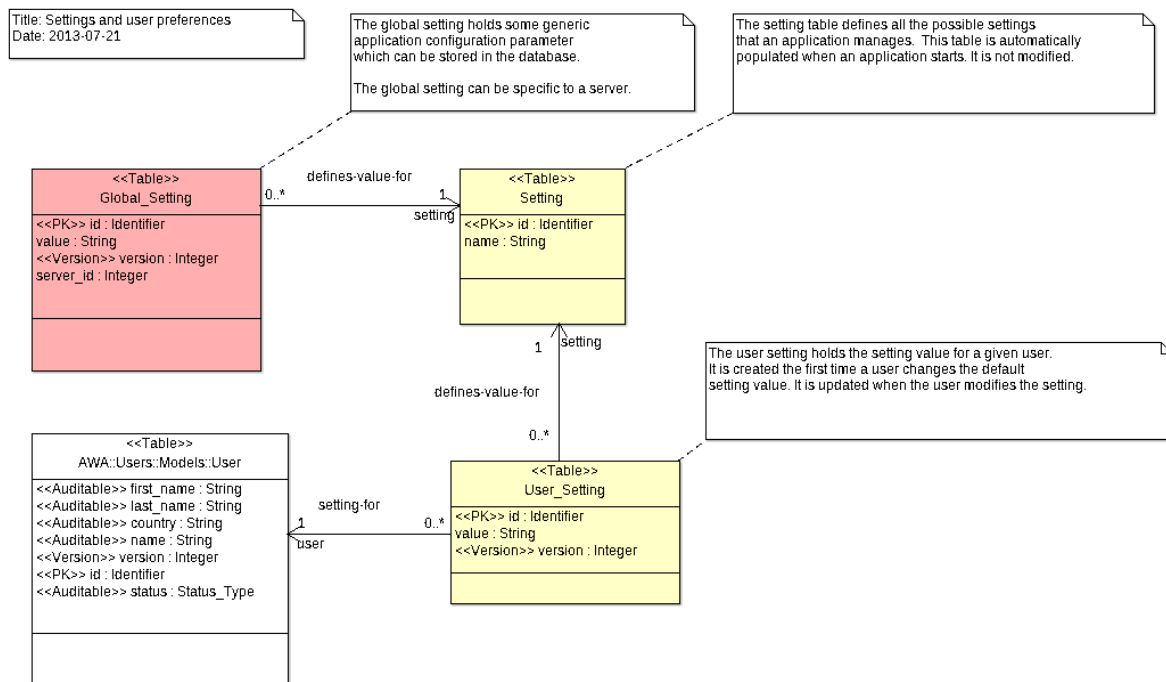
The `Setting_Module` manages the application and user settings. An instance of the `Setting_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 type Application is new AWA.Applications.Application with record  
2   Setting_Module : aliased AWA.Settings.Modules.Setting_Module;  
3 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,  
2           Name   => AWA.Settings.Modules.NAME,  
3           URI    => "settings",  
4           Module => App.Setting_Module'Access);
```

18.4 Data model



19 SEO Module

The `AWA.SEO` module provides tools for search engine optimization when publishing pages that must be discovered and scanned by search engines. It provides `sitemap` generation and allows other modules to announce what they publish for search engines in the form of a sitemap entry.

A sitemap entry is described by the `Sitemap_Entry` type and it is composed of an absolute URI location, a last modification date, a priority, an optional image URI location and an optional image title. A module that wants to provide a list of sitemap entries must implement the `Sitemap_Provider` limited interface and implement the `Create_Sitemap` procedure. That procedure must populate a `Sitemap_Info` record which contains the sitemap entries that must be exposed. For example:

```
1 with AWA.SEO;
2 ...
3 procedure Create_Sitemap (Provider : in Sitemap_Provider;
4                           Sitemap  : in out Sitemap_Info) is
5     Item : Sitemap_Entry;
6 begin
7     ...
8     Sitemap.Entries.Append (Item);
9     ...
10 end Create_Sitemap;
```

The module that implements such sitemap provider must register itself to the `SEO` module by using the `Register` procedure and giving the name of the sitemap file:

```
1 AWA.SEO.Register ("my-sitemap.xml", My_Provider'Access);
```

Such registration should be made in the `Configure` procedure of the module.

19.1 Integration

To be able to use the `SEO` module, you will need to add the following line in your GNAT project file:

```
1 with "awa_seo";
```

The `SEO_Module` type manages the registration of sitemap providers, the registration of the sitmap servlet and the generation of sitemaps. Sitemap entries are cached globally by the module in a protected object. An instance of the `SEO_Module` must be declared and registered in the AWA application. The module instance can be defined as follows:

```
1 with AWA.SEO.Modules;
2 ...
3 type Application is new AWA.Applications.Application with record
4     SEO_Module : aliased AWA.SEO.Modules.SEO_Module;
```

```
5 end record;
```

And registered in the `Initialize_Modules` procedure by using:

```
1 Register (App    => App.Self.all'Access,
2           Name   => AWA.SEO.Modules.NAME,
3           URI    => "seo",
4           Module => App.SEO_Module'Access);
```

the `SEO_Module` should be registered before a module that provides a sitemap provider.

19.2 Configuration

The `SEO` module defines the following configuration parameters:

Name	Description
seo.sitemap_prefix	The URL base prefix to be used for sitemap XML files. #{app_url_base}/sitemaps/
seo.sitemap_refresh	The cache delay in seconds before refreshing the sitemap and querying the sitemap provider. 3600

19.3 SEO Servlet

The `SEO_Servlet` type is the servlet that generates the sitemap XML files by using the sitemap providers that have been registered in the `SEO` module. By default, the `AWA.SEO` module provides a configuration to register that servlet and expose the sitemap URIs but such configuration can be overridden. The default configuration is:

```
1 <servlet-mapping>
2   <servlet-name>sitemaps</servlet-name>
3   <url-pattern>/sitemaps/*</url-pattern>
4 </servlet-mapping>
5 <filter-mapping>
6   <filter-name>service</filter-name>
7   <url-pattern>/sitemaps/*</url-pattern>
8 </filter-mapping>
```

20 Setup Application

The AWA.Setup package implements a simple setup application that allows to configure the database, the Google and Facebook application identifiers and some other configuration parameters. It is intended to help in the installation process of any AWA-based application.

It defines a specific web application that is installed in the web container for the duration of the setup. The setup application takes control over all the web requests during the lifetime of the installation. As soon as the installation is finished, the normal application is configured and installed in the web container and the user is automatically redirected to it.

20.1 Integration

To be able to use the setup application, you will need to add the following line in your GNAT project file:

```
1 with "awa_setup";
```

The setup application can be integrated as an AWA command by instantiating the `AWA.Commands.Setup` generic package. To integrate the `setup` command, you will do:

```
1 with AWA.Commands.Start;  
2 with AWA.Commands.Setup;  
3 ...  
4 package Start_Command is new AWA.Commands.Start (Server_Commands);  
5 package Setup_Command is new AWA.Commands.Setup (Start_Command);
```

20.2 Setup Procedure Instantiation

The setup process is managed by the *Configure* generic procedure. The procedure must be instantiated with the application class type and the application initialize procedure.

```
1 procedure Setup is  
2   new AWA.Setup.Applications.Configure (MyApp.Application'Class,  
3                                         MyApp.Application_Access,  
4                                         MyApp.Initialize);
```

20.3 Setup Operation

The *Setup* instantiated operation must then be called with the web container. The web container is started first and the *Setup* procedure gets as parameter the web container, the application instance to

configure, the application name and the application context path.

```
1 Setup (WS, App, "atlas", MyApp.CONTEXT_PATH)
```

The operation will install the setup application to handle the setup actions. Through the setup actions, the installer will be able to:

- Configure the database (MySQL or SQLite),
- Configure the Google+ and Facebook OAuth authentication keys,
- Configure the application name,
- Configure the mail parameters to be able to send email.

After the setup and configure is finished, the file `.initialized` is created in the application directory to indicate the application is configured. The next time the *Setup* operation is called, the installation process will be skipped.

To run again the installation, remove manually the `.initialized` file.

21 Tips

21.1 UI Presentation Tips

21.1.1 Adding a simple page

To add a new presentation page in the application, you can use the Dynamo code generator. The web page is an XHTML file created under the `web` directory. The page name can contain a directory that will be created if necessary. The new web page can be configured to use a given layout. The layout file must exist to be used. The default layout is `layout`. You can create a new layout with `add-layout` command. You can also write your layout by adding an XHTML file in the directory:

```
1 web/WEB-INF/layouts
```

To create the new web page `web/todo/list.xhtml`, you will use:

```
1 dynamo add-page todo/list
```

Depending on your application configuration and the URL used by the new page, you may have to add or modify a `url-policy`. By default, if the new URL does not match an existing `url-policy`, the access will be denied for security reasons. To allow anonymous users to access the page, use the following `url-policy`:

```
1 <url-policy>
2   <permission>anonymous</permission>
3   <url-pattern>/todo/list.html</url-pattern>
4 </url-policy>
```

and if you want only logged users, use the following:

```
1 <url-policy>
2   <permission>logged-user</permission>
3   <url-pattern>/todo/list.html</url-pattern>
4 </url-policy>
```

Note: make sure to replace the `.xhtml` extension by `.html`.

21.1.2 Add Open Graph

Use a combination of `fn:trim`, `fn:substring` and `util:escapeJavaScript` to create the Open Graph description. The first two functions will remove spaces at beginning and end of the description and will truncate the string. The `util:escapeJavaScript` is then necessary to make a value HTML attribute when the description contains special characters.

```
1 <meta property="og:description"
2   content="{util:escapeJavaScript(fn:substring(fn:trim(post.description
   ),1,128))}"/>
```

21.1.3 Formatting dates

A date can easily be formatted by following the Java Server Faces patterns by using the `f:convertDateTime` converter. The pattern attribute controls how the date is formatted and it takes into account the locale used to render the request. For example:

```
1 <h:outputText value="{event.member.payment_date}">
2   <f:convertDateTime pattern="%A %d %B %Y"/>
3 </h:outputText>
```

Sometimes, the date must be formatted within an attribute of some HTML element. In that case, the `h:outputText` cannot be used but instead we can use the `util:formatDate` EL function. For example:

```
1 date="{util:formatDate(post.date, '%Y-%m-%d')}"
```

21.2 Configuration Tips

21.2.1 Adding a permission on user creation

Sometimes it is useful to add some permission when a user is created. This can be done programatically but also through some simple configuration. By using the `on-event` XML definition, it is possible to create a set of permissions when the `user-create` event is posted, hence during user creation.

The following XML extract from Atlas demonstrator will add several Wiki permissions to the new user.

```
1 <on-event name="user-create">
2   <action>#{permission.create}</action>
3   <property name="entity_type">awa_wiki_space</property>
4   <property name="entity_id">1</property>
5   <property name="workspace_id">1</property>
6   <property name="permission">wiki-page-create, wiki-page-update, wiki-
   page-delete, wiki-page-view, wiki-space-delete, wiki-space-update</
   property>
7 </on-event>
```

21.2.2 Secure configuration

Setting up the secure configuration is made by using a secure keystore with the `akt` tool. The secure configuration is stored in the keystore file that `akt` will protect by encrypting each configuration property with their own encryption key. In order to setup and use the secure configuration the following steps are necessary:

- Create the keystore file and protect it with a password or a GPG key,
- Populate the keystore file with the configuration values,
- Launch the server with specific options in order to use and access the keystore file.

The two initial steps are done by using the `akt` tool.

To create the keystore file, one way is to run the `akt` command and give it the keystore password:

```
1 mkdir secure
2 akt create --wallet-key-file=secure/wallet.key -c 100000:300000 \
3   secure/config.akt
```

This will generate the `secure/wallet.key` file and setup the keystore file in `secure/config.akt`. The password must be given to the server when it is started. To avoid that, it is possible to store it in a file and make sure the file is protected against read and write access. If the password is stored in such file, the keystore is created by using:

```
1 akt create --wallet-key-file=secure/wallet.key \
2   --passfile=secure/master.key -c 100000:300000 \
3   secure/config.akt
```

Once the keystore is created, the configuration are inserted. Because the server command can use only one keystore and have several applications, the configuration parameter must be prefixed by the application name. For example, to setup the `database` configuration for the `atlas` application, you will use the command:

```
1 akt set --wallet-key-file=secure/wallet.key \
2   --passfile=secure/master.key \
3   secure/config.akt \
4   atlas.database 'mysql://localhost:3306/atlas?user=atlas&password=
    PiX2ShaimohW6eno'
```

To avoid having to specify several configuration parameters when launching the server, it is good practice to create a server global configuration file and indicate several parameters that the server will use. Create a file `secure/config.properties` that contains:

```
1 keystore-path=secure/config.akt
2 keystore-masterkey-path=secure/master.key
3 keystore-password-path=secure/password.key
```

Then, to start the server we just need to give it the server global configuration path:

```
1 bin/atlas-server -c secure/config.properties start
```

Note that in order to use this configuration setup, the directory must have the `rwX-----` rights and files must have the `rw-----` rights.

21.2.3 Overriding some AWA navigation rules

AWA and its modules are using Ada Server Faces for the presentation pages and navigation. The navigation rule describes what is the next page to render when a form is submitted. In some cases, the default navigation rules provided by AWA or some of its module may not fit your application needs. You can override them by creating an application override XML file that you configure to load after the `awa.xml` configuration file. For example, to override the default redirection used after a login, you can use the following XML file:

```
1 <?xml version='1.0' encoding='UTF-8'?>
2 <module version="1.0">
3
4   <navigation-rule>
5     <from-view-id>/auth/login.xhtml</from-view-id>
6     <navigation-case>
7       <from-outcome>success</from-outcome>
8       <to-view-id>/index.xhtml</to-view-id>
9       <redirect/>
10    </navigation-case>
11  </navigation-rule>
12
13 </module>
```

and then configure the `app.config` property to define the list of XML configuration file to load. Each name must be separated by `;`. For example:

```
1 app.config=awa.xml;app-override.xml
```

21.3 Trouble shooting Tips

21.3.1 No AWA service context

When the AWA framework emits the following error:

```
1 ERROR - AWA.Services.Contexts - : No AWA service context: may be a '
    filter-mapping' is missing to activate the 'service' filter in the
    request path
```


it is often followed by a `Constraint Error` such as:

```
1 CONSTRAINT_ERROR: awa-services-contexts.adb:53 access check failed
```

and it is caused by an application that uses the `AWA.Services.Contexts.Current` operation on an incoming request but there is no AWA service context. In most cases, the root cause is that a servlet filter is missing in the configuration for the current URL request. You may add such servlet filter by using the configuration:

```
1 <filter-mapping>
2   <filter-name>service</filter-name>
3   <url-pattern>/*.*html</url-pattern>
4 </filter-mapping>
```

and replace the `/*.*html` pattern by the URL that caused the error.